

算法刷题与 C++ 面试教材

数据结构、算法设计、复杂度证明与工程化题解

从暴力枚举到优化推理，从容器原理到面试表达

面向长期能力建设的完整学习稿

前言

这本书的目标不是堆题解，也不是把学习压缩进某个固定周期，而是把数据结构与算法训练成一套可以长期迁移的工程能力。面试题看似零散，背后其实围绕两条主线：一条是数据结构（data structure），也就是信息怎样存、怎样查、怎样更新；另一条是算法（algorithm），也就是在约束下怎样组织计算、怎样减少重复工作、怎样证明结果正确。数据结构和算法不能分开学。只会说“这题用滑动窗口”，但不知道窗口状态应该放在数组、哈希表还是队列里，代码一定会不稳；只会背 `unordered_map`，但不知道它解决的是历史查询问题，也很难在新题里想到它。

本书按“知识完整性”组织，而不是按倒计时组织。前面的章节先建立复杂度、暴力法、复盘方法和 C++ 代码标准；中间把数组、字符串、链表、栈、队列、哈希表、堆、树、图和并查集讲成数据结构主线；后面再进入双指针、滑动窗口、前缀和、二分、搜索、动态规划、回溯、贪心和综合题。训练周期可以按短期集中训练、学期式推进或长期滚动复习来调整，但正文必须把原理、背景、案例、代码、证明、边界和复盘方法讲完整。

每一类算法都必须从暴力方法开始。暴力方法不是低级答案，而是理解问题空间的方式。两数之和先枚举下标对，才能看出慢点是反复查找补数；最短子数组先枚举所有区间，才能看出正数数组存在窗口单调性；岛屿数量先把每个格子当成节点，才能把题目翻译成图的连通块。优化不是凭感觉，而是来自重复工作、单调性、可维护状态、候选淘汰、最优子结构和数据结构查询能力。

代码使用 C++20。示例代码优先使用 `const` 引用、明确边界、清楚的变量名和可测试函数。书里会解释 `vector`、`string`、`deque`、`stack`、`queue`、`priority_queue`、`unordered_map`、`set`、树、图和并查集的用法与底层复杂度，也会说明迭代器失效、哈希退化、扩容、拷贝、溢出和递归栈这些真实面试和线上评测里会遇到的问题。

读法

读这本书时，不要只看最终代码。每道题至少走四遍：第一遍只读题并写暴力思路；第二遍找慢点和可优化性质；第三遍写 C++ 优化代码；第四遍复盘边界、复杂度和面试表达。

每天训练时，把题目记录成固定格式：题型、输入规模、暴力枚举对象、优化来源、不变量、使用的数据结构、复杂度、错因和一周后是否能独立重写。只写“没想到”没有意义，必须写成具体原因：没有发现单调性、哈希表插入顺序错、窗口收缩条件错、二分边界没有排除 `mid`、`int` 溢出、递归深度过大、容器选择导致复杂度不对。

本书目录只显示部和章，小节不进入目录。章内标题用于串联正文，不把内容切碎成大量编号。你应该把每章当成一条推理链来读，而不是把每个小标题当成孤立知识点。

目录

第一部分	学习路线：从基础到综合能力	
第 1 章	刷题方法：从题目到可复用能力	2
第 2 章	训练路线：从基础题到综合能力	3
第 3 章	复杂度、暴力法和优化来源	4
第二部分	数据结构：算法能力的骨架	
第 4 章	线性数据结构：数组、字符串、链表、栈和队列	6
第 5 章	非线性数据结构：哈希表、堆、树、图和并查集	11
第三部分	核心算法：从暴力到优化	
第 6 章	数组、双指针和滑动窗口	18
第 7 章	哈希表和前缀和	23
第 8 章	二分查找和答案空间	26
第 9 章	树、图、BFS 和 DFS	29
第 10 章	回溯、枚举和搜索空间剪枝	36
第 11 章	动态规划：状态、转移和重复子问题	42
第四部分	面试专项：从题型到工程表达	
第 12 章	线性结构专项：链表、栈、队列和单调结构	47
第 13 章	排序、选择和区间问题	58
第 14 章	堆、Top K 和贪心证明	68
第 15 章	图论进阶：并查集、最短路和状态图	79
第 16 章	动态规划进阶、容器原理和源码阅读	93
附录 A	力扣刷题手册	107

第零部分

学习路线：从基础到综合能力

第 1 章

刷题方法：从题目到可复用能力

刷题和面试训练的核心不是“见过多少题”，而是能否把陌生题拆成熟悉结构。题目只是外壳，真正要识别的是数据怎么组织、状态怎么变化、答案依赖哪些历史信息、是否存在单调性、是否能淘汰候选、是否有重叠子问题。

面对一道题，第一步不是想模板，而是读约束。输入规模决定复杂度上限，值域决定能不能用计数数组或位图，有序性决定能不能双指针或二分，是否允许重复决定哈希表和去重逻辑，是否有负数决定滑动窗口和前缀和的选择。很多错误题解不是算法错，而是约束没看清。

第二步写暴力方法。暴力方法要明确枚举对象：枚举元素、下标对、子数组、子序列、路径、排列、状态，还是答案值。枚举对象一旦明确，复杂度就能估出来。比如两数之和枚举下标对是 $O(n^2)$ ；所有子数组是 $O(n^2)$ ；所有排列是 $O(n!)$ ；网格 BFS 扫所有格子是 $O(mn)$ 。

第三步找重复工作。重复工作有几种典型形态：反复查询某个历史值，适合哈希表；反复求区间和，适合前缀和；反复维护窗口内统计，适合滑动窗口；反复找当前最大或最小，适合堆或单调队列；反复搜索同一状态，适合记忆化或动态规划。

面试表达原则

不要直接说“这题用哈希表”。更好的表达是：暴力会枚举所有下标对，时间是 $O(n^2)$ ；慢点在于每个数都要反复查找另一个数是否存在；我从左到右扫描，用哈希表保存已经看到的值到下标的映射；处理当前值时查 `target - nums[i]` 是否出现过；这样每个元素只插入和查询一次，平均时间 $O(n)$ ，空间 $O(n)$ 。

第四步写不变量。不变量是优化算法正确性的核心。滑动窗口的不变量可能是“窗口内没有重复字符”；单调栈的不变量可能是“栈内下标对应的值单调递减”；BFS 的不变量是“当前层所有节点距离源点相同”；二分的不变量是“答案一定在当前左右边界之间”。没有不变量，代码只是碰巧通过样例。

第五步落到 C++。C++ 代码要讲容器选择。`vector` 适合连续存储和下标访问；`unordered_map` 适合平均常数查询历史信息；`deque` 适合两端弹入弹出；`priority_queue` 适合动态极值但不能删除任意元素；`set` 适合有序前驱后继；图的邻接表通常用 `vector<vector<int>>`，稀疏图比邻接矩阵更节省空间。

Listing 1.1 两数之和：暴力与哈希优化的共同接口

```
std::vector<int> two_sum_hash_table(const std::vector<int>& nums, int target)
{
    std::unordered_map<int, int> value_to_index;
    value_to_index.reserve(nums.size());

    for (int index = 0; index < static_cast<int>(nums.size()); ++index) {
        const int need = target - nums[index];
        const auto found = value_to_index.find(need);
        if (found != value_to_index.end()) {
            return {found->second, index};
        }
        value_to_index.emplace(nums[index], index);
    }
    return {};
}
```

这段代码有几个细节。参数用 `const std::vector<int>&`，避免复制输入数组。哈希表先查再插入，避免同一个元素和自己配对。`reserve` 不是必须，但能减少扩容和 rehash。返回空数组表示没有答案，真实力扣题如果保证有答案，也可以直接返回。

第 2 章

训练路线：从基础题到综合能力

学习路线不是倒计时。有人一个月集中准备，有人一边工作一边长期训练，也有人先补 C++ 和数据结构基础，再进入高强度题目练习。因此本章只给出能力阶段和训练节奏，不把整本书绑定到固定月份。你可以把每个阶段压缩成一周，也可以展开成一个月；真正不能省略的是每个阶段要掌握的知识、题型、代码能力和复盘方法。

第一阶段建立基础语言和复杂度意识。目标是能读懂输入规模，估计可接受复杂度，写出暴力解，并说清楚暴力解慢在哪里。这个阶段学习数组、字符串、`vector`、`string`、排序、二分库函数、下标边界、引用传参和整数溢出。推荐题包括两数之和、移动零、删除有序数组重复项、合并两个有序数组、最大子数组和、反转字符串和有效回文。做题时不要急着背模板，先把枚举对象写清楚：枚举元素、下标对、区间，还是答案边界。

第二阶段学习线性结构和窗口类算法。双指针、滑动窗口、前缀和、单调栈和单调队列看起来像技巧，本质都是“线性序列上的状态维护”。窗口题要写出左右边界的含义，说明什么时候扩张、什么时候收缩；单调结构题要说明哪些候选被淘汰，为什么淘汰后不会丢答案。推荐题包括盛最多水的容器、三数之和、长度最小的子数组、无重复字符的最长子串、找到字符串中所有字母异位词、最小覆盖子串、每日温度、柱状图最大矩形和滑动窗口最大值。

第三阶段把哈希、堆、树和图串起来。哈希表负责历史查询，堆负责动态极值，树负责层次和有序关系，图负责任意状态转移。这个阶段要形成建模习惯：节点是什么，边是什么，状态保存在哪里，访问标记何时更新，答案在入队、出队、入栈、出栈还是回溯时产生。推荐题包括字母异位词分组、最长连续序列、和为 K 的子数组、前 K 个高频元素、数组中的第 K 个最大元素、二叉树层序遍历、验证二叉搜索树、最近公共祖先、岛屿数量、腐烂的橘子、课程表和课程表 II。

第四阶段进入搜索、回溯和动态规划。回溯题要写清选择列表、路径状态、终止条件和撤销动作；动态规划题要写清状态定义、转移来源、初始化、遍历顺序和空间优化条件。不要先背方程，先问一个状态是否足够描述未来。推荐题包括全排列、子集、组合总和、括号生成、单词搜索、爬楼梯、打家劫舍、零钱兑换、最长递增子序列、最长公共子序列、编辑距离、分割等和子集和目标。

第五阶段处理贪心、并查集和综合题。贪心题的关键是证明局部选择为什么能推出全局最优；并查集题的关键是把动态连通关系建模成集合合并；综合题则常常把哈希、排序、堆、图和 DP 混在一起。推荐题包括跳跃游戏、加油站、合并区间、冗余连接、账户合并、实现 Trie、网络连接、单词接龙和数据流中位数。这个阶段不要只追求 AC，要训练面试表达：暴力解是什么，瓶颈在哪里，优化依赖哪条性质，代码如何保证边界正确。

如果需要集中训练，可以把这些阶段压成十二周：基础复杂度与数组字符串，双指针与窗口，哈希与前缀和，栈队列堆与单调结构，二分查找，链表与树，图和拓扑排序，回溯，一维动态规划，二维动态规划和背包，贪心与并查集，综合模拟与查漏补缺。这只是节奏表，不是本书的边界。书的目标是把知识完整讲清楚，训练周期由你的基础、时间和目标岗位决定。

阶段交付

每个阶段至少留下四类材料：一组代表题的暴力解和优化解，一份数据结构选择说明，一份 C++ 容器和边界错误清单，一份一周后能独立重写的回访题表。刷题报告不是形式，它是把题解转化成长期能力的工具。

第 3 章

复杂度、暴力法和优化来源

复杂度是算法设计的预算。输入规模如果是 $n \leq 100$, $O(n^3)$ 可能可以接受; 如果是 $n \leq 2 \times 10^5$, 通常要接近 $O(n \log n)$ 或 $O(n)$; 如果网格规模是 $m * n \leq 10^5$, BFS 或 DFS 扫一遍通常合理; 如果状态空间是指数级, 就要考虑剪枝、记忆化或动态规划。

暴力法的作用是把问题完整展开。两数之和暴力枚举所有下标对, 最长无重复子串暴力枚举所有起点和终点, 岛屿数量暴力从每个未访问陆地扩展连通块。暴力方法慢, 但它通常是正确性的起点。

Listing 3.1 两数之和暴力解

```
std::vector<int> two_sum_bruteforce(const std::vector<int>& nums, int target)
{
    for (int left = 0; left < static_cast<int>(nums.size()); ++left) {
        for (int right = left + 1; right < static_cast<int>(nums.size()); ++right) {
            if (nums[left] + nums[right] == target) {
                return {left, right};
            }
        }
    }
    return {};
}
```

这段代码枚举的是下标对, 数量约为 $n * (n - 1) / 2$, 时间复杂度是 $O(n^2)$ 。它慢在每个数都要重新扫描后面的数。优化思路不是凭空出现, 而是把“扫描后面所有数”改成“快速查询某个补数是否出现过”。这就是哈希表。

常见优化来源有六类。第一类是缓存重复计算, 例如前缀和、记忆化搜索和动态规划。第二类是单调性, 例如二分、双指针和某些滑动窗口。第三类是局部状态可维护, 例如字符计数、窗口和、不同元素个数。第四类是候选淘汰, 例如单调栈和单调队列。第五类是数据结构查询能力, 例如哈希表、堆、平衡树、并查集。第六类是改变建模方式, 例如把网格问题建成图, 把区间问题排序, 把字符串问题转为频次。

写完暴力法后, 应该固定问几个问题: 我枚举的对象是什么? 有没有重复计算同一段信息? 是否只需要历史信息的一小部分? 是否存在有序或单调? 是否可以把区间查询变成前缀差? 是否可以用哈希把线性查询变成平均常数? 是否可以把指数搜索合并成状态 DP?

面试时要把这些问题讲出来。一个好答案通常不是从“我用某某模板”开始, 而是从暴力法、瓶颈和优化依据开始。

第零部分

数据结构：算法能力的骨架

第 4 章

线性数据结构：数组、字符串、链表、栈和队列

数据结构不是算法之前的一张 API 表，而是算法能够成立的物理基础。刷题时遇到一个问题，第一步不是急着套模板，而是判断答案依赖什么信息：需要按位置访问、按最近关系回退、按先进先出扩展，还是只在两端维护候选。数组、字符串、链表、栈、队列和双端队列构成了这条主线。它们的抽象接口很简单，但背后的存储方式、缓存局部性、迭代器失效规则和边界条件，决定了暴力方法能不能被优化成面试可接受的复杂度。

线性结构的共同特点是元素被组织成一条序列。区别在于这条序列是否连续存储，是否支持下标随机访问，插入删除发生在什么位置，算法状态如何随着左右边界移动而更新。真正掌握这一章，意味着看到题目时可以回答四个问题：序列是否固定，窗口是否连续，历史状态能否增量维护，删除或回退是否只发生在某一端。

从能力边界看，`vector` 负责连续存储、随机访问和顺序扫描，是 DP 表、排序数组、邻接表和双指针题的默认选择；`string` 负责字符序列和子串窗口，但要警惕 `substr` 的复制成本；链表负责节点重连，适合反转、合并、环检测和快慢指针；`stack` 负责最近未解决状态，常用于括号、表达式、路径和单调栈；`queue` 负责按层扩展，是 BFS 的自然结构；`deque` 负责两端维护候选，尤其适合滑动窗口极值。把这些结构先放在同一张能力地图里，后面每个算法就不会变成孤立模板。

数组和 `std::vector` 是刷题默认容器。连续存储带来两个关键优势：下标访问是 $O(1)$ ，相邻元素通常落在相近内存位置，CPU 缓存命中率高。很多初学者只记住第一个优势，忽略第二个优势。实际程序中，同样是线性扫描，`vector` 往往比链表快得多，并不是因为复杂度写法不同，而是因为连续内存更容易被硬件预取。代价也来自连续存储：中间插入和删除必须移动后续元素；容量不足时扩容会重新申请一段更大的空间，把旧元素移动或复制过去，原来的指针、引用和迭代器都可能失效。

Listing 4.1 `vector` 的容量、大小和失效规则

```
std::vector<int> nums;
nums.reserve(n);      // 只预留容量，不构造元素，size 仍然是 0。
nums.push_back(10);
nums.emplace_back(20);

std::vector<int> dp(n + 1, 0); // 直接创建 n + 1 个元素。

const int* first = nums.data();
nums.push_back(30);          // 如果触发扩容，first 立刻失效。
```

`reserve` 和 `resize` 必须分清。`reserve` 只改变容量，不构造元素，适合已经知道将要追加多少元素的场景；`resize` 改变大小，会构造或销毁元素，适合直接创建 DP 数组、计数数组或矩阵行。写 `vector<int> dp(n + 1)` 后，`dp[i]` 是合法的；只写 `reserve(n + 1)` 后访问 `dp[i]` 是越界。面试里这类错误很隐蔽，因为代码可能在小样例上运行，到了评测环境才崩溃。

数组题的暴力方法通常是枚举一个区间、一个配对或一个分割点。优化的关键，是让“下一步答案”复用“上一步状态”。例如区间和暴力做法是每次重新累加 `nums[left..right]`，复杂度 $O(n^3)$ 或 $O(n^2)$ ；前缀和把任意区间和转成两个端点差值，使查询变成 $O(1)$ 。滑动窗口则在区间右端加入一个元素、左端删除一个元素，用增量更新维护窗口状态。它们都依赖数组按下标稳定访问。

Listing 4.2 从重新求和到前缀和查询

```
std::vector<long long> build_prefix_sum(const std::vector<int>& nums)
{
    std::vector<long long> prefix(nums.size() + 1, 0);
    for (std::size_t i = 0; i < nums.size(); ++i) {
        prefix[i + 1] = prefix[i] + nums[i];
    }
    return prefix;
}
```

```

long long range_sum(const std::vector<long long>& prefix,
                    std::size_t left,
                    std::size_t right)
{
    return prefix[right + 1] - prefix[left];
}

```

这段代码里 `prefix[i]` 表示前 `i` 个元素之和，因此区间 `[left, right]` 的和是 `prefix[right + 1] - prefix[left]`。多出来的第 0 位看似绕，但它消除了 `left == 0` 的特殊判断。很多高质量题解都会主动设计这种哨兵位，因为它能让边界条件服从同一条公式。

字符串 `std::string` 可以看成字符数组，但不能只按字符数组理解。第一，`substr` 会创建新字符串，频繁调用会引入大量复制；第二，`size()` 返回字节数，不等于所有自然语言里的“字符数”；第三，题目如果限定小写英文，计数结构可以用 `std::array<int, 26>`，如果字符集不固定，就应使用哈希表。刷题常见输入大多是 ASCII 或小写英文，面试时仍然要主动说明自己的假设。

Listing 4.3 异位词窗口：避免反复 `substr`

```

std::vector<int> find_anagrams(const std::string& s, const std::string& p)
{
    constexpr int ALPHABET_SIZE = 26;
    std::vector<int> answer;
    if (s.size() < p.size()) {
        return answer;
    }

    std::array<int, ALPHABET_SIZE> need{};
    std::array<int, ALPHABET_SIZE> window{};
    for (const char ch : p) {
        ++need[ch - 'a'];
    }

    for (std::size_t right = 0; right < s.size(); ++right) {
        ++window[s[right] - 'a'];
        if (right >= p.size()) {
            --window[s[right - p.size()] - 'a'];
        }
        if (window == need) {
            answer.push_back(static_cast<int>(right + 1 - p.size()));
        }
    }
    return answer;
}

```

暴力版本会枚举每个长度为 `p.size()` 的子串，复制出来再排序或统计。复制子串本身就是 $O(k)$ ，排序还要 $O(k \log k)$ 。窗口版本只维护 26 个计数，右端进入一个字符，左端离开一个字符，每步常数更新。优化成立的原因不是“用了滑动窗口”这个名词，而是题目只关心固定长度窗口内的字符频次，窗口相邻状态只差两个字符。

链表是线性结构里最容易被误解的一个。教材常说链表插入删除是 $O(1)$ ，但这句话有前提：你已经拿到了要操作位置的前驱节点或当前节点。如果还需要从头查找位置，查找本身就是 $O(n)$ 。链表的价值不在于比数组更快，而在于节点位置稳定、局部重连便宜、结构可以自然表达“下一个”。面试链表题考的不是容器 API，而是指针不丢、边界不漏、连接顺序清楚。

Listing 4.4 迭代反转链表：先保存后继，再改变指向

```

struct ListNode {
    int value = 0;
    ListNode* next = nullptr;
};

ListNode* reverse_list(ListNode* head)
{
    ListNode* previous = nullptr;
    ListNode* current = head;

    while (current != nullptr) {
        ListNode* next = current->next;

```

```

        current->next = previous;
        previous = current;
        current = next;
    }

    return previous;
}

```

这段代码只有四行核心操作，顺序却不能乱。先保存 `next`，因为一旦执行 `current->next = previous`，原来的后继就找不到了；再把当前节点接到已反转部分前面；最后整体向后推进。写链表题时，建议在纸上画三个指针：已经处理好的部分、当前节点、尚未处理的部分。只要这三段关系清楚，反转、两两交换、K 个一组反转都可以从同一套指针动作推出来。

虚拟头节点是链表题的另一个重要技巧。删除倒数第 N 个节点、合并两个链表、分隔链表时，真实头节点可能被删除或替换。如果每次都单独判断头节点，代码会被边界分支切碎。虚拟头节点把“修改头节点”和“修改普通节点”统一成“修改某个前驱节点的 `next`”。

Listing 4.5 虚拟头节点统一删除逻辑

```

ListNode* remove_nth_from_end(ListNode* head, int n)
{
    ListNode dummy;
    dummy.next = head;

    ListNode* fast = &dummy;
    ListNode* slow = &dummy;

    for (int step = 0; step < n; ++step) {
        fast = fast->next;
    }
    while (fast->next != nullptr) {
        fast = fast->next;
        slow = slow->next;
    }

    ListNode* removed = slow->next;
    slow->next = removed->next;
    return dummy.next;
}

```

快慢指针的本质是让两个指针之间保持固定距离或固定速度差。删除倒数第 N 个节点时，快指针先走 N 步，之后快慢一起走；当快指针到尾部，慢指针正好停在待删节点前驱。环检测时，快指针每次走两步，慢指针每次走一步；如果存在环，速度差会让它们在环内相遇。不要把快慢指针背成模板，应该说清楚“两个指针之间的不变量是什么”。

栈是后进先出结构，适合处理“最近出现、最先解决”的关系。括号匹配中，最近的左括号必须先被右括号匹配；路径简化中，最近进入的目录会被 `..` 回退；表达式求值中，最近的运算符和操作数构成局部归约。栈的强大之处在于把嵌套结构压平成一条处理过程。

Listing 4.6 有效括号：最近未匹配结构

```

bool is_valid_parentheses(const std::string& s)
{
    std::vector<char> stack;
    stack.reserve(s.size());

    for (const char ch : s) {
        if (ch == '(' || ch == '[' || ch == '{') {
            stack.push_back(ch);
            continue;
        }
        if (stack.empty()) {
            return false;
        }

        const char left = stack.back();
        stack.pop_back();
        const bool matched = (left == '(' && ch == ')') ||
                               (left == '[' && ch == ']') ||
                               (left == '{' && ch == '}');
    }
}

```

```

        (left == '{' && ch == '}');

        if (!matched) {
            return false;
        }
    }

    return stack.empty();
}

```

这里直接使用 `std::vector<char>` 作为栈，比 `std::stack<char>` 更方便查看和预留容量。`std::stack` 是容器适配器，只暴露 `push`、`pop`、`top`、`empty` 等接口；`pop()` 不返回元素，因此必须先 `top()` 再 `pop()`。面试写代码时两者都可以，但要明白适配器牺牲了一部分访问能力，换来更明确的抽象边界。

单调栈是栈在刷题中的高频变体。它维护的不是时间顺序本身，而是一组“尚未找到答案的候选”。每日温度中，栈保存还没有找到更高温度的下标，并且这些下标对应的温度从栈底到栈顶单调不减。新温度到来时，只要它高于栈顶下标的温度，就可以确定栈顶答案；因为这是栈顶下标右侧第一个更高温度的下标。

Listing 4.7 每日温度：单调栈保存下标

```

std::vector<int> daily_temperatures(const std::vector<int>& temperatures)
{
    std::vector<int> answer(temperatures.size(), 0);
    std::vector<int> stack;
    stack.reserve(temperatures.size());

    for (int day = 0; day < static_cast<int>(temperatures.size()); ++day) {
        while (!stack.empty() && temperatures[day] > temperatures[stack.back()]) {
            const int previous_day = stack.back();
            stack.pop_back();
            answer[previous_day] = day - previous_day;
        }
        stack.push_back(day);
    }

    return answer;
}

```

单调栈常保存下标而不是值，因为题目经常需要距离、宽度，或者需要回到原数组访问更多信息。它能把暴力枚举“右侧第一个更大元素”的 $O(n^2)$ 降到 $O(n)$ ，原因是每个下标最多入栈一次、出栈一次。这个摊还分析会说：虽然某一次循环里可能弹出很多元素，但全局弹出次数不超过 n 。

队列是先进先出结构，适合按层扩展。BFS 的正确性来自队列顺序：先入队的是距离更短或时间更早的状态，因此第一次到达一个未访问状态时，就已经得到了最短步数。二叉树层序遍历、腐烂橘子、多源最短扩散、无权图最短路，都依赖这个性质。与栈不同，队列不会深挖某一条路径，而是把同一层的状态先处理完。

Listing 4.8 层序遍历：队列保存当前边界

```

std::vector<std::vector<int>> level_order(TreeNode* root)
{
    std::vector<std::vector<int>> levels;
    if (root == nullptr) {
        return levels;
    }

    std::queue<TreeNode*> queue;
    queue.push(root);

    while (!queue.empty()) {
        const int level_size = static_cast<int>(queue.size());
        std::vector<int> level;
        level.reserve(level_size);

        for (int i = 0; i < level_size; ++i) {
            TreeNode* node = queue.front();
            queue.pop();
            level.push_back(node->value);

            if (node->left) queue.push(node->left);
            if (node->right) queue.push(node->right);
        }

        levels.push_back(level);
    }

    return levels;
}

```

```

        if (node->left != nullptr) {
            queue.push(node->left);
        }
        if (node->right != nullptr) {
            queue.push(node->right);
        }
    }
    levels.push_back(std::move(level));
}

return levels;
}

```

这段代码里的 `level_size` 是关键。没有它，也可以 BFS，但很难知道哪些节点属于同一层。层序遍历、最短时间、多源扩散都需要把“当前层的节点数量”固定下来，然后处理完这一层再增加距离或分钟数。

`std::deque` 支持两端 $O(1)$ 插入删除，是 `std::queue` 的默认底层容器，也是单调队列的常用实现。滑动窗口最大值的暴力方法是每个窗口扫描一遍最大值，复杂度 $O(nk)$ 。如果用堆，需要处理过期下标。单调队列更贴合题意：队头永远是当前窗口最大值下标，队尾负责淘汰不可能再成为最大值的候选。

Listing 4.9 滑动窗口最大值：单调队列

```

std::vector<int> max_sliding_window(const std::vector<int>& nums, int k)
{
    std::vector<int> answer;
    if (nums.empty() || k <= 0) {
        return answer;
    }
    answer.reserve(nums.size());

    std::deque<int> deque;
    for (int right = 0; right < static_cast<int>(nums.size()); ++right) {
        while (!deque.empty() && nums[deque.back()] <= nums[right]) {
            deque.pop_back();
        }
        deque.push_back(right);

        const int left = right - k + 1;
        if (deque.front() < left) {
            deque.pop_front();
        }
        if (left >= 0) {
            answer.push_back(nums[deque.front()]);
        }
    }

    return answer;
}

```

单调队列的优化理由和单调栈相似。一个新元素如果大于等于队尾元素，那么队尾元素在未来任何包含新元素的窗口里都不可能成为最大值，因为它更小且更早过期。于是可以安全删除。每个下标最多进队一次、出队一次，总复杂度 $O(n)$ 。

线性结构选型

需要稳定下标和顺序扫描，优先 `vector`；需要字符窗口，使用 `string` 加计数结构，不要频繁 `substr`；需要局部重连，考虑链表并画清楚前驱、当前、后继；需要最近未解决状态，用栈；需要按层或最短步数扩展，用队列；需要窗口两端同时维护候选，用双端队列。选择结构时同时说出它让哪一步查询、删除或更新变快。

本章建议配套练习：数组与前缀和做 303、304、560；字符串窗口做 3、438、567、76；链表做 19、21、24、141、142、206、92、25；栈做 20、71、150、155、739、84；队列和双端队列做 102、199、239、994。复盘时不要只写最终代码，要写三行说明：暴力方法慢在哪里，数据结构保存了什么状态，为什么每个元素只被处理有限次。

第 5 章

非线性数据结构：哈希表、堆、树、图和并查集

线性结构擅长处理顺序，非线性结构擅长处理关系和查询。哈希表回答“这个状态以前出现过吗”；堆回答“当前最极端的候选是谁”；树表达层次、有序性和递归子问题；图表达任意状态之间的连接；并查集维护“这些元素现在是否属于同一组”。这些结构不是刷题技巧的附属品，而是算法优化的来源。很多题从 $O(n^2)$ 变成 $O(n)$ ，并不是因为循环少写了一层，而是因为某个昂贵查询被换成了合适的数据结构查询。

哈希表是保存历史信息的核心结构。`std::unordered_map` 和 `std::unordered_set` 的平均查询、插入、删除复杂度是 $O(1)$ ，适合把“回头查找”变成“即时查询”。两数之和的暴力方法枚举两个下标，慢在每个数都要向后找补数；哈希表版本边扫描边记录已经见过的值，当前数只需要查询一次补数。这个优化成立的前提是题目只要求找到一对下标，且补数是否出现过可以由历史集合回答。

Listing 5.1 两数之和：哈希表把补数查询变成平均常数时间

```
std::vector<int> two_sum(const std::vector<int>& nums, int target)
{
    std::unordered_map<int, int> index_by_value;
    index_by_value.reserve(nums.size());

    for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
        const int complement = target - nums[i];
        const auto found = index_by_value.find(complement);
        if (found != index_by_value.end()) {
            return {found->second, i};
        }
        index_by_value.emplace(nums[i], i);
    }

    return {};
}
```

代码里先查再插入，是为了避免同一个元素被用两次。例如目标是 6，当前数是 3，如果先插入再查，可能错误地返回同一个下标。这里用 `find` 而不是 `operator[]`，因为 `operator[]` 在 key 不存在时会插入默认值。只想判断存在性时，C++20 可以用 `contains`；需要同时取值时，用 `find`，这样只查一次。刷题中还应该习惯 `reserve`，它能减少 rehash 次数。rehash 会重新组织桶，导致迭代器失效；如果保存了哈希表元素迭代器，扩容后不能继续使用。

哈希表不仅能存值到下标，也能存频次、前缀和、状态编码和访问标记。前缀和加哈希表很重要的一类优化。和为 K 的子数组如果暴力枚举左右端点，需要 $O(n^2)$ ；如果知道当前位置前缀和是 `current`，那么以当前位置结尾、和为 k 的子数组数量，等于以前出现过多少次 `current - k`。哈希表保存的是“某个前缀和出现次数”，查询的是“能和当前前缀配成目标的历史前缀”。

Listing 5.2 和为 K 的子数组：前缀和频次

```
int subarray_sum_equals_k(const std::vector<int>& nums, int k)
{
    std::unordered_map<int, int> prefix_count;
    prefix_count.reserve(nums.size() + 1);
    prefix_count.emplace(0, 1);

    int prefix = 0;
    int answer = 0;
    for (const int x : nums) {
        prefix += x;
        const auto found = prefix_count.find(prefix - k);
        if (found != prefix_count.end()) {
            answer += found->second;
        }
    }
}
```

```

    ++prefix_count[prefix];
}

return answer;
}

```

这个算法容易错在初始化。空前缀和 0 必须先出现一次，否则从下标 0 开始的合法子数组会漏掉。它也解释了为什么有负数时不能随便用滑动窗口：窗口和不再随右端单调增加，左端移动也不保证更接近目标；前缀和加哈希表不依赖正数单调性，所以适用范围更广。

哈希表的限制同样要清楚。第一，它不维护顺序，如果题目要求按排序顺序取最小、最大或区间，就该考虑 `std::map`、`std::set`、堆、树状数组或线段树。第二，平均 $O(1)$ 不等于绝对 $O(1)$ ，糟糕哈希或恶意数据可能退化。第三，自定义 key 需要提供相等比较和哈希函数。面试里一般不会刻意卡哈希攻击，但你要知道 `unordered_map<pair<int,int>, int>` 不能直接在所有标准库环境下工作，需要自定义哈希。

Listing 5.3 为 pair 坐标提供哈希函数

```

struct PairHash {
    std::size_t operator()(const std::pair<int, int>& point) const noexcept
    {
        constexpr unsigned COORDINATE_BITS = 32U;
        const auto first = static_cast<std::uint64_t>(<
            static_cast<std::uint32_t>(point.first));
        const auto second = static_cast<std::uint64_t>(<
            static_cast<std::uint32_t>(point.second));
        return std::hash<std::uint64_t>{}((first << COORDINATE_BITS) ^ second);
    }
};

std::unordered_set<std::pair<int, int>, PairHash> visited;

```

堆解决的是动态极值问题。C++ 里通常使用 `std::priority_queue`，默认是大顶堆；如果要小顶堆，需要显式给出容器和比较器。堆的插入和弹出堆顶都是 $O(\log n)$ ，查看堆顶是 $O(1)$ 。它不适合频繁删除任意元素，也不适合遍历出有序结果而不破坏结构。换句话说，堆只承诺“我能很快告诉你当前最大或最小是谁”。

Listing 5.4 `priority_queue` 的大顶堆和小顶堆

```

std::priority_queue<int> max_heap;

std::priority_queue<int,
    std::vector<int>,
    std::greater<int>> min_heap;

max_heap.push(3);
max_heap.push(10);
const int largest = max_heap.top();
max_heap.pop();

```

Top K 是堆最经典的用法。求数组中第 K 大元素，直觉做法是排序，复杂度 $O(n \log n)$ 。如果只关心前 K 大，不需要完整排序，可以维护一个大小为 K 的小顶堆。堆顶是当前前 K 大中最小的元素；新元素如果不超过堆顶，就不可能进入前 K；如果超过堆顶，就弹出堆顶并插入新元素。最终堆顶就是第 K 大。

Listing 5.5 第 K 大元素：大小为 K 的小顶堆

```

int find_kth_largest(const std::vector<int>& nums, int k)
{
    std::priority_queue<int,
        std::vector<int>,
        std::greater<int>> heap;

    for (const int x : nums) {
        if (static_cast<int>(heap.size()) < k) {
            heap.push(x);
        } else if (x > heap.top()) {

```

```

        heap.pop();
        heap.push(x);
    }
}

return heap.top();
}

```

这个方法的复杂度是 $O(n \log k)$ ，适合 k 远小于 n 的场景。还有一种平均线性复杂度的快速选择算法，后文会讲。面试表达时要说明选择堆的理由：不需要完整有序，只需要维护 K 个候选的边界。类似地，合并 K 个升序链表用小顶堆保存每条链表当前头节点；任务调度用堆保存当前最早结束或最高优先级任务；数据流中位数用一个大顶堆和一个小顶堆分别维护较小一半和较大一半。

树把数据组织成层次关系。二叉树题看似在考递归，实际在考状态如何沿边传播。状态可以从父节点传给子节点，例如验证二叉搜索树时传递取值上下界；也可以从子节点汇总到父节点，例如求树的高度、直径、最大路径和；还可以按层从左到右传播，例如层序遍历。只背前序、中序、后序三个词是不够的，必须知道遍历顺序服务于哪类状态。

Listing 5.6 二叉搜索树验证：显式栈中序遍历

```

bool is_valid_bst(TreeNode* root)
{
    std::vector<TreeNode*> stack;
    TreeNode* current = root;
    std::optional<long long> previous;

    while (current != nullptr || !stack.empty()) {
        while (current != nullptr) {
            stack.push_back(current);
            current = current->left;
        }

        current = stack.back();
        stack.pop_back();

        if (previous.has_value() && current->value <= previous.value()) {
            return false;
        }
        previous = current->value;
        current = current->right;
    }

    return true;
}

```

二叉搜索树的关键性质是中序遍历严格递增。验证 BST 的暴力想法可能是对每个节点扫描整棵左子树和右子树，判断左边都小、右边都大，这会重复访问大量节点。中序遍历把全局约束转成相邻访问值的递增关系，每个节点只处理一次。这里用显式栈而不是递归，是为了避免极端深树导致 C++ 调用栈过深，同时也让“遍历状态”更清楚。

树题经常需要区分“节点为空的返回值”和“答案的初始值”。求最大深度时，空节点深度是 0；求最大路径和时，答案不能初始化为 0，因为所有节点可能都是负数；求最近公共祖先时，返回值表示“当前子树是否找到了目标节点或祖先”。这些不是小细节，而是树形 DP 的语义。每个返回值都应该能用一句话解释：它代表当前子树对父节点贡献什么信息。

图是最通用的关系结构。只要有状态和一步转移，就可以建模成图。图题最重要的不是 DFS 或 BFS 写法，而是建模四问：节点是什么，边是什么，访问状态是什么，队列或栈里保存什么。岛屿数量里，节点是陆地格子，边是上下左右相邻；课程表里，节点是课程，有向边表示先修依赖；单词接龙里，节点是单词，边表示一次字符变化；开锁问题里，节点是四位密码状态，边是转动一位数字。

图的存储方式要跟输入规模匹配。稠密图可以用邻接矩阵，判断两点是否有边是 $O(1)$ ，但空间是 $O(n^2)$ 。大多数面试题是稀疏图，更适合邻接表，空间是 $O(n + m)$ 。如果节点编号连续，用 `vector<vector<int>>`；如果节点是字符串或稀疏编号，用哈希表映射到编号，或者直接用 `unordered_map<string, vector<string>>`。

Listing 5.7 课程表：邻接表和入度数组

```

bool can_finish(int num_courses,
                const std::vector<std::vector<int>>& prerequisites)
{
    std::vector<std::vector<int>> graph(num_courses);
    std::vector<int> indegree(num_courses, 0);

    for (const auto& edge : prerequisites) {
        const int course = edge[0];
        const int prerequisite = edge[1];
        graph[prerequisite].push_back(course);
        ++indegree[course];
    }

    std::queue<int> queue;
    for (int course = 0; course < num_courses; ++course) {
        if (indegree[course] == 0) {
            queue.push(course);
        }
    }

    int learned = 0;
    while (!queue.empty()) {
        const int course = queue.front();
        queue.pop();
        ++learned;

        for (const int next : graph[course]) {
            --indegree[next];
            if (indegree[next] == 0) {
                queue.push(next);
            }
        }
    }

    return learned == num_courses;
}

```

拓扑排序的暴力直觉是反复扫描所有课程，找当前没有前置依赖的课。这样每学一门课都可能重新扫描所有边。入度数组把“还有多少依赖没被满足”变成可增量更新的状态；队列保存当前入度为 0 的课程。每条边只在它的起点被移除时处理一次，复杂度 $O(n + m)$ 。如果最后学完的课程数少于总数，说明剩余节点互相依赖，图中有环。

DFS 和 BFS 都可以遍历图，但语义不同。BFS 适合无权最短步数和层次扩散，因为它按距离递增访问状态；DFS 适合连通块标记、路径搜索、拓扑状态检测和回溯枚举，因为它沿一条路径深入。C++ 里递归 DFS 简洁，但深度过大可能栈溢出。本书讲递归思想，但实现上会经常给出显式栈版本，让遍历过程、访问标记和状态回退更加可控。

Listing 5.8 岛屿数量：显式栈标记连通块

```

int num_islands(std::vector<std::vector<char>>& grid)
{
    if (grid.empty() || grid[0].empty()) {
        return 0;
    }

    constexpr char LAND = '1';
    constexpr char WATER = '0';
    constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
        std::pair{1, 0}, std::pair{-1, 0},
        std::pair{0, 1}, std::pair{0, -1}
    };

    const int rows = static_cast<int>(grid.size());
    const int cols = static_cast<int>(grid[0].size());
    int islands = 0;

    for (int row = 0; row < rows; ++row) {
        for (int col = 0; col < cols; ++col) {
            if (grid[row][col] != LAND) {
                continue;
            }

```

```

++islands;
std::vector<std::pair<int, int>> stack{{row, col}};
grid[row][col] = WATER;

while (!stack.empty()) {
    const auto [r, c] = stack.back();
    stack.pop_back();

    for (const auto [dr, dc] : DIRECTIONS) {
        const int nr = r + dr;
        const int nc = c + dc;
        if (nr < 0 || nr >= rows || nc < 0 || nc >= cols ||
            grid[nr][nc] != LAND) {
            continue;
        }
        grid[nr][nc] = WATER;
        stack.emplace_back(nr, nc);
    }
}

return islands;
}

```

这段代码把访问过的陆地直接改成水，省掉单独的 `visited` 数组。这样做是否合适取决于题目是否允许修改输入。如果不允许，就创建同样大小的布尔矩阵。图题里访问标记必须在入队或入栈时就设置，而不是弹出时才设置；否则同一个节点可能被多个邻居重复加入，轻则浪费时间，重则导致状态爆炸。

并查集维护动态连通性。它有两个核心操作：`find` 找到一个元素所在集合的代表，`unite` 合并两个集合。路径压缩让查找时顺手把沿途节点直接挂到根上，按大小或按秩合并让树保持较浅。两者结合后，单次操作的均摊复杂度接近常数。它适合回答“是否在同一组”“合并后还有几个连通块”，不适合回答两点之间的具体路径或最短距离。

Listing 5.9 并查集：路径压缩与按大小合并

```

class DisjointSetUnion {
public:
    explicit DisjointSetUnion(int n)
        : parent_(n), size_(n, 1)
    {
        std::iota(this->parent_.begin(), this->parent_.end(), 0);
    }

    int find(int x)
    {
        int root = x;
        while (this->parent_[root] != root) {
            root = this->parent_[root];
        }
        while (this->parent_[x] != x) {
            const int next = this->parent_[x];
            this->parent_[x] = root;
            x = next;
        }
        return root;
    }

    bool unite(int a, int b)
    {
        int root_a = find(a);
        int root_b = find(b);
        if (root_a == root_b) {
            return false;
        }
        if (this->size_[root_a] < this->size_[root_b]) {
            std::swap(root_a, root_b);
        }
        this->parent_[root_b] = root_a;
    }
};

```

```

        this->size[root_a] += this->size[root_b];
        return true;
    }

private:
    std::vector<int> parent_;
    std::vector<int> size_;
};

```

冗余连接是并查集的标准例题。给一棵树额外加一条边，依次扫描边。如果一条边连接的两个点已经在同一集合里，那么再加入它就会形成环，这条边就是冗余边。暴力方法可以每次加边后做一次 DFS 检查连通性，复杂度很高；并查集把“两个点是否已经连通”维护成近乎常数时间查询。

账户合并也能体现并查集的建模能力。邮箱是连接账户的证据：两个账户只要共享一个邮箱，就属于同一个人。可以把账户编号作为并查集元素，用哈希表记录邮箱第一次出现在哪个账户；再次看到同一邮箱时，把两个账户合并。最后按根节点收集邮箱并排序。这里哈希表负责从字符串邮箱找到账户编号，并查集负责维护账户之间的连通分量。一个题常常需要多个数据结构协作，这也是“数据结构和算法不分家”的真实含义。

数据结构和算法不分家

优化通常来自查询模型的改变。哈希表把历史查找变快，堆把动态极值变快，树把层次状态和有序性质显式化，图把状态转移统一成边，并查集把动态连通性维护变快。刷题复盘时必须同时写出算法动作和数据结构职责：保存什么，支持什么查询，删除什么过期状态，为什么不会漏答案。

本章建议配套练习：哈希表做 1、49、128、560、146；堆做 215、347、23、295、621；树做 94、98、102、104、105、124、236、230；图做 200、207、210、417、994、127、752；并查集做 547、684、721、990、1319。每道题至少复盘一次建模：如果不用这个结构，暴力瓶颈在哪里；用了这个结构，哪一步查询或合并被降下来了；结构本身有什么边界和失效条件。

第零部分

核心算法：从暴力到优化

第 6 章

数组、双指针和滑动窗口

数组题是刷题的地基。双指针和滑动窗口不是孤立技巧，而是从暴力枚举子结构中优化出来的方法。双指针常用于有序数组、两端收缩和左右边界协作；滑动窗口常用于连续子数组或子串，并且窗口状态能够增量维护。

盛最多水的容器暴力解枚举左右边界。面积由宽度和较短边决定，所以每个下标对都可能成为候选。

Listing 6.1 盛水容器暴力解

```
int max_area_bruteforce(const std::vector<int>& height)
{
    int best = 0;
    for (int left = 0; left < static_cast<int>(height.size()); ++left) {
        for (int right = left + 1; right < static_cast<int>(height.size()); ++right) {
            const int width = right - left;
            const int bounded_height = std::min(height[left], height[right]);
            best = std::max(best, width * bounded_height);
        }
    }
    return best;
}
```

暴力法是 $O(n^2)$ 。优化来自候选淘汰。初始取最宽的左右边界，如果移动较高的一边，宽度变小，而较短边没有变高，面积不可能变大。因此只能移动较短边，才有机会提高瓶颈高度。

Listing 6.2 盛水容器双指针解

```
int max_area_two_pointers(const std::vector<int>& height)
{
    int left = 0;
    int right = static_cast<int>(height.size()) - 1;
    int best = 0;

    while (left < right) {
        const int width = right - left;
        const int bounded_height = std::min(height[left], height[right]);
        best = std::max(best, width * bounded_height);

        if (height[left] <= height[right]) {
            ++left;
        } else {
            --right;
        }
    }
    return best;
}
```

不变量是：每一步排除的短边，不会和更内侧的任何边形成更优答案，因为宽度只会变小，短边高度仍然是瓶颈。这个证明比“左右指针往中间走”重要得多。

长度最小的子数组展示滑动窗口。暴力法枚举所有起点和终点，遇到和达到目标时更新答案。它慢在每个起点都重新累加。

Listing 6.3 长度最小子数组滑动窗口

```
int min_subarray_len_sliding_window(int target, const std::vector<int>& nums)
{
    int best = std::numeric_limits<int>::max();
    int left = 0;
    int sum = 0;

    for (int right = 0; right < static_cast<int>(nums.size()); ++right) {
```

```

        sum += nums[right];
        while (sum >= target) {
            best = std::min(best, right - left + 1);
            sum -= nums[left];
            ++left;
        }
    }

    return best == std::numeric_limits<int>::max() ? 0 : best;
}

```

这个优化依赖正整数。右边界右移时窗口和只会增加，左边界右移时窗口和只会减少。如果数组包含负数，这个单调性消失，滑动窗口可能失效，需要考虑前缀和、单调队列或其他结构。

无重复字符的最长子串也是窗口题，但窗口状态不是和，而是字符最后出现位置。窗口不变量是 `[left, right]` 内没有重复字符。

Listing 6.4 无重复字符最长子串

```

int longest_substring_without_repeat_window(const std::string& s)
{
    constexpr int ASCII_RANGE = 256;
    std::array<int, ASCII_RANGE> last_seen{};
    last_seen.fill(-1);

    int left = 0;
    int best = 0;

    for (int right = 0; right < static_cast<int>(s.size()); ++right) {
        const auto ch = static_cast<unsigned char>(s[right]);
        if (last_seen[ch] >= left) {
            left = last_seen[ch] + 1;
        }
        last_seen[ch] = right;
        best = std::max(best, right - left + 1);
    }

    return best;
}

```

这里用 `std::array<int, 256>` 是因为按 ASCII 字符处理。如果题目要求完整 Unicode 字符，需要先明确字符单元，或使用哈希表保存字符到位置。

三数之和是双指针的第二个标准模型。题目要求找出所有不重复三元组，使得三个数之和为 0。暴力方法枚举三个下标，时间复杂度 $O(n^3)$ ，还要处理去重；如果用哈希表固定两数再找第三数，可以降到 $O(n^2)$ ，但去重和重复值处理仍然复杂。真正稳定的做法是先排序，再固定第一个数，把剩下两个数变成有序数组上的两数之和。

排序后的关键性质是：固定 `first` 后，如果 `nums[left] + nums[right]` 太小，说明左端值不够大，只能右移 `left`；如果和太大，说明右端值太大，只能左移 `right`。这就是有序性带来的候选淘汰。去重也依赖排序：同一层循环里，如果当前值和前一个值相同，就跳过；找到一个答案后，左右指针都要跳过连续重复值。

Listing 6.5 三数之和：排序加双指针

```

std::vector<std::vector<int>> three_sum(std::vector<int> nums)
{
    std::sort(nums.begin(), nums.end());
    std::vector<std::vector<int>> answer;

    for (int first = 0; first < static_cast<int>(nums.size()); ++first) {
        if (first > 0 && nums[first] == nums[first - 1]) {
            continue;
        }
        if (nums[first] > 0) {
            break;
        }

        int left = first + 1;
        int right = static_cast<int>(nums.size()) - 1;
    }
}

```

```

while (left < right) {
    const int sum = nums[first] + nums[left] + nums[right];
    if (sum == 0) {
        answer.push_back({nums[first], nums[left], nums[right]});
        const int left_value = nums[left];
        const int right_value = nums[right];
        while (left < right && nums[left] == left_value) {
            ++left;
        }
        while (left < right && nums[right] == right_value) {
            --right;
        }
    } else if (sum < 0) {
        ++left;
    } else {
        --right;
    }
}

return answer;
}

```

这里参数按值传入,是因为函数内部要排序。如果不想修改调用者的数组,按值传参是合理的;如果题目允许原地修改,也可以传引用。复杂度是排序 $O(n \log n)$ 加双指针枚举 $O(n^2)$, 总时间 $O(n^2)$, 额外空间不计答案为 $O(1)$ 或排序栈空间。面试表达要强调: 去重不是最后把答案放进 set, 而是在生成答案时就避免重复, 这样更可控。

找到字符串中所有字母异位词是固定长度窗口。题目给定字符串 *s* 和模式串 *p*, 要求找出 *s* 中所有长度等于 *p* 且字符频次完全相同的起点。暴力方法枚举每个长度为 *p.size()* 的子串, 然后排序比较或统计比较。如果每次排序, 复杂度是 $O(nk \log k)$; 如果每次重新统计, 复杂度是 $O(nk)$ 。慢点在于相邻窗口高度重叠, 却重复计算了大部分字符频次。

优化方法是维护固定长度窗口的频次数组。窗口右端每进入一个字符, 就把它的计数加一; 当窗口长度超过模式串长度, 左端字符离开, 计数减一。由于题目通常限定小写英文, 可以用 `std::array<int, 26>`, 比较两个数组是常数时间。窗口不变量是: 处理到 *right* 时, *window* 正好记录当前长度不超过模式串长度的后缀窗口字符频次。

Listing 6.6 找到所有字母异位词: 固定长度窗口

```

std::vector<int> find_anagrams_window(const std::string& s,
                                     const std::string& p)
{
    constexpr int ALPHABET_SIZE = 26;
    std::vector<int> answer;
    if (s.size() < p.size()) {
        return answer;
    }

    std::array<int, ALPHABET_SIZE> need{};
    std::array<int, ALPHABET_SIZE> window{};
    for (const char ch : p) {
        ++need[ch - 'a'];
    }

    for (std::size_t right = 0; right < s.size(); ++right) {
        ++window[s[right] - 'a'];
        if (right >= p.size()) {
            --window[s[right - p.size()] - 'a'];
        }
        if (window == need) {
            answer.push_back(static_cast<int>(right + 1 - p.size()));
        }
    }

    return answer;
}

```

这题容易错在下标。窗口第一次达到目标长度是在 `right + 1 == p.size()`, 起点是 `right + 1 - p.size()`。如果用 `int` 写下标, 要小心 `s.size()` 是无符号类型; 本书示例这里使用 `std::size_t`

控制循环，最终起点再转成 `int`，因为力扣返回 `vector<int>`。

最小覆盖子串是变长窗口里最重要的一题。题目要求在 `s` 中找最短子串，覆盖 `t` 中所有字符及其次数。暴力方法枚举所有子串，再检查是否覆盖，复杂度至少 $O(n^2|\Sigma|)$ 。优化的核心是：右边界扩张直到窗口满足覆盖条件，然后尽可能移动左边界缩短窗口；一旦左移导致窗口不满足，再继续右移。这个过程成立，因为右移只会增加字符，左移只会删除字符，覆盖关系可以被增量维护。

Listing 6.7 最小覆盖子串：变长窗口

```
std::string min_window_substring(const std::string& s, const std::string& t)
{
    constexpr int ASCII_RANGE = 128;
    std::array<int, ASCII_RANGE> need{};
    std::array<int, ASCII_RANGE> window{};
    int required = 0;

    for (const char ch : t) {
        const auto index = static_cast<unsigned char>(ch);
        if (need[index] == 0) {
            ++required;
        }
        ++need[index];
    }

    int formed = 0;
    int left = 0;
    int best_left = 0;
    int best_length = std::numeric_limits<int>::max();

    for (int right = 0; right < static_cast<int>(s.size()); ++right) {
        const auto in = static_cast<unsigned char>(s[right]);
        ++window[in];
        if (need[in] > 0 && window[in] == need[in]) {
            ++formed;
        }

        while (formed == required) {
            const int length = right - left + 1;
            if (length < best_length) {
                best_length = length;
                best_left = left;
            }

            const auto out = static_cast<unsigned char>(s[left]);
            --window[out];
            if (need[out] > 0 && window[out] < need[out]) {
                --formed;
            }
            ++left;
        }
    }

    if (best_length == std::numeric_limits<int>::max()) {
        return "";
    }
    return s.substr(best_left, best_length);
}
```

`required` 表示需要满足的不同字符种类数，`formed` 表示当前窗口里已经满足需求的字符种类数。不能只用窗口总长度判断，因为 `t = "AABC"` 时，两个 `A` 都必须覆盖。复杂度是 $O(n + m + |\Sigma|)$ ，在 ASCII 数组固定时可视为 $O(n + m)$ 。面试表达要说清楚：这题不是固定窗口，因为答案长度未知；它是“先满足，再收缩”的最短覆盖模型。

本章题目复盘模板

数组和窗口题复盘时至少写五句话：暴力枚举什么；慢点是否来自重复扫描、重复统计或候选过多；优化使用了有序性、窗口增量状态还是候选淘汰；左右指针移动条件是什么；每个元素为什么只会被处理常数次数。只写“用双指针”或“用滑动窗口”不算会。

本章力扣主线：11 盛最多水的容器、15 三数之和、209 长度最小的子数组、3 无重复字符的最

长子串、438 找到字符串中所有字母异位词、76 最小覆盖子串。复盘时必须写出窗口不变量和指针移动条件。

第 7 章

哈希表和前缀和

哈希表解决“快速查历史信息”，前缀和解决“快速计算区间信息”。两者结合后，可以把很多子数组问题从 $O(n^2)$ 降到 $O(n)$ 。

两数之和是哈希表入门，但真正要记住的是模型：当前元素需要查询历史中是否存在某个补数。只要查询对象可以作为 key，哈希表就能把线性查找降为平均常数查找。暴力方法枚举 i 和 j ，判断 $\text{nums}[i] + \text{nums}[j] == \text{target}$ ，时间复杂度 $O(n^2)$ 。慢点不在加法，而在“为了每个数反复查找另一个数”。哈希表版本从左到右扫描，用 `value_to_index` 保存已经出现过的值及其下标；处理当前值时，只查 $\text{target} - \text{nums}[i]$ 是否在历史中出现过。

Listing 7.1 两数之和：保存历史值到下标

```
std::vector<int> two_sum_hash(const std::vector<int>& nums, int target)
{
    std::unordered_map<int, int> value_to_index;
    value_to_index.reserve(nums.size());

    for (int index = 0; index < static_cast<int>(nums.size()); ++index) {
        const int need = target - nums[index];
        const auto found = value_to_index.find(need);
        if (found != value_to_index.end()) {
            return {found->second, index};
        }
        value_to_index.emplace(nums[index], index);
    }

    return {};
}
```

这段代码必须先查再插入，避免同一个元素和自己配对。如果数组里有重复值，`emplace` 会保留第一次出现的下标；对力扣 1 这种保证唯一答案的题足够。如果题目要求所有答案，就不能这样提前返回，而要继续扫描并处理去重。面试时可以这样表达：哈希表保存历史值到下标的映射，查询 key 是当前值的补数，平均时间 $O(n)$ ，空间 $O(n)$ 。

前缀和的定义是：`prefix[i]` 表示前 i 个元素的和。那么子数组 `[left, right]` 的和是 `prefix[right + 1] - prefix[left]`。如果题目问和为 k 的子数组个数，就等价于对每个当前位置查找之前有多少个前缀和等于 `current_prefix - k`。

Listing 7.2 和为 K 的子数组

```
int subarray_sum(const std::vector<int>& nums, int k)
{
    std::unordered_map<int, int> prefix_count;
    prefix_count.reserve(nums.size() + 1);
    prefix_count[0] = 1;

    int prefix = 0;
    int answer = 0;
    for (int x : nums) {
        prefix += x;
        const int need = prefix - k;
        if (const auto found = prefix_count.find(need); found != prefix_count.end()) {
            answer += found->second;
        }
        ++prefix_count[prefix];
    }
    return answer;
}
```

为什么先查再插入？因为当前前缀只能和之前的前缀组成非空子数组。如果先插入，在 $k == 0$ 时可能把当前位置和自己配对。为什么保存次数而不是下标？因为题目问个数，一个当前前缀可以

和多个历史前缀组成答案。

最长连续序列展示 `unordered_set` 的另一种用法。把所有数放进集合后，只从没有前驱 $x - 1$ 的数开始扩展。这样每条连续链只从头部扫描一次，每个数最多被访问常数次数，平均时间 $O(n)$ 。

哈希题常见错误包括：用 `operator[]` 做纯查询导致插入多余 key；忘记处理重复值；依赖 `unordered_map` 遍历顺序；没有考虑 `long long` 前缀和溢出；把滑动窗口误用到含负数数组。

最长连续序列要求在未排序数组中找最长的连续整数链。排序可以做，复杂度 $O(n \log n)$ ；暴力方法从每个数开始向后找 $x + 1$ 、 $x + 2$ ，如果每次都线性查找，会退化到 $O(n^2)$ 。哈希集合优化了“某个数是否存在”的查询，但如果从每个数都向后扩展，连续链中间的数会被重复扫描。真正的关键是只从链头开始扩展：如果 $x - 1$ 存在，说明 x 不是开头，跳过。

Listing 7.3 最长连续序列：只从链头扩展

```
int longest_consecutive(const std::vector<int>& nums)
{
    std::unordered_set<int> values;
    values.reserve(nums.size());
    for (const int x : nums) {
        values.insert(x);
    }

    int best = 0;
    for (const int x : values) {
        if (values.contains(x - 1)) {
            continue;
        }

        int current = x;
        int length = 1;
        while (values.contains(current + 1)) {
            ++current;
            ++length;
        }
        best = std::max(best, length);
    }

    return best;
}
```

这题的摊还分析要讲清楚：每条连续链只会从最小值开始扫描一次，链中每个元素只在这次扫描里被访问，因此平均时间是 $O(n)$ 。不能在遍历 `values` 时删除元素，除非你非常清楚迭代器失效规则；本书这里选择不删除，代码更直接。

四数相加 II 展示“拆半”的哈希思想。题目给四个数组，统计 $a + b + c + d == 0$ 的元组数量。暴力枚举四个下标是 $O(n^4)$ 。如果固定前三个再查第四个，仍然是 $O(n^3)$ 。更好的做法是把四个数组拆成两组：先统计所有 $a + b$ 的和出现次数，再枚举 $c + d$ ，查询 $-(c + d)$ 出现过多少次。

Listing 7.4 四数相加 II：两两拆分

```
int four_sum_count(const std::vector<int>& nums1,
                  const std::vector<int>& nums2,
                  const std::vector<int>& nums3,
                  const std::vector<int>& nums4)
{
    std::unordered_map<int, int> pair_sum_count;
    pair_sum_count.reserve(nums1.size() * nums2.size());

    for (const int a : nums1) {
        for (const int b : nums2) {
            ++pair_sum_count[a + b];
        }
    }

    int answer = 0;
    for (const int c : nums3) {
        for (const int d : nums4) {
            const auto found = pair_sum_count.find(-(c + d));
            if (found != pair_sum_count.end()) {
                answer += found->second;
            }
        }
    }
}
```

```
    }  
}  
  
return answer;  
}
```

拆半的本质是把 n^4 的组合空间拆成两个 n^2 的集合,再用哈希表连接它们。空间复杂度是 $O(n^2)$ 。这类题的面试表达要说明为什么保存次数:同一个两数和可能由多个下标对产生,每一个都能和后半部分组成不同元组。

哈希题复盘模板

哈希题不要只写“用 map”。复盘时必须写: key 是什么, value 是什么; 查询发生在插入前还是插入后; 重复 key 怎么处理; 是否需要次数、最早位置、最新位置还是集合存在性; 平均复杂度和最坏情况分别是什么。

本章力扣主线: 1 两数之和、49 字母异位词分组、128 最长连续序列、560 和为 K 的子数组、523 连续的子数组和、525 连续数组、454 四数相加 II。复盘时必须说明哈希表保存的是值、下标、次数还是最早位置。

第 8 章

二分查找和答案空间

二分查找的本质不是在数组里找数，而是在单调结构中找边界。只要能定义判断函数 `check(x)`，并且它在答案空间上单调，就可以二分。

基础二分在有序数组中查找目标值。写法看似简单，错误通常来自边界没有排除 `mid`、返回值不清楚或 `left + right` 溢出。

Listing 8.1 基础二分查找

```
int binary_search_index(const std::vector<int>& nums, int target)
{
    int left = 0;
    int right = static_cast<int>(nums.size()) - 1;
    while (left <= right) {
        const int mid = left + (right - left) / 2;
        if (nums[mid] == target) {
            return mid;
        }
        if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;
}
```

找左边界时，标准库 `lower_bound` 是最好的参照：它找第一个不小于目标的位置。很多题不是找一个等于目标的位置，而是找第一个满足条件的位置。

搜索插入位置就是左边界模型。题目要求如果目标存在，返回下标；如果不存在，返回应该插入的位置。这个位置其实就是第一个 `nums[i] >= target` 的下标。如果所有数都小于目标，答案就是 `nums.size()`。暴力方法从左到右扫描，时间 $O(n)$ ；有序数组给了单调性，所以可以二分。

Listing 8.2 搜索插入位置：lower_bound 写法

```
int search_insert_position(const std::vector<int>& nums, int target)
{
    int left = 0;
    int right = static_cast<int>(nums.size());

    while (left < right) {
        const int mid = left + (right - left) / 2;
        if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid;
        }
    }

    return left;
}
```

这里使用左闭右开区间 `[left, right)`。循环不变量是答案一定在这个区间里；当 `nums[mid] < target`，`mid` 以及左侧都不可能是第一个不小于目标的位置，所以令 `left = mid + 1`；否则 `mid` 可能就是答案，不能排除，只能令 `right = mid`。这个“保留 `mid`”是左边界二分的核心。

答案二分把“位置”换成“答案值”。爱吃香蕉的珂珂中，答案是速度。速度越大越容易在规定时间内吃完，所以 `check(speed)` 从 false 变成 true。我们要找第一个 true。分割数组最大值、运输包裹能力、制作花束最少天数也都是这个模型。

Listing 8.3 爱吃香蕉的珂珂：答案二分

```

int min_eating_speed(const std::vector<int>& piles, int hours)
{
    int left = 1;
    int right = *std::max_element(piles.begin(), piles.end());

    auto can_finish = [&](int speed) {
        long long used_hours = 0;
        for (const int pile : piles) {
            used_hours += (pile + speed - 1LL) / speed;
        }
        return used_hours <= hours;
    };

    while (left < right) {
        const int mid = left + (right - left) / 2;
        if (can_finish(mid)) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }

    return left;
}

```

暴力方法可以从速度 1 一直试到最大堆大小，每次计算总小时数，复杂度 $O(mn)$ ，其中 m 是最大速度范围。二分优化来自单调性：速度越大，吃完所需小时数越少；如果某个速度可以完成，那么更大的速度也一定可以完成。我们要找最小可行速度，所以是“第一个 true”。计算向上取整时用 $(pile + speed - 1LL) / speed$ ，其中 $1LL$ 避免整数溢出。

搜索旋转排序数组是另一类二分：单调性不是全局存在，而是每一步至少有一半区间有序。暴力扫描是 $O(n)$ 。二分时先判断 $[left, mid]$ 是否有序；如果有序，再判断目标是否落在这一半里，落在里面就收缩到左半，否则去右半。若左半无序，右半一定有序，用同样逻辑判断。这个题不要死背条件，应该画出旋转数组的两段有序结构。

Listing 8.4 搜索旋转排序数组

```

int search_rotated_array(const std::vector<int>& nums, int target)
{
    int left = 0;
    int right = static_cast<int>(nums.size()) - 1;

    while (left <= right) {
        const int mid = left + (right - left) / 2;
        if (nums[mid] == target) {
            return mid;
        }

        if (nums[left] <= nums[mid]) {
            if (nums[left] <= target && target < nums[mid]) {
                right = mid - 1;
            } else {
                left = mid + 1;
            }
        } else {
            if (nums[mid] < target && target <= nums[right]) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
    }

    return -1;
}

```

这题默认数组没有重复值。如果允许重复，例如 $[1, 0, 1, 1, 1]$ ， $nums[left] == nums[mid]$ 时无法判断哪边有序，可能需要移动边界退化处理，最坏复杂度会变成 $O(n)$ 。面试时主动说明这个前提，会显得你不是机械套模板。

答案二分三步

第一，写出答案范围。第二，写出 `check(x)`。第三，证明单调性，并说明要找最小可行值还是最大可行值。没有单调性，不要二分。

本章力扣主线：704 二分查找、35 搜索插入位置、34 查找第一个和最后一个位置、33 搜索旋转排序数组、162 寻找峰值、875 爱吃香蕉的珂珂、1011 在 D 天内送达包裹的能力、410 分割数组的最大值。

第 9 章

树、图、BFS 和 DFS

树和图题的第一难点是建模。题目未必直接给你图，但只要有状态和状态之间的一步转移，就可以看成图。网格题的格子是节点，相邻方向是边；课程表的课程是节点，先修关系是有向边；单词接龙的单词是节点，一次变化是边。

树是没有环的特殊图。二叉树节点通常只有左右孩子，父子关系天然给出层次；普通图没有固定根，可能有环，必须显式维护访问状态。二者的共同点是：算法不会凭空发生，所有遍历都在回答“当前节点能把什么信息交给相邻节点”。BFS 按层推进，适合无权最短步数、层序遍历和多源扩散；DFS 沿路径推进，适合连通块、路径搜索、状态检测和回溯枚举。本书会讲递归思想，但 C++ 实现优先使用显式队列、栈和循环，因为在线评测里极端深树和大图可能让递归调用栈失控。

为了让代码统一，本章二叉树示例使用下面的节点字段名。力扣原题多使用 `val`，实际提交时把 `value` 改成 `val` 即可，核心逻辑不变。

Listing 9.1 二叉树节点示意

```
struct TreeNode {
    int value = 0;
    TreeNode* left = nullptr;
    TreeNode* right = nullptr;
};
```

二叉树层序遍历是 BFS 的入门题，但它不是“把队列模板背下来”这么简单。题目要求按层返回节点值，因此队列里保存的是下一批待访问节点，而外层循环开始时，队列必须恰好包含当前层的所有节点。暴力做法可以先求树高，再对每一层从根节点重新扫描一次，把深度等于目标层的节点收集出来。这个思路正确，但一棵高度为 h 的树会被重复扫描很多次，最坏退化到 $O(nh)$ 。

优化来自队列的先进先出顺序。根节点先入队，处理当前层时，把它的孩子按从左到右的顺序放到队尾。当前层节点全部弹出后，队列中剩下的正好是下一层节点。这里的关键变量是 `level_size`：它把“当前层的边界”固定下来，避免新入队的下一层节点被提前处理。

Listing 9.2 二叉树层序遍历：固定当前层边界

```
std::vector<std::vector<int>> level_order(TreeNode* root)
{
    std::vector<std::vector<int>> levels;
    if (root == nullptr) {
        return levels;
    }

    std::queue<TreeNode*> queue;
    queue.push(root);

    while (!queue.empty()) {
        const int level_size = static_cast<int>(queue.size());
        std::vector<int> level;
        level.reserve(level_size);

        for (int i = 0; i < level_size; ++i) {
            TreeNode* node = queue.front();
            queue.pop();
            level.push_back(node->value);

            if (node->left != nullptr) {
                queue.push(node->left);
            }
            if (node->right != nullptr) {
                queue.push(node->right);
            }
        }
    }
}
```

```

        levels.push_back(std::move(level));
    }

    return levels;
}

```

这题的复杂度是 $O(n)$ ，每个节点入队一次、出队一次。空间复杂度是 $O(w)$ ， w 是树的最大宽度；如果把输出也计入空间，则是 $O(n)$ 。常见错误有三个：空树忘记返回空数组；没有记录 `level_size`，导致层边界混在一起；把右孩子放到左孩子前面，破坏题目要求的从左到右顺序。面试时要把“不变量”说出来：每次进入外层循环，队列中保存的是同一层、尚未输出的全部节点。

最大深度看起来更像递归题，数学定义也很自然：空树深度为 0，非空树深度是左右子树最大深度加 1。递归写法短，但深度很大的链式树会把调用栈压得很深。用 BFS 写法时，最大深度就是层序遍历处理过的层数；用显式栈写法时，栈里保存节点和它所在深度。两种实现都能避免把深度交给语言调用栈管理。

Listing 9.3 最大深度：显式栈保存节点深度

```

int max_depth(TreeNode* root)
{
    if (root == nullptr) {
        return 0;
    }

    int answer = 0;
    std::vector<std::pair<TreeNode*, int>> stack{{root, 1}};

    while (!stack.empty()) {
        const auto [node, depth] = stack.back();
        stack.pop_back();
        answer = std::max(answer, depth);

        if (node->left != nullptr) {
            stack.emplace_back(node->left, depth + 1);
        }
        if (node->right != nullptr) {
            stack.emplace_back(node->right, depth + 1);
        }
    }

    return answer;
}

```

这段代码的状态非常明确：栈里的每个元素都是一个尚未展开的节点，以及从根到它经过的节点数。暴力和优化的差别不在复杂度，递归和显式栈都是 $O(n)$ ；差别在工程边界。显式栈把潜在的系统调用栈风险变成可控的堆上容器，也更容易扩展成“最大深度路径”“带权深度”“同时记录父节点”等变体。易错点是深度定义：力扣 104 问节点数层数，根节点深度应是 1；如果题目问边数高度，根节点高度才可能记作 0。

验证二叉搜索树经常暴露“只看局部”的错误。很多人会写成：当前节点大于左孩子、小于右孩子，就继续检查左右子树。这个判断不够，因为 BST 的约束是全局的：左子树所有节点都必须小于当前节点，右子树所有节点都必须大于当前节点。暴力补救方法是对每个节点扫描整棵左子树最大值和右子树最小值，但会重复访问，最坏 $O(n^2)$ 。

更稳定的优化是利用 BST 的中序遍历性质。对合法 BST 做中序遍历，访问值必须严格递增。于是全局范围约束被转换成相邻访问值比较：只要当前值不大于前一个中序值，就一定非法。这里要强调“严格递增”，因为力扣 98 不允许重复值出现在左右子树。

Listing 9.4 验证二叉搜索树：显式栈中序遍历

```

bool is_valid_bst(TreeNode* root)
{
    std::vector<TreeNode*> stack;
    TreeNode* current = root;
    std::optional<long long> previous;

    while (current != nullptr || !stack.empty()) {
        while (current != nullptr) {

```

```

        stack.push_back(current);
        current = current->left;
    }

    current = stack.back();
    stack.pop_back();

    if (previous.has_value() && current->value <= previous.value()) {
        return false;
    }
    previous = current->value;
    current = current->right;
}

return true;
}

```

这题的时间复杂度是 $O(n)$ ，空间复杂度是 $O(h)$ ， h 是树高。代码使用 `long long` 保存前一个值，是为了避免节点值接近 `int` 边界时用哨兵值出错；用 `std::optional` 比写一个“极小初值”更严谨。面试表达可以这样组织：先指出局部比较错误，再给出全局约束，最后说明中序遍历把全局约束转成单调序列检查。

最近公共祖先的题意是：给定二叉树中两个节点 `p` 和 `q`，找出最深的那个节点，使它同时是 `p` 和 `q` 的祖先。暴力想法是对每个候选节点分别判断它的子树里是否包含 `p` 和 `q`，这会产生大量重复搜索。经典递归解法会让子树向父节点汇报“是否找到了目标”，但同样存在深树调用栈问题。本书给出显式父指针版本：先遍历树，记录每个节点的父节点；再把 `p` 的所有祖先放入集合；最后从 `q` 不断向上走，第一次遇到的已访问祖先就是答案。

Listing 9.5 最近公共祖先：父指针和祖先集合

```

TreeNode* lowest_common_ancestor(TreeNode* root, TreeNode* p, TreeNode* q)
{
    if (root == nullptr || p == nullptr || q == nullptr) {
        return nullptr;
    }

    std::unordered_map<TreeNode*, TreeNode*> parent;
    std::vector<TreeNode*> stack{root};
    parent.emplace(root, nullptr);

    while (!stack.empty() && (!parent.contains(p) || !parent.contains(q))) {
        TreeNode* node = stack.back();
        stack.pop_back();

        if (node->left != nullptr) {
            parent.emplace(node->left, node);
            stack.push_back(node->left);
        }
        if (node->right != nullptr) {
            parent.emplace(node->right, node);
            stack.push_back(node->right);
        }
    }

    std::unordered_set<TreeNode*> ancestors;
    TreeNode* current = p;
    while (current != nullptr) {
        ancestors.insert(current);
        current = parent[current];
    }

    current = q;
    while (current != nullptr) {
        if (ancestors.contains(current)) {
            return current;
        }
        current = parent[current];
    }

    return nullptr;
}

```

为什么从 q 向上遇到的第一个公共祖先就是“最近”的？因为向上走的顺序是从深到浅，越早遇到越靠近 q ，同时它已经在 p 的祖先集合中。这个算法时间 $O(n)$ ，空间 $O(n)$ 。如果题目保证 p 和 q 一定在树中，前面的遍历可以在二者父指针都找到后提前停止；如果不保证，就需要完整遍历并检查二者是否出现。常见错误是把“节点值相同”当成“节点相同”，但力扣 236 给的是节点指针，应该比较节点地址。

岛屿数量把网格转成无向图。节点是值为 '1' 的格子，边是上下左右相邻的陆地。题目要求岛屿个数，就是要求连通块个数。暴力思路可能会对每个陆地格子都搜索一次，看它属于哪个岛，这会反复走过同一片陆地。正确优化是：扫描网格时，只要遇到一块尚未访问的陆地，就说明发现了一个新连通块；从它出发把整个连通块标记掉，答案加一。

Listing 9.6 岛屿数量：显式栈标记连通块

```
int num_islands(std::vector<std::vector<char>>& grid)
{
    if (grid.empty() || grid[0].empty()) {
        return 0;
    }

    constexpr char LAND = '1';
    constexpr char WATER = '0';
    constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
        std::pair{1, 0}, std::pair{-1, 0},
        std::pair{0, 1}, std::pair{0, -1}
    };

    const int rows = static_cast<int>(grid.size());
    const int cols = static_cast<int>(grid[0].size());
    int islands = 0;

    for (int row = 0; row < rows; ++row) {
        for (int col = 0; col < cols; ++col) {
            if (grid[row][col] != LAND) {
                continue;
            }

            ++islands;
            std::vector<std::pair<int, int>> stack{{row, col}};
            grid[row][col] = WATER;

            while (!stack.empty()) {
                const auto [r, c] = stack.back();
                stack.pop_back();

                for (const auto [dr, dc] : DIRECTIONS) {
                    const int next_row = r + dr;
                    const int next_col = c + dc;
                    const bool out_of_bounds =
                        next_row < 0 || next_row >= rows ||
                        next_col < 0 || next_col >= cols;
                    if (out_of_bounds || grid[next_row][next_col] != LAND) {
                        continue;
                    }

                    grid[next_row][next_col] = WATER;
                    stack.emplace_back(next_row, next_col);
                }
            }
        }
    }

    return islands;
}
```

这个实现把访问过的陆地直接改成水，因此不需要额外 `visited` 数组。如果题目不允许修改输入，就用同样大小的布尔矩阵保存访问状态。不变量是：扫描到任意位置时，所有已经发现过的岛都被完全标记为水，尚未被扫描到的陆地如果仍为 '1'，它一定属于一个尚未计数的新岛。时间复杂度 $O(mn)$ ，空间复杂度最坏 $O(mn)$ ，来自显式栈。易错点是访问标记时机：应该在入栈或入队时

立刻标记，而不是弹出后再标记，否则同一格子可能被多个邻居重复加入。

腐烂橘子是多源 BFS。每个新鲜橘子变烂的时间，等于它到最近初始腐烂橘子的最短四联通距离。暴力想法是每一分钟扫描整张网格，看哪些新鲜橘子旁边有腐烂橘子。这种写法虽然直观，却会反复扫描空格、已经腐烂的格子 and 暂时不会变化的格子。优化方法是把所有初始腐烂橘子同时入队，只从当前腐烂边界向外扩散。

Listing 9.7 腐烂橘子：多源 BFS 计算最短腐烂时间

```
int oranges_rotting(std::vector<std::vector<int>>& grid)
{
    if (grid.empty() || grid[0].empty()) {
        return 0;
    }

    constexpr int EMPTY = 0;
    constexpr int FRESH = 1;
    constexpr int ROTTEN = 2;
    constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
        std::pair{1, 0}, std::pair{-1, 0},
        std::pair{0, 1}, std::pair{0, -1}
    };

    const int rows = static_cast<int>(grid.size());
    const int cols = static_cast<int>(grid[0].size());
    std::queue<std::pair<int, int>> queue;
    int fresh_count = 0;

    for (int row = 0; row < rows; ++row) {
        for (int col = 0; col < cols; ++col) {
            if (grid[row][col] == ROTTEN) {
                queue.emplace(row, col);
            } else if (grid[row][col] == FRESH) {
                ++fresh_count;
            }
        }
    }

    int minutes = 0;
    while (!queue.empty() && fresh_count > 0) {
        const int level_size = static_cast<int>(queue.size());
        for (int step = 0; step < level_size; ++step) {
            const auto [row, col] = queue.front();
            queue.pop();

            for (const auto [dr, dc] : DIRECTIONS) {
                const int next_row = row + dr;
                const int next_col = col + dc;
                const bool out_of_bounds =
                    next_row < 0 || next_row >= rows ||
                    next_col < 0 || next_col >= cols;
                if (out_of_bounds || grid[next_row][next_col] != FRESH) {
                    continue;
                }

                grid[next_row][next_col] = ROTTEN;
                --fresh_count;
                queue.emplace(next_row, next_col);
            }
        }
        ++minutes;
    }

    return fresh_count == 0 ? minutes : -1;
}
```

这里 `level_size` 表示同一分钟内会继续扩散的腐烂橘子数量。处理完这一层后，才让 `minutes` 加一。初始没有新鲜橘子时，答案是 0；队列空了但仍有新鲜橘子时，说明有橘子被空格隔开，永远不会腐烂，答案是 -1。代码中 `EMPTY` 没有参与判断，但保留常量能让输入语义更清楚。复杂度仍是 $O(mn)$ ，因为每个格子最多入队一次。

课程表是有向图判环。每门课是节点，先修关系 `[course, prerequisite]` 表示必须先学

`prerequisite`，再学 `course`，所以边的方向应是 `prerequisite -> course`。暴力想法是反复扫描所有课程，找当前没有未完成先修课的课程；每学一门课又重新检查所有依赖，容易写成 $O(nm)$ 。拓扑排序的优化点是入度数组：`indegree[x]` 表示课程 `x` 还有多少前置依赖没有被满足。

Listing 9.8 课程表 II：拓扑排序输出可行学习顺序

```
std::vector<int> find_order(
    int num_courses,
    const std::vector<std::vector<int>>& prerequisites)
{
    std::vector<std::vector<int>> graph(num_courses);
    std::vector<int> indegree(num_courses, 0);

    for (const auto& edge : prerequisites) {
        const int course = edge[0];
        const int prerequisite = edge[1];
        graph[prerequisite].push_back(course);
        ++indegree[course];
    }

    std::queue<int> queue;
    for (int course = 0; course < num_courses; ++course) {
        if (indegree[course] == 0) {
            queue.push(course);
        }
    }

    std::vector<int> order;
    order.reserve(num_courses);

    while (!queue.empty()) {
        const int course = queue.front();
        queue.pop();
        order.push_back(course);

        for (const int next : graph[course]) {
            --indegree[next];
            if (indegree[next] == 0) {
                queue.push(next);
            }
        }
    }

    if (static_cast<int>(order.size()) != num_courses) {
        return {};
    }
    return order;
}
```

这段代码同时覆盖力扣 207 和 210。207 只问能否完成所有课程，可以返回 `order.size() == num_courses`；210 要返回一个具体顺序，若有环则返回空数组。正确性的核心是：入度为 0 的节点当前没有任何未满足依赖，可以安全学习；学习它只会减少它指向的后继课程入度，不会影响其他边。每个节点最多入队一次，每条边处理一次，复杂度 $O(n + m)$ 。最常见错误是把边方向建反。边建反时，有时仍然能判断是否有环，但输出顺序会变成反向，面试中必须主动说清方向。

单词接龙是无权图最短路。节点是单词，两个单词只差一个字符就连一条边；答案是起点词到终点词的最短转换长度。直接建图的暴力方法是枚举所有单词对，逐字符比较是否只差一位，复杂度约为 $O(n^2L)$ ，其中 L 是单词长度。BFS 本身没有问题，慢在找邻居。

优化思路是用通配模式把邻居查询变成哈希查询。比如 `hot` 可以生成 `*ot`、`h*t`、`ho*` 三个模式；所有能匹配同一模式的单词，彼此之间只差一个字符。预处理时把模式映射到单词列表，BFS 时对当前单词生成模式，从桶里取候选邻居。这样避免了每一步都扫描全词表。

Listing 9.9 单词接龙：通配模式加 BFS

```
int ladder_length(const std::string& begin_word,
    const std::string& end_word,
    const std::vector<std::string>& word_list)
{
    constexpr char WILDCARD = '*';
```

```

const int word_length = static_cast<int>(begin_word.size());

std::unordered_set<std::string> dictionary(
    word_list.begin(), word_list.end());
if (!dictionary.contains(end_word)) {
    return 0;
}

std::unordered_map<std::string, std::vector<std::string>> buckets;
for (const std::string& word : word_list) {
    for (int i = 0; i < word_length; ++i) {
        std::string pattern = word;
        pattern[i] = WILDCARD;
        buckets[pattern].push_back(word);
    }
}

std::queue<std::pair<std::string, int>> queue;
std::unordered_set<std::string> visited;
queue.emplace(begin_word, 1);
visited.insert(begin_word);

while (!queue.empty()) {
    const auto [word, distance] = queue.front();
    queue.pop();
    if (word == end_word) {
        return distance;
    }

    for (int i = 0; i < word_length; ++i) {
        std::string pattern = word;
        pattern[i] = WILDCARD;
        auto found = buckets.find(pattern);
        if (found == buckets.end()) {
            continue;
        }

        for (const std::string& next : found->second) {
            if (visited.contains(next)) {
                continue;
            }
            visited.insert(next);
            queue.emplace(next, distance + 1);
        }
        found->second.clear();
    }
}

return 0;
}

```

这题的 BFS 不变量是：队列中每个状态的 `distance` 都是从起点到该单词的最短转换长度；第一次到达 `endWord` 时，答案已经最短。为什么可以在访问后清空模式桶？因为一个模式桶只需要被展开一次。之后再遇到同一模式，桶里的所有邻居要么已经访问，要么已经入队；重复扫描只会增加时间。预处理复杂度约为 $O(nL^2)$ ，因为每次生成模式会复制长度为 `L` 的字符串；BFS 展开在清桶优化后也是同量级。若要继续优化，可以把单词编号化，模式只保存编号，但面试中这个版本已经足够清晰。

树图题复盘模板

每道树图题都先写四句话：节点是什么，边是什么，状态里保存什么，访问标记何时设置。二叉树题再补一句返回值语义或队列不变量；图题再补一句为什么 BFS 第一次到达就是最短，或者为什么 DFS 标记完整连通块。写完代码后，用空输入、单节点、链式树、完全树、孤立节点、有环依赖、不可达终点各测一遍。能把这些边界说清楚，才算真正掌握，而不是只背了遍历模板。

回溯、枚举和搜索空间剪枝

回溯题表面上是在“试所有可能”，本质上是在一棵决策树上搜索。每一层做一个选择，每条从根到叶子的路径代表一个候选答案。真正要学会的不是某个递归模板，而是四个问题：当前层决定什么，已经选择了什么，下一步还能选什么，什么时候可以停止或剪枝。只要这四件事说不清，代码再短也只是碰巧通过样例。

暴力枚举和回溯的边界并不绝对。最朴素的暴力会把所有组合全部生成出来，再逐个检查是否合法；回溯则把“合法性判断”提前到生成过程中。一旦发现当前前缀已经不可能形成答案，就不再向下扩展。这个动作叫剪枝。剪枝不是为了让代码看起来高级，而是减少决策树里根本不需要访问的分支。

C++面试题常用递归表达回溯，因为调用栈天然保存了“走到第几层、下一步尝试哪个分支、回退后恢复什么状态”。但递归深度不受控时会带来工程风险。本章讲递归心智模型，代码尽量把栈帧显式写出来，让搜索过程、状态恢复和边界条件更可见。这样写会比递归略长，但它能训练你看清楚回溯真正维护的状态。

全排列是回溯的第一道标准题。题目给定互不相同的数字，要求返回所有排列。暴力想法是枚举长度为 n 的所有序列，再检查是否每个数字恰好使用一次。若把每个位置都看成有 n 种选择，总分支数是 n^n ，绝大多数序列会因为重复使用元素而被丢弃。优化方法是在生成过程中维护 `used` 数组，保证同一个下标不会被选两次。这样搜索树只保留真正的排列分支，叶子数变成 $n!$ 。

下面的实现用显式栈模拟递归。每个栈帧保存当前层接下来要尝试的候选下标 `next_index`。进入下一层时，把选择加入 `path` 并标记 `used`；一层候选全部尝试完时，弹出栈帧，并撤销上一层做出的选择。

Listing 10.1 全排列：显式栈模拟回溯

```
std::vector<std::vector<int>> permute(const std::vector<int>& nums)
{
    struct Frame {
        int next_index = 0;
    };

    const int n = static_cast<int>(nums.size());
    std::vector<std::vector<int>> answer;
    std::vector<int> path;
    std::vector<char> used(n, false);
    std::vector<Frame> stack{{0}};

    while (!stack.empty()) {
        if (static_cast<int>(path.size()) == n) {
            answer.push_back(path);
            stack.pop_back();
            if (!path.empty()) {
                const int last_value = path.back();
                path.pop_back();
                const auto found = std::find(nums.begin(), nums.end(), last_value);
                used[static_cast<int>(found - nums.begin())] = false;
            }
            continue;
        }

        Frame& frame = stack.back();
        while (frame.next_index < n && used[frame.next_index]) {
            ++frame.next_index;
        }
        if (frame.next_index == n) {
            stack.pop_back();
            if (!path.empty()) {
                const int last_value = path.back();
                path.pop_back();
            }
        }
    }
}
```

```

        const auto found = std::find(nums.begin(), nums.end(), last_value);
        used[static_cast<int>(found - nums.begin())] = false;
    }
    continue;
}

const int chosen_index = frame.next_index;
++frame.next_index;
used[chosen_index] = true;
path.push_back(nums[chosen_index]);
stack.push_back(Frame{0});
}

return answer;
}

```

这段代码为了突出栈帧，撤销选择时用值反查下标；在生产代码里可以把“本层选择的下标”也放进下一层栈帧，避免反查。它的时间复杂度必须按输出规模理解：一共有 $n!$ 个排列，每个排列复制长度为 n 的路径，所以输出成本是 $O(n \cdot n!)$ 。额外空间是路径、标记和栈，都是 $O(n)$ ，不计答案。复盘这题时要能说清楚：`used` 是路径合法性的约束，`path.size() == n` 是叶子条件，回退必须撤销最后一次选择。

如果输入含重复数字，全排列 II 的难点就变成去重。很多人会先生成所有排列，再放进 `set` 去重；这当然能做，但浪费大量重复分支。更好的方法是先排序，然后在同一层跳过“相同数值且前一个相同值还没有被本路径使用”的候选。这个条件的含义是：同一层里，相同值只允许最左边那个代表这一类选择，避免生成形如交换两个相同值位置的重复排列。

Listing 10.2 全排列 II：排序后按层去重

```

std::vector<std::vector<int>> permute_unique(std::vector<int> nums)
{
    std::sort(nums.begin(), nums.end());

    const int n = static_cast<int>(nums.size());
    std::vector<std::vector<int>> answer;
    std::vector<int> path;
    std::vector<char> used(n, false);

    struct State {
        std::vector<int> path;
        std::vector<char> used;
    };

    std::vector<State> stack{{path, used}};
    while (!stack.empty()) {
        State state = std::move(stack.back());
        stack.pop_back();

        if (static_cast<int>(state.path.size()) == n) {
            answer.push_back(std::move(state.path));
            continue;
        }

        for (int i = n - 1; i >= 0; --i) {
            if (state.used[i]) {
                continue;
            }
            if (i > 0 && nums[i] == nums[i - 1] && !state.used[i - 1]) {
                continue;
            }

            State next = state;
            next.used[i] = true;
            next.path.push_back(nums[i]);
            stack.push_back(std::move(next));
        }
    }

    return answer;
}

```

这个版本为了让去重条件清楚，直接把状态复制进栈，适合教材解释；高性能版本会用一份路径和标记做原地回退。去重条件最容易写反。排序后，相同数字挨在一起，`!state.used[i - 1]` 表示前一个相同数字在当前层还没有被拿来作为代表，此时选择后一个会制造重复分支，所以跳过。时间复杂度仍然与唯一排列数量成正比，最坏不超过 $O(n \cdot n!)$ 。

子集题的决策树更简单。每个元素只有两种选择：选或者不选。暴力枚举可以用二进制掩码，从 0 到 $2^n - 1$ ，第 i 位表示是否选择第 i 个元素。回溯写法则是沿着数组位置向后决定。两种方法本质相同，都是遍历大小为 2^n 的搜索空间。

Listing 10.3 子集：迭代扩展已有答案

```
std::vector<std::vector<int>> subsets(const std::vector<int>& nums)
{
    std::vector<std::vector<int>> answer{{}};

    for (const int x : nums) {
        const int old_size = static_cast<int>(answer.size());
        for (int i = 0; i < old_size; ++i) {
            std::vector<int> next = answer[i];
            next.push_back(x);
            answer.push_back(std::move(next));
        }
    }

    return answer;
}
```

这段代码的不变量是：处理完前 i 个元素后，`answer` 包含这些元素的全部子集。加入新元素 x 时，旧子集保持“不选 x ”的版本；每个旧子集复制一份并加上 x ，得到“选 x ”的版本。这个算法没有剪枝，因为所有子集都是合法答案。时间复杂度 $O(n2^n)$ ，空间复杂度同样由输出规模主导。

组合总和比子集多了一个目标约束。题目给定候选数组和目标值，要求找出所有和为目标值的组合，同一个数字可以重复使用。暴力思路是不断选数字，直到长度达到某个上限再检查和；问题是上限不好定，而且大量分支会在总和已经超过目标后继续扩展。优化来自两个事实：候选数字都是正数；排序后，如果当前候选已经大于剩余目标，后面的更大候选也不可能使用。

Listing 10.4 组合总和：显式状态保存起点和剩余目标

```
std::vector<std::vector<int>> combination_sum(std::vector<int> candidates,
                                             int target)
{
    std::sort(candidates.begin(), candidates.end());

    struct State {
        int start = 0;
        int remaining = 0;
        std::vector<int> path;
    };

    std::vector<std::vector<int>> answer;
    std::vector<State> stack{{0, target, {}}};

    while (!stack.empty()) {
        State state = std::move(stack.back());
        stack.pop_back();

        if (state.remaining == 0) {
            answer.push_back(std::move(state.path));
            continue;
        }

        for (int i = static_cast<int>(candidates.size()) - 1;
             i >= state.start;
             --i) {
            if (candidates[i] > state.remaining) {
                continue;
            }
            State next = state;
            next.path.push_back(candidates[i]);
            next.remaining -= candidates[i];
        }
    }
}
```

```

        next.start = i;
        stack.push_back(std::move(next));
    }
}

return answer;
}

```

`start` 的语义很重要：组合不关心顺序，`[2,3,2]` 和 `[2,2,3]` 是同一个答案，因此后续只能从当前下标及其右侧继续选，避免同一组合以不同顺序重复出现。因为允许重复使用当前数字，下一层的 `start` 仍然是 `i`；如果题目变成每个数字只能用一次，就应设为 `i + 1`。这就是组合类题最核心的变体开关。复杂度取决于目标值和候选分布，不能简单写成多项式；面试里应说明它是指数级搜索，但剪枝能显著减少无效分支。

括号生成的关键不是枚举所有长度为 $2n$ 的括号串再检查。那样有 2^{2n} 个候选，大多数不合法。合法前缀必须满足两个计数约束：左括号数量不能超过 `n`；任何前缀中右括号数量不能超过左括号数量。把这两个约束提前放进生成过程，就不会产生明显非法的前缀。

Listing 10.5 括号生成：前缀合法性剪枝

```

std::vector<std::string> generate_parenthesis(int n)
{
    struct State {
        std::string text;
        int left = 0;
        int right = 0;
    };

    std::vector<std::string> answer;
    std::vector<State> stack{{"", 0, 0}};

    while (!stack.empty()) {
        State state = std::move(stack.back());
        stack.pop_back();

        if (static_cast<int>(state.text.size()) == 2 * n) {
            answer.push_back(std::move(state.text));
            continue;
        }

        if (state.right < state.left) {
            State next = state;
            next.text.push_back(')');
            ++next.right;
            stack.push_back(std::move(next));
        }
        if (state.left < n) {
            State next = state;
            next.text.push_back('(');
            ++next.left;
            stack.push_back(std::move(next));
        }
    }

    return answer;
}

```

这题的正确性可以用前缀不变量解释：任何时刻 `left <= n` 且 `right <= left`。如果 `right > left`，说明某个右括号没有左括号匹配，后面再加字符也无法修复这个非法前缀；如果 `left > n`，也已经超过题目允许。输出数量是第 `n` 个卡特兰数，算法复杂度至少与答案数量成正比。易错点是终止条件：必须长度达到 $2n$ 才能加入答案，只看 `left == right` 会把短前缀误加进去。

单词搜索把回溯放进网格图。节点是格子，边是上下左右相邻，路径不能重复使用同一个格子。暴力想法是从每个格子出发尝试所有方向，最后检查路径字符串；优化点是边走边比较目标单词的当前位置，一旦字符不匹配立即停止。访问标记也必须属于当前路径，而不是全局永久标记，因为同一个格子可以出现在不同起点或不同路径中。

Listing 10.6 单词搜索：显式栈保存路径访问状态

```

bool exist(std::vector<std::vector<char>>& board, const std::string& word)
{
    if (word.empty()) {
        return true;
    }
    if (board.empty() || board[0].empty()) {
        return false;
    }

    constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
        std::pair{1, 0}, std::pair{-1, 0},
        std::pair{0, 1}, std::pair{0, -1}
    };

    const int rows = static_cast<int>(board.size());
    const int cols = static_cast<int>(board[0].size());

    struct State {
        int row = 0;
        int col = 0;
        int index = 0;
        std::vector<std::vector<char>> visited;
    };

    for (int row = 0; row < rows; ++row) {
        for (int col = 0; col < cols; ++col) {
            if (board[row][col] != word[0]) {
                continue;
            }

            std::vector<std::vector<char>> visited(rows, std::vector<char>(cols, false));
            visited[row][col] = true;
            std::vector<State> stack{{row, col, 0, std::move(visited)}};

            while (!stack.empty()) {
                State state = std::move(stack.back());
                stack.pop_back();
                if (state.index + 1 == static_cast<int>(word.size())) {
                    return true;
                }

                for (const auto [dr, dc] : DIRECTIONS) {
                    const int next_row = state.row + dr;
                    const int next_col = state.col + dc;
                    const int next_index = state.index + 1;
                    const bool out_of_bounds =
                        next_row < 0 || next_row >= rows ||
                        next_col < 0 || next_col >= cols;
                    if (out_of_bounds || state.visited[next_row][next_col] ||
                        board[next_row][next_col] != word[next_index]) {
                        continue;
                    }

                    State next = state;
                    next.row = next_row;
                    next.col = next_col;
                    next.index = next_index;
                    next.visited[next_row][next_col] = true;
                    stack.push_back(std::move(next));
                }
            }
        }
    }

    return false;
}

```

这个教材版把 **visited** 放进状态里，语义最清楚，但会复制较多布尔矩阵；原地回退版性能更好。本题最坏复杂度接近 $O(mn \cdot 3^L)$ ：第一步最多从每个格子出发，之后每一步通常不能回到上一个格子，所以分支近似 3， L 是单词长度。常见错误是把访问标记设成全局，导致某个格子被一条失败路径占用后，其他合法路径无法使用它。

N 皇后是回溯剪枝的代表。暴力方法在 $n \times n$ 棋盘上选 n 个格子，再检查行、列、对角线是否冲突，搜索空间巨大。优化方式是按行放皇后：第 row 层只决定第 row 行皇后放在哪一列。这样行冲突天然消失，只需要维护列、主对角线和副对角线是否被占用。

Listing 10.7 N 皇后：按行放置并维护三类占用

```
std::vector<std::vector<std::string>> solve_n_queens(int n)
{
    struct State {
        int row = 0;
        std::vector<int> queens;
        std::vector<char> cols;
        std::vector<char> diag1;
        std::vector<char> diag2;
    };

    std::vector<std::vector<std::string>> answer;
    std::vector<State> stack{{0,
        {},
        std::vector<char>(n, false),
        std::vector<char>(2 * n - 1, false),
        std::vector<char>(2 * n - 1, false)}};

    while (!stack.empty()) {
        State state = std::move(stack.back());
        stack.pop_back();

        if (state.row == n) {
            std::vector<std::string> board(n, std::string(n, '.'));
            for (int row = 0; row < n; ++row) {
                board[row][state.queens[row]] = 'Q';
            }
            answer.push_back(std::move(board));
            continue;
        }

        for (int col = n - 1; col >= 0; --col) {
            const int main_diag = state.row - col + n - 1;
            const int anti_diag = state.row + col;
            if (state.cols[col] || state.diag1[main_diag] ||
                state.diag2[anti_diag]) {
                continue;
            }

            State next = state;
            next.queens.push_back(col);
            next.cols[col] = true;
            next.diag1[main_diag] = true;
            next.diag2[anti_diag] = true;
            ++next.row;
            stack.push_back(std::move(next));
        }
    }

    return answer;
}
```

主对角线上的格子满足 $row - col$ 相同；为了把负数下标变成数组下标，代码加上 $n - 1$ 。副对角线满足 $row + col$ 相同，范围天然从 0 到 $2n - 2$ 。这题的面试重点不是把棋盘画出来，而是把冲突检测从扫描整张棋盘优化成三个数组的常数查询。复杂度仍然是指数级，但剪枝把不可行分支尽早挡在下一层之外。

回溯题复盘模板

回溯题不要先背代码。先画出决策树的一层，写清本层选择、路径状态、可选集合、终止条件和剪枝条件。组合题重点看 **start** 是否允许重复使用当前元素；排列题重点看 **used** 和同层去重；网格题重点看访问标记是路径级还是全局级；括号和棋盘题重点把非法前缀提前剪掉。最后再问一句：答案数量本身有多大？如果答案就是指数级，复杂度不可能被写成线性或多项式。

第 11 章

动态规划：状态、转移和重复子问题

动态规划不是“看到最值就开数组”。它解决的是有重叠子问题、且大问题能由小问题稳定推出的问题。暴力搜索之所以慢，是因为同一个子问题被不同路径反复计算；动态规划的优化，就是给子问题命名、保存结果，并按照依赖顺序把结果推出来。学 DP 最重要的五件事是：状态表示什么，答案在哪里，初始状态是什么，转移从哪里来，遍历顺序为什么满足依赖。

爬楼梯是最小的 DP 模型。题目问到达第 n 阶有多少种方法，每次可以爬 1 或 2 阶。暴力递归会写成 $\text{ways}(n) = \text{ways}(n - 1) + \text{ways}(n - 2)$ ，但 $\text{ways}(n - 2)$ 、 $\text{ways}(n - 3)$ 等子问题会被反复计算，形成指数级时间。优化方法是从小到大保存每个阶数的答案。

Listing 11.1 爬楼梯：滚动变量保存最近两个状态

```
int climb_stairs(int n)
{
    if (n <= 2) {
        return n;
    }

    int previous_two = 1;
    int previous_one = 2;
    for (int step = 3; step <= n; ++step) {
        const int current = previous_one + previous_two;
        previous_two = previous_one;
        previous_one = current;
    }

    return previous_one;
}
```

这里状态可以写成 $\text{dp}[i]$ ：到达第 i 阶的方法数。最后一步要么从 $i - 1$ 走 1 阶，要么从 $i - 2$ 走 2 阶，因此转移是 $\text{dp}[i] = \text{dp}[i - 1] + \text{dp}[i - 2]$ 。由于每个状态只依赖前两个状态，可以把数组压成两个变量。时间复杂度 $O(n)$ ，空间复杂度 $O(1)$ 。这题的价值不在难度，而在于展示 DP 的完整骨架。

打家劫舍把“相邻不能同时选”的约束放进状态。暴力方法是对每间房决定偷或不偷，枚举 2^n 个子集，再过滤相邻冲突。更好的建模是让 $\text{dp}[i]$ 表示考虑前 i 间房能偷到的最大金额。第 i 间房只有两种结局：不偷它，答案是 $\text{dp}[i - 1]$ ；偷它，就不能偷第 $i - 1$ 间，答案是 $\text{dp}[i - 2] + \text{nums}[i - 1]$ 。

Listing 11.2 打家劫舍：选与不选的状态转移

```
int rob(const std::vector<int>& nums)
{
    int previous_two = 0;
    int previous_one = 0;

    for (const int money : nums) {
        const int current = std::max(previous_one, previous_two + money);
        previous_two = previous_one;
        previous_one = current;
    }

    return previous_one;
}
```

这段代码把 $\text{dp}[i - 2]$ 和 $\text{dp}[i - 1]$ 压成两个变量。遍历到当前房屋时，`previous_one` 是不偷当前房屋时可继承的最优值，`previous_two + money` 是偷当前房屋时的最优值。正确性来自最优子结构：当前决策之后，剩余可用的历史范围只与前一间或前两间有关，不需要知道更早的具体选择。易错点是下标：如果用数组，建议定义 $\text{dp}[0] = 0$ ， $\text{dp}[1] = \text{nums}[0]$ ，这样第 i 个状态表

示前 i 间房，避免把房屋下标和状态下标混在一起。

零钱兑换展示“最少数量”型 DP。题目给硬币面额和目标金额，要求凑成目标金额所需的最少硬币数。暴力搜索会尝试所有硬币序列，许多不同顺序会到达同一个剩余金额，例如先选 1 再选 2，与先选 2 再选 1，都可能进入同一个子问题。状态应定义为 $dp[amount]$ ：凑出金额 $amount$ 的最少硬币数。

Listing 11.3 零钱兑换：从可达金额推出新金额

```
int coin_change(const std::vector<int>& coins, int amount)
{
    constexpr int INF = 1'000'000'000;
    std::vector<int> dp(amount + 1, INF);
    dp[0] = 0;

    for (int value = 1; value <= amount; ++value) {
        for (const int coin : coins) {
            if (coin > value || dp[value - coin] == INF) {
                continue;
            }
            dp[value] = std::min(dp[value], dp[value - coin] + 1);
        }
    }

    return dp[amount] == INF ? -1 : dp[amount];
}
```

转移的含义是：如果最后一枚硬币是 $coin$ ，那么前面必须先凑出 $value - coin$ ，所以候选答案是 $dp[value - coin] + 1$ 。这里用一个足够大的 INF 表示不可达，而不是用 -1 参与最小值计算，因为 $-1 + 1$ 会污染转移。时间复杂度 $O(amount \cdot c)$ ， c 是硬币种类数。完全背包类题还要区分“组合数”和“排列数”：本题只问最少硬币数，内外循环顺序不会影响最优值；如果问方案数量，循环顺序就会改变计数语义。

最长递增子序列有两种经典解法。第一种 DP 定义 $dp[i]$ 为以 $nums[i]$ 结尾的最长递增子序列长度。为了求它，需要看所有 $j < i$ 且 $nums[j] < nums[i]$ 的位置，转移为 $dp[i] = \max(dp[j] + 1)$ 。这比枚举所有子序列的 2^n 好很多，但仍是 $O(n^2)$ 。

Listing 11.4 最长递增子序列：二次 DP

```
int length_of_lis_dp(const std::vector<int>& nums)
{
    if (nums.empty()) {
        return 0;
    }

    std::vector<int> dp(nums.size(), 1);
    int answer = 1;

    for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
        for (int j = 0; j < i; ++j) {
            if (nums[j] < nums[i]) {
                dp[i] = std::max(dp[i], dp[j] + 1);
            }
        }
        answer = std::max(answer, dp[i]);
    }

    return answer;
}
```

第二种优化来自“只关心长度对应的最小结尾”。维护数组 $tails$ ，其中 $tails[len - 1]$ 表示长度为 len 的递增子序列中，最小可能结尾值。结尾越小，后面越容易接上更大的数。遍历新数 x 时，用二分找到第一个大于等于 x 的位置并替换；如果找不到，说明 x 可以扩展出更长序列。

Listing 11.5 最长递增子序列：贪心尾值加二分

```
int length_of_lis_binary_search(const std::vector<int>& nums)
{
    std::vector<int> tails;
```

```

for (const int x : nums) {
    auto position = std::lower_bound(tails.begin(), tails.end(), x);
    if (position == tails.end()) {
        tails.push_back(x);
    } else {
        *position = x;
    }
}

return static_cast<int>(tails.size());
}

```

这个算法经常被误解成 `tails` 本身就是最终的子序列。它不一定是。它保存的是每个长度下最有潜力的最小结尾。替换操作不会减少已经找到的长度，只会让同长度的结尾更小，从而给未来元素更多机会。时间复杂度 $O(n \log n)$ ，空间复杂度 $O(n)$ 。如果面试官追问为什么能替换，要回答：相同长度下，更小结尾不会比更大结尾更差，因为它能接纳的后续元素集合只会更多。

单词拆分把字符串前缀变成状态。题目给字符串和词典，问字符串能否被切成若干词典单词。暴力搜索会尝试每个切分点，遇到相同后缀时重复判断。状态 `dp[i]` 表示前 `i` 个字符是否可以被合法拆分。若存在 `j < i`，使得 `dp[j]` 为真且子串 `s[j..i)` 在词典中，则 `dp[i]` 为真。

Listing 11.6 单词拆分：前缀可达状态

```

bool word_break(const std::string& s, const std::vector<std::string>& word_dict)
{
    std::unordered_set<std::string> words(word_dict.begin(), word_dict.end());
    std::vector<char> dp(s.size() + 1, false);
    dp[0] = true;

    for (std::size_t right = 1; right <= s.size(); ++right) {
        for (std::size_t left = 0; left < right; ++left) {
            if (!dp[left]) {
                continue;
            }
            if (words.contains(s.substr(left, right - left))) {
                dp[right] = true;
                break;
            }
        }
    }

    return dp[s.size()];
}

```

这里 `dp[0] = true` 代表空前缀可拆分，是所有从 0 开始的单词能够成立的基础。代码直接使用 `substr`，写法清晰，但会复制子串；如果输入规模更大，可以限制最大单词长度，或者用字符串视图和自定义哈希、字典树来减少复制。复杂度通常写成 $O(n^3)$ 的保守形式，因为有两层切分枚举，子串构造和哈希也与长度有关；若把子串查询视作均摊常数，则是 $O(n^2)$ 。面试中最好主动说明这个实现的复制成本。

最长公共子序列是二维 DP 的代表。子序列可以删除字符但不能改变相对顺序。暴力方法会枚举两个字符串的所有子序列再比较，数量指数级。状态 `dp[i][j]` 表示 `text1` 的前 `i` 个字符和 `text2` 的前 `j` 个字符的最长公共子序列长度。

Listing 11.7 最长公共子序列：二维前缀 DP

```

int longest_common_subsequence(const std::string& text1,
                               const std::string& text2)
{
    const int rows = static_cast<int>(text1.size());
    const int cols = static_cast<int>(text2.size());
    std::vector<std::vector<int>> dp(rows + 1, std::vector<int>(cols + 1, 0));

    for (int i = 1; i <= rows; ++i) {
        for (int j = 1; j <= cols; ++j) {
            if (text1[i - 1] == text2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {

```

```

        dp[i][j] = std::max(dp[i - 1][j], dp[i][j - 1]);
    }
}

return dp[rows][cols];
}

```

为什么相等时来自左上角？因为当前两个字符可以配成公共子序列的最后一个字符，剩下问题变成两个前缀各去掉这个字符。为什么不等时取上方和左方最大值？因为两个末尾字符不能同时作为同一个结尾，至少要丢掉其中一个。多出来的第 0 行和第 0 列代表空串，避免了边界分支。时间复杂度 $O(mn)$ ，空间复杂度 $O(mn)$ ；如果只要长度，可以滚动数组压到 $O(n)$ 。

编辑距离比 LCS 更综合。题目允许插入、删除、替换，把一个单词变成另一个单词，求最少操作数。状态 $dp[i][j]$ 表示把 `word1` 的前 i 个字符变成 `word2` 的前 j 个字符的最少操作数。这个定义必须精确，因为三种操作都围绕“前缀末尾”展开。

Listing 11.8 编辑距离：插入、删除、替换三种转移

```

int min_distance(const std::string& word1, const std::string& word2)
{
    const int rows = static_cast<int>(word1.size());
    const int cols = static_cast<int>(word2.size());
    std::vector<std::vector<int>> dp(rows + 1, std::vector<int>(cols + 1, 0));

    for (int i = 0; i <= rows; ++i) {
        dp[i][0] = i;
    }
    for (int j = 0; j <= cols; ++j) {
        dp[0][j] = j;
    }

    for (int i = 1; i <= rows; ++i) {
        for (int j = 1; j <= cols; ++j) {
            if (word1[i - 1] == word2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1];
                continue;
            }

            const int delete_cost = dp[i - 1][j] + 1;
            const int insert_cost = dp[i][j - 1] + 1;
            const int replace_cost = dp[i - 1][j - 1] + 1;
            dp[i][j] = std::min({delete_cost, insert_cost, replace_cost});
        }
    }

    return dp[rows][cols];
}

```

初始化含义是：把长度为 i 的前缀变成空串，需要删除 i 次；把空串变成长度为 j 的前缀，需要插入 j 次。若末尾字符相同，不需要新增操作，直接继承左上角；若不同，删除对应上方，插入对应左方，替换对应左上角。时间和空间都是 $O(mn)$ 。易错点是从哪个字符串视角解释插入和删除：只要状态定义固定， $dp[i - 1][j] + 1$ 就是删掉 `word1` 的末尾， $dp[i][j - 1] + 1$ 就是在 `word1` 末尾插入 `word2[j - 1]`。

动态规划题经常可以做空间优化，但不要为了炫技先写滚动数组。二维表能展示依赖关系，滚动数组则要求你非常清楚哪些旧值还要用、哪些可以覆盖。面试时，先给出清晰状态和二维实现，再说明可压缩空间，通常比一上来写难读的一维代码更稳。

动态规划题复盘模板

每道 DP 题按五步复盘：第一，状态是什么，必须能用自然语言解释；第二，答案在 $dp[n]$ 、 $dp[m][n]$ 还是某个最大值里；第三，空状态和最小状态如何初始化；第四，转移枚举的是最后一步、最后一个选择，还是最后一段区间；第五，遍历顺序是否保证依赖状态已经算完。若你只能背出公式，却说不出公式对应的最后一步选择，这题就还没有真正掌握。

第零部分

面试专项：从题型到工程表达

第 12 章

线性结构专项：链表、栈、队列和单调结构

线性结构专项题看上去都很短，但它们是面试中最容易暴露基本功的一类。数组和字符串通常考察下标、窗口和局部状态；链表考察节点关系和指针顺序；栈考察最近未解决状态；队列考察层次推进；单调栈和单调队列考察候选淘汰。很多候选人的代码错在一个小边界，但真正的原因不是粗心，而是没有说清楚“当前结构里保存的对象到底代表什么”。

链表题首先要把“值”和“节点”分开。数组里交换两个元素，很多时候只要交换值；链表里更重要的是节点之间的连接关系。删除节点、反转链表、合并链表、K 个一组翻转，都不是在移动一段连续内存，而是在改变若干个 `next` 指针。写链表题之前，建议先在纸上画三段：已经处理好的部分、当前正在处理的部分、尚未处理的部分。每次改指针前都问一句：原来的后继是否已经保存？新的前驱是谁？最终头节点从哪里返回？

删除倒数第 N 个节点的暴力想法是先遍历一遍得到长度 `len`，再删除第 `len - n + 1` 个节点。这个做法完全正确，复杂度也是 $O(n)$ ，但需要两次遍历。面试更常希望你展示快慢指针：让快指针先走 `n` 步，然后快慢一起走；当快指针到达尾部时，慢指针正好停在待删除节点的前驱。优化的本质不是少了一行代码，而是把“倒数位置”转换成“两个指针之间保持固定距离”的不变量。

Listing 12.1 删除倒数第 N 个节点：虚拟头节点和快慢指针

```
struct ListNode {
    int value = 0;
    ListNode* next = nullptr;
};

ListNode* remove_nth_from_end(ListNode* head, int n)
{
    ListNode dummy;
    dummy.next = head;

    ListNode* fast = &dummy;
    ListNode* slow = &dummy;

    for (int step = 0; step < n; ++step) {
        fast = fast->next;
    }
    while (fast->next != nullptr) {
        fast = fast->next;
        slow = slow->next;
    }

    ListNode* removed = slow->next;
    slow->next = removed->next;
    return dummy.next;
}
```

虚拟头节点的价值在这里非常明显。如果删除的是原来的头节点，没有虚拟头，就要单独写一段分支；有了虚拟头，删除头节点和删除中间节点统一成“修改某个前驱节点的 `next`”。这类题的面试表达要明确：快慢指针之间相隔 `n` 个真实节点，循环结束时快指针停在最后一个真实节点，慢指针停在待删节点前驱。边界要测三个：只有一个节点，删除头节点，删除尾节点。

合并两个有序链表是链表和双指针结合的题。暴力方法可以把两个链表的值全部放进数组，排序后重新建链表；这样会丢掉链表题真正考察的节点重连能力，也多用了 $O(n)$ 额外空间。更自然的做法是维护一个结果链表尾指针，每次从两个链表头部取较小节点接到尾部。由于两个输入链表本来有序，每一步的局部最小节点就是全局下一个节点。

Listing 12.2 合并两个有序链表：尾指针接入节点

```
ListNode* merge_two_lists(ListNode* first, ListNode* second)
{

```

```

ListNode dummy;
ListNode* tail = &dummy;

while (first != nullptr && second != nullptr) {
    if (first->value <= second->value) {
        tail->next = first;
        first = first->next;
    } else {
        tail->next = second;
        second = second->next;
    }
    tail = tail->next;
}

tail->next = first != nullptr ? first : second;
return dummy.next;
}

```

这段代码没有创建新节点，只是复用原链表节点。因此如果题目要求不能修改输入，就要改成新建节点版本。稳定性也值得说明：当两个节点值相等时，代码优先取 `first`，这会保持第一条链表中相等元素的相对顺序。复杂度是 $O(m+n)$ ，空间是 $O(1)$ 。错误高发点是忘记移动 `tail`，或者循环结束后忘记把剩余链表整体接上。

K 个一组翻转链表比普通反转多了“分组边界”。暴力想法是把所有节点放进数组，按 K 个一组交换指针或值，再重新连接；这样虽然容易写，但牺牲了链表题的原地处理能力。原地做法要先检查当前组是否有 K 个节点，不足 K 个就保持原样；足够 K 个时，只翻转这一段，并把上一组的尾部接到新头，把新尾接到下一组开头。

Listing 12.3 K 个一组翻转链表：先找边界再局部反转

```

ListNode* reverse_k_group(ListNode* head, int k)
{
    if (k <= 1 || head == nullptr) {
        return head;
    }

    ListNode dummy;
    dummy.next = head;
    ListNode* group_previous = &dummy;

    while (true) {
        ListNode* kth = group_previous;
        for (int step = 0; step < k && kth != nullptr; ++step) {
            kth = kth->next;
        }
        if (kth == nullptr) {
            break;
        }

        ListNode* group_next = kth->next;
        ListNode* previous = group_next;
        ListNode* current = group_previous->next;

        while (current != group_next) {
            ListNode* next = current->next;
            current->next = previous;
            previous = current;
            current = next;
        }

        ListNode* new_group_tail = group_previous->next;
        group_previous->next = kth;
        group_previous = new_group_tail;
    }

    return dummy.next;
}

```

这题最重要的是命名。没有 `group_previous`、`kth`、`group_next` 这三个边界名，代码会变成一堆难以验证的指针操作。反转循环里让 `previous` 初始等于 `group_next`，是为了反转完成后原组头

节点自然指向下一组开头。这个技巧能减少额外连接分支。面试时不要急着写代码，先画出一组翻转前后：上一组尾、当前组头、当前组尾、下一组头，四个位置说清楚后，代码才有依据。

栈题的核心是“最近未解决”。有效括号中，最近出现的左括号必须最先被匹配；路径简化中，最近进入的目录会被 `..` 回退；表达式求值中，最近的操作数和运算符构成局部归约。栈的优化不是为了降低所有问题的复杂度，而是把嵌套结构或最近依赖关系变成线性扫描。

最小栈是一个很好的设计题。只用一个普通栈保存元素，`top` 和 `pop` 都是常数时间，但 `getMin` 需要扫描所有元素。优化方法是额外维护一个同步最小值栈。每压入一个值，同时压入“截至当前栈顶的最小值”；弹出时两个栈一起弹。这样任意时刻最小值栈栈顶就是当前全部元素的最小值。

Listing 12.4 最小栈：数据栈和最小值栈同步

```
class MinStack {
public:
    void push(int value)
    {
        this->values_.push_back(value);
        if (this->minimums_.empty()) {
            this->minimums_.push_back(value);
        } else {
            this->minimums_.push_back(std::min(value, this->minimums_.back()));
        }
    }

    void pop()
    {
        this->values_.pop_back();
        this->minimums_.pop_back();
    }

    int top() const
    {
        return this->values_.back();
    }

    int get_min() const
    {
        return this->minimums_.back();
    }

private:
    std::vector<int> values_;
    std::vector<int> minimums_;
};
```

这段代码故意使用 `std::vector` 作为栈，因为 `vector` 支持连续存储、`back`、`push_back` 和 `pop_back`，足够表达栈语义。类成员访问使用 `this->`，避免成员和局部变量混淆。常见错误是只在新值更小时才压入最小值栈，但弹出重复最小值时就会错。例如依次压入 2, 2, 3，如果只记录一个 2，弹出一个 2 后最小值仍应是 2。同步栈写法用空间换清晰和可靠。

单调栈是“最近未解决”的强化版：栈里保存的不是所有历史元素，而是一组仍有可能成为答案的候选。每日温度中，栈保存还没找到更高温度的下标，且栈中温度从底到顶单调不增。新温度到来时，如果高于栈顶，就说明栈顶那一天的答案已经确定，因为当前天是它右侧第一个更高温度的位置。

柱状图中最大的矩形更能体现单调栈的威力。暴力做法枚举每根柱子作为矩形高度，再向左右扩展找到第一个更矮柱子，最坏 $O(n^2)$ 。优化的关键是：当我们知道某根柱子左右两侧第一个更矮的位置时，以它为最低高度的最大矩形宽度就确定了。单调递增栈可以在一次扫描中确定“右侧第一个更矮”。

Listing 12.5 柱状图最大矩形：单调递增栈

```
int largest_rectangle_area(const std::vector<int>& heights)
{
    std::vector<int> stack;
    stack.reserve(heights.size() + 1);
    int answer = 0;

    for (int i = 0; i <= static_cast<int>(heights.size()); ++i) {
        const int current_height =
```

```

        i == static_cast<int>(heights.size()) ? 0 : heights[i];

        while (!stack.empty() && heights[stack.back()] > current_height) {
            const int height = heights[stack.back()];
            stack.pop_back();
            const int left_less_index = stack.empty() ? -1 : stack.back();
            const int width = i - left_less_index - 1;
            answer = std::max(answer, height * width);
        }
        stack.push_back(i);
    }

    return answer;
}

```

这里末尾人为加入高度 0 的哨兵，目的是把栈里剩余柱子全部弹出结算。弹出某个下标时，当前下标 *i* 是它右侧第一个更矮的位置；弹出后新的栈顶是它左侧第一个更矮的位置。因此宽度是两者之间的柱子数量。这个解释比“套单调栈模板”重要得多。复杂度是 $O(n)$ ，因为每个下标最多入栈一次、出栈一次。易错点是比较符号：使用 $>$ 或 $>=$ 都能做，但对重复高度的归属不同，必须和宽度计算保持一致。

队列和双端队列负责维护“按时间进入、按窗口过期”的候选。滑动窗口最大值的暴力解对每个窗口扫描一遍最大值，复杂度 $O(nk)$ 。堆也能做，但需要处理过期下标。单调队列更贴合题目：队头永远是当前窗口最大值下标，队尾负责删除不可能再成为最大值的候选。

Listing 12.6 滑动窗口最大值：队头保存当前最大值下标

```

std::vector<int> max_sliding_window(const std::vector<int>& nums, int k)
{
    std::vector<int> answer;
    if (nums.empty() || k <= 0) {
        return answer;
    }

    std::deque<int> deque;
    for (int right = 0; right < static_cast<int>(nums.size()); ++right) {
        while (!deque.empty() && nums[deque.back()] <= nums[right]) {
            deque.pop_back();
        }
        deque.push_back(right);

        const int left = right - k + 1;
        if (deque.front() < left) {
            deque.pop_front();
        }
        if (left >= 0) {
            answer.push_back(nums[deque.front()]);
        }
    }
    return answer;
}

```

为什么队尾较小元素可以删除？设队尾下标为 *j*，新下标为 *right*，如果 $nums[j] \leq nums[right]$ ，那么在未来任何同时包含 *j* 和 *right* 的窗口里，*j* 都不可能比 *right* 更优；而且 *j* 更早过期。因此删除 *j* 不会影响任何未来答案。这就是候选淘汰的证明。单调队列的复盘要说清两个动作：入队前从队尾删掉被新元素支配的候选；出答案前从队头删掉已过期的候选。

环形链表是快慢指针的另一条主线。暴力方法可以把走过的节点地址放进哈希集合，每到一个节点就检查是否见过；如果见过，说明有环；如果走到空指针，说明无环。这个方法直观，时间 $O(n)$ ，空间 $O(n)$ 。快慢指针把空间降到常数：慢指针每次走一步，快指针每次走两步。如果没有环，快指针会先到空；如果有环，快指针进入环后会不断追赶慢指针，最终相遇。

Listing 12.7 环形链表：快慢指针检测相遇

```

bool has_cycle(ListNode* head)
{
    ListNode* slow = head;
    ListNode* fast = head;

```

```

while (fast != nullptr && fast->next != nullptr) {
    slow = slow->next;
    fast = fast->next->next;
    if (slow == fast) {
        return true;
    }
}

return false;
}

```

这题的正确性可以用相对速度解释。进入环之后，快指针相对慢指针每轮多走一步；环长度有限，因此相对距离会按环长度取模不断变化，必然出现 0，也就是相遇。不要把这题说成“快指针总会追上慢指针”就结束，最好补一句“因为在环内相对距离每次加一，对环长取模”。如果题目进一步要求找到入环节点，做法是在相遇后让一个指针回到头节点，两个指针都每次走一步，再次相遇的位置就是入口。这来自公式：头到入口距离加上入口到相遇点距离，与快慢指针走过路程差之间存在整环倍数关系。面试里不一定要完整推公式，但要知道这不是经验结论。

LRU 缓存是线性结构和哈希表组合的设计题。题目要求 `get` 和 `put` 都是 $O(1)$ ，并且容量满时淘汰最近最少使用的键。单独用数组或链表，可以维护使用顺序，但查找 key 是 $O(n)$ ；单独用哈希表可以常数查找，但无法常数时间移动“最近使用”的位置。正确组合是：双向链表维护从最近到最久的顺序，哈希表从 key 映射到链表节点。

Listing 12.8 LRU 缓存：哈希表定位节点，双向链表维护顺序

```

class LRUCache {
public:
    explicit LRUCache(int capacity)
        : capacity_(capacity)
    {
    }

    int get(int key)
    {
        const auto found = this->items_.find(key);
        if (found == this->items_.end()) {
            return -1;
        }
        this->touch(found->second);
        return found->second->second;
    }

    void put(int key, int value)
    {
        const auto found = this->items_.find(key);
        if (found != this->items_.end()) {
            found->second->second = value;
            this->touch(found->second);
            return;
        }

        this->entries_.emplace_front(key, value);
        this->items_[key] = this->entries_.begin();

        if (static_cast<int>(this->entries_.size()) > this->capacity_) {
            const int removed_key = this->entries_.back().first;
            this->items_.erase(removed_key);
            this->entries_.pop_back();
        }
    }

private:
    using Entry = std::pair<int, int>;
    using Iterator = std::list<Entry>::iterator;

    void touch(Iterator iterator)
    {
        this->entries_.splice(this->entries_.begin(), this->entries_, iterator);
    }

    int capacity_ = 0;

```

```
std::list<Entry> entries_;
std::unordered_map<int, Iterator> items_;
};
```

这段代码使用 `std::list`，不是因为链表在刷题中通常更快，而是因为 LRU 需要在已知节点迭代器的情况下常数时间移动节点到表头。`splice` 不复制节点，只改变链表连接关系；哈希表保存的迭代器仍指向同一节点。这里 `std::list` 的节点稳定性正好匹配需求。反过来，如果用 `vector` 保存顺序，移动到表头会导致大量元素移动，还可能让保存的下标或迭代器失效。LRU 的复盘重点是“为什么必须两个结构一起用”：哈希表负责定位，双向链表负责顺序，任何一个单独使用都达不到题目复杂度。

接雨水可以用多个思路解决。暴力方法枚举每个位置，向左找最高柱子，向右找最高柱子，该位置能接的水是左右最高的较小值减当前高度；每个位置都向两边扫描，复杂度 $O(n^2)$ 。前后缀数组可以把左右最高值预处理出来，时间降到 $O(n)$ ，空间 $O(n)$ 。双指针进一步把空间降到 $O(1)$ ：左右两侧各维护当前最高柱子，哪边较低，就结算哪边。

Listing 12.9 接雨水：双指针维护两侧最高柱

```
int trap(const std::vector<int>& height)
{
    int left = 0;
    int right = static_cast<int>(height.size()) - 1;
    int left_max = 0;
    int right_max = 0;
    int water = 0;

    while (left < right) {
        if (height[left] < height[right]) {
            left_max = std::max(left_max, height[left]);
            water += left_max - height[left];
            ++left;
        } else {
            right_max = std::max(right_max, height[right]);
            water += right_max - height[right];
            --right;
        }
    }

    return water;
}
```

为什么可以结算较低的一侧？假设 `height[left] < height[right]`，右侧至少存在一个高度大于左侧当前位置的柱子，因此当前位置水位的右边界不会低于 `height[left]`；它能接多少水只取决于左侧最高柱 `left_max`。如果左侧最高还不够高，就接不到水；如果够高，就能立刻计算。单调栈也能做接雨水，它在遇到右边界时弹出低洼底部并结算横向宽度。双指针更节省空间，单调栈更接近“找左右边界”的模型，面试中可以比较两者。

表达式求值考察栈对局部归约的建模。最简单的四则表达式如果没有括号，但有乘除优先级，可以扫描字符串，用栈保存已经归约到加减层面的项。遇到数字时，根据前一个运算符决定：加号就压入正数，减号压入负数，乘除就和栈顶立即合并。这样乘除不会等到最后才处理，加减则通过最后求和完成。

Listing 12.10 基本计算器 II：栈处理乘除优先级

```
int calculate(const std::string& s)
{
    std::vector<int> stack;
    char operation = '+';
    int number = 0;

    for (int i = 0; i <= static_cast<int>(s.size()); ++i) {
        const char ch = i == static_cast<int>(s.size()) ? '+' : s[i];
        if (std::isdigit(static_cast<unsigned char>(ch))) {
            number = number * 10 + (ch - '0');
            continue;
        }
        if (ch == ')') {
```

```

        continue;
    }

    if (operation == '+') {
        stack.push_back(number);
    } else if (operation == '-') {
        stack.push_back(-number);
    } else if (operation == '*') {
        stack.back() *= number;
    } else if (operation == '/') {
        stack.back() /= number;
    }

    operation = ch;
    number = 0;
}

return std::accumulate(stack.begin(), stack.end(), 0);
}

```

这里在循环末尾人为补一个加号，是为了触发最后一个数字的归约。表达式题的易错点非常多：多位数要持续累积；空格要跳过；除法在 C++ 中对整数向 0 截断，和题目语义是否一致要确认；如果有括号，就需要另一个层次的栈或显式状态机。不要一开始就写复杂解析器，先按题目支持的语法范围选择最小可行模型。面试表达时可以说：栈里保存的是已经处理完乘除优先级、等待最终加总的项。

线性结构题的实验训练不应只跑样例。链表题要构造空链表、单节点、删除头、删除尾、K 不整除长度；栈题要构造重复最小值、空弹出是否题目保证、嵌套和交叉括号；单调结构要构造严格递增、严格递减、全相等、窗口为 1、窗口等于数组长度。面试中如果能主动说出这些测试，说明你不仅知道算法，也知道边界来自哪里。

相交链表是链表“节点身份”意识的经典题。两个链表如果相交，意思不是某个节点的值相等，而是从某个节点开始共享同一批节点对象。暴力方法可以把第一条链表所有节点地址放入哈希集合，再扫描第二条链表，第一个出现在集合中的节点就是交点，复杂度 $O(m+n)$ ，空间 $O(m)$ 。双指针优化的思路更巧：指针 A 走完第一条链后转到第二条链，指针 B 走完第二条链后转到第一条链。若两链相交，它们会在交点相遇；若不相交，最终同时到达空指针。

Listing 12.11 相交链表：双指针走完两条路径

```

ListNode* get_intersection_node(ListNode* first, ListNode* second)
{
    ListNode* a = first;
    ListNode* b = second;

    while (a != b) {
        a = a == nullptr ? second : a->next;
        b = b == nullptr ? first : b->next;
    }

    return a;
}

```

为什么这段代码成立？设第一条链表独有长度为 x ，第二条独有长度为 y ，共享尾部长度为 z 。指针 A 走过的路径是 $x + z + y$ 后到达交点，指针 B 走过的路径是 $y + z + x$ 后到达交点，总长度相同。若没有共享尾部，两个指针都会在走完 $m+n$ 后变成空指针。这里不能比较节点值，因为不同节点可以有相同值；也不能修改链表结构，除非题目允许。

回文链表可以用数组保存所有值后双指针判断，时间 $O(n)$ 、空间 $O(n)$ 。如果要求 $O(1)$ 额外空间，就要找到中点、反转后半段、逐个比较，最后最好把链表恢复。这个题综合考察快慢指针和链表反转。快指针每次走两步，慢指针每次走一步；当快指针到达尾部时，慢指针在中点附近。奇数长度时，中间节点不影响回文判断，可以让后半段从慢指针之后开始。

Listing 12.12 回文链表：反转后半段后比较

```

ListNode* reverse_list(ListNode* head)
{
    ListNode* previous = nullptr;

```

```

ListNode* current = head;
while (current != nullptr) {
    ListNode* next = current->next;
    current->next = previous;
    previous = current;
    current = next;
}
return previous;
}

bool is_palindrome_list(ListNode* head)
{
    if (head == nullptr || head->next == nullptr) {
        return true;
    }

    ListNode* slow = head;
    ListNode* fast = head;
    while (fast->next != nullptr && fast->next->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }

    ListNode* second_half = reverse_list(slow->next);
    ListNode* restore_head = second_half;
    ListNode* first_half = head;
    bool ok = true;

    while (second_half != nullptr) {
        if (first_half->value != second_half->value) {
            ok = false;
            break;
        }
        first_half = first_half->next;
        second_half = second_half->next;
    }

    slow->next = reverse_list(restore_head);
    return ok;
}

```

这段代码刻意恢复链表，因为真实工程里“检查是否回文”不应该悄悄改变输入结构。力扣题可能不检查恢复，但教材要训练副作用意识。中点循环使用 `fast->next != nullptr && fast->next->next != nullptr`，能让 `slow` 停在前半段尾部。比较时只遍历后半段长度即可。易错点是反转后半段之前没有保存头节点，导致无法恢复。

链表排序可以用归并排序。数组排序可以随机访问中点，链表不行；链表找中点要用快慢指针，并从中点断开。暴力做法是把节点值放进数组排序再写回链表，这改变的是值，不是节点顺序；如果节点携带更多字段或外部引用，这种做法就不等价。链表归并排序的优势是稳定、时间 $O(n \log n)$ ，额外空间如果用递归会有栈成本；为了符合工程中避免深递归的习惯，可以使用自底向上的迭代归并。

Listing 12.13 链表排序：自底向上迭代归并

```

ListNode* split_list(ListNode* head, int size)
{
    for (int i = 1; head != nullptr && i < size; ++i) {
        head = head->next;
    }
    if (head == nullptr) {
        return nullptr;
    }
    ListNode* second = head->next;
    head->next = nullptr;
    return second;
}

ListNode* merge_sorted_lists(ListNode* first, ListNode* second, ListNode** tail)
{
    ListNode dummy;
    ListNode* current = &dummy;

```

```

while (first != nullptr && second != nullptr) {
    if (first->value <= second->value) {
        current->next = first;
        first = first->next;
    } else {
        current->next = second;
        second = second->next;
    }
    current = current->next;
}
current->next = first != nullptr ? first : second;
while (current->next != nullptr) {
    current = current->next;
}
*tail = current;
return dummy.next;
}

ListNode* sort_list(ListNode* head)
{
    int length = 0;
    for (ListNode* node = head; node != nullptr; node = node->next) {
        ++length;
    }

    ListNode dummy;
    dummy.next = head;
    for (int size = 1; size < length; size *= 2) {
        ListNode* previous = &dummy;
        ListNode* current = dummy.next;
        while (current != nullptr) {
            ListNode* first = current;
            ListNode* second = split_list(first, size);
            current = split_list(second, size);
            ListNode* merged_tail = nullptr;
            previous->next = merge_sorted_lists(first, second, &merged_tail);
            previous = merged_tail;
        }
    }
    return dummy.next;
}

```

自底向上归并的思想是先合并长度为 1 的有序段，再合并长度为 2、4、8 的段，直到段长覆盖整个链表。每次 `split_list` 都把一段切断，避免合并时越界串到后续段。代码较长，但它把递归栈换成了显式循环。面试中如果时间有限，可以先讲递归归并，再说明生产环境可改成迭代版本。这里的质量点不是炫技，而是知道链表排序的节点重连语义和递归深度风险。

删除排序链表中的重复元素也有两个版本。简单版保留一个重复值，只要当前节点和下一个节点值相等，就跳过下一个节点。进阶版删除所有重复值，只保留原链表中没有重复出现的值；这时必须使用虚拟头节点和前驱指针，因为重复段可能从头节点开始。先识别一整段相等值，再决定是保留还是跳过整段。

Listing 12.14 删除所有重复节点：识别重复段后整体跳过

```

ListNode* delete_duplicates_all(ListNode* head)
{
    ListNode dummy;
    dummy.next = head;
    ListNode* previous = &dummy;
    ListNode* current = head;

    while (current != nullptr) {
        bool duplicated = false;
        while (current->next != nullptr &&
            current->value == current->next->value) {
            duplicated = true;
            current = current->next;
        }
        if (duplicated) {
            previous->next = current->next;
        } else {
            previous->next = current;
        }
        previous = current;
        current = current->next;
    }
    return dummy.next;
}

```

```

        previous = previous->next;
    }
    current = current->next;
}

return dummy.next;
}

```

这题的重点是 `previous` 的含义：它始终指向结果链表的尾部，也就是最后一个确认保留的节点。遇到重复段时，`previous` 不动，只修改它的后继；遇到非重复节点时，`previous` 才前进。边界包括全是重复值、头部重复、尾部重复、没有重复。链表题里，前驱指针比当前指针更重要，因为删除动作发生在前驱的 `next` 上。

路径简化是栈在字符串解析中的应用。Unix 路径中，多个斜杠等价于一个斜杠，`.` 表示当前目录，`..` 表示回到上一级目录。暴力按字符处理容易被连续斜杠和边界搞乱。更清晰的做法是把路径按斜杠切分成段，栈保存有效目录名：普通目录入栈，`..` 弹出一个目录，空段和 `.` 忽略。最后用栈中目录重建规范路径。

Listing 12.15 简化路径：栈保存有效目录段

```

std::string simplify_path(const std::string& path)
{
    std::vector<std::string> stack;
    int index = 0;
    while (index < static_cast<int>(path.size())) {
        while (index < static_cast<int>(path.size()) && path[index] == '/') {
            ++index;
        }
        int start = index;
        while (index < static_cast<int>(path.size()) && path[index] != '/') {
            ++index;
        }
        const std::string part = path.substr(start, index - start);
        if (part.empty() || part == ".") {
            continue;
        }
        if (part == "..") {
            if (!stack.empty()) {
                stack.pop_back();
            }
        } else {
            stack.push_back(part);
        }
    }

    if (stack.empty()) {
        return "/";
    }
    std::string result;
    for (const std::string& part : stack) {
        result += "/";
        result += part;
    }
    return result;
}

```

这里的“栈”不是必须用 `std::stack`，用 `vector` 更方便遍历重建路径。容器选择要服务于操作：我们需要尾部压入弹出，也需要最后从头到尾遍历，所以 `vector` 很合适。若路径很长，频繁字符串拼接可以用 `ostringstream` 或先估算长度，但刷题中清晰优先。易错点是根目录下遇到 `..` 不能继续向上，只能保持根。

括号相关题还会扩展为“最长有效括号”。暴力枚举所有子串再判断是否合法，复杂度高。栈做法保存下标，而不是保存字符。栈底保存最近一个无法匹配的位置，作为当前有效段的左边界。遇到左括号入栈；遇到右括号先弹出一个左括号，若弹出后栈空，说明当前右括号无法匹配，把它的下标作为新边界；否则当前有效长度是当前下标减栈顶下标。

Listing 12.16 最长有效括号：栈保存边界下标

```

int longest_valid_parentheses(const std::string& s)

```

```

{
    std::vector<int> stack;
    stack.push_back(-1);
    int answer = 0;

    for (int i = 0; i < static_cast<int>(s.size()); ++i) {
        if (s[i] == '(') {
            stack.push_back(i);
            continue;
        }

        stack.pop_back();
        if (stack.empty()) {
            stack.push_back(i);
        } else {
            answer = std::max(answer, i - stack.back());
        }
    }

    return answer;
}

```

这个题中 `-1` 是哨兵边界，表示在字符串开始之前有一个不可跨越位置。栈顶在计算长度时不是当前有效段的第一个字符，而是当前有效段左侧的边界。这个语义非常重要。若只保存括号字符，就无法计算长度；若只保存左括号数量，就无法处理中间断裂。单调栈、括号栈、路径栈都在提醒同一件事：栈里保存的不是“看起来像栈的东西”，而是后续计算真正需要的历史状态。

队列题的进阶点是层次和边界。普通 BFS 中队列保存待扩展节点；若需要输出每层，就在每轮开始记录队列大小；若需要最短步数，就把层数和状态一起维护，或每处理一层步数加一。双端队列出现在两类场景：一类是单调队列维护窗口候选，另一类是 0-1 BFS 中根据边权把状态放队首或队尾。它们都不是“队列模板”，而是对候选优先级的精确表达。

线性结构实验：第一，用同一组链表节点分别测试删除倒数节点、回文判断、排序，检查函数是否意外破坏输入。第二，把最长有效括号的栈内容逐步打印出来，观察栈顶为何是边界下标。第三，分别用数组扫描、单调队列解决滑动窗口最大值，统计窗口大小增大时的运行差异。第四，给路径简化加入连续斜杠、根目录回退、隐藏文件名等测试，确认规则没有被字符串细节干扰。

线性结构题的统一问题

链表问“哪条边会断，断之前有没有保存”；栈问“哪个历史状态最近且必须先解决”；队列问“当前层或当前窗口的边界在哪里”；单调结构问“哪个候选被新元素支配，为什么永远不可能再成为答案”。只要这四个问题能回答，线性结构题就不再是零散模板。

第 13 章

排序、选择和区间问题

排序不是为了把数组排好看，而是为了制造结构。无序输入里，两个元素之间没有稳定关系；排序之后，大小关系变成单调关系，重复元素挨在一起，区间端点可以按起点或终点扫描，双指针和二分才有施展空间。面试中很多题的关键不是“调用 `std::sort`”，而是说明排序之后哪个约束变简单了，排序代价是否值得，以及排序是否会破坏题目要求的原始下标或稳定性。

C++ 的 `std::sort` 通常是内省排序，综合快速排序、堆排序和插入排序思想，平均和最坏复杂度都能控制在 $O(n \log n)$ 级别。刷题时可以放心使用它，但要理解比较器必须满足严格弱序。比较器不能写成 `a <= b`，也不能依赖会变化的外部状态，否则排序行为未定义。对结构体排序时，比较器应该只表达排序键，例如先按起点升序，再按终点升序。

Listing 13.1 区间排序比较器：严格弱序

```
struct Interval {
    int start = 0;
    int end = 0;
};

void sort_intervals(std::vector<Interval>& intervals)
{
    std::sort(intervals.begin(), intervals.end(),
        [](const Interval& lhs, const Interval& rhs) {
            if (lhs.start != rhs.start) {
                return lhs.start < rhs.start;
            }
            return lhs.end < rhs.end;
        });
}
```

这个比较器在 `lhs.start == rhs.start && lhs.end == rhs.end` 时返回 `false`，满足严格弱序要求。很多线上题数据小，错误比较器也可能侥幸通过，但这是 C++ 基本功问题。排序后，如果题目仍需要原始下标，就不能只排序值；要把值和下标一起保存成 `pair` 或结构体。

颜色分类是荷兰国旗问题。题目只包含 0、1、2 三种值，要求原地排序。直接调用排序当然能过，复杂度 $O(n \log n)$ ；计数排序也能 $O(n)$ ，但需要两次扫描。更经典的一次扫描做法是维护三个区域：左侧全是 0，中间是已经确认的 1，右侧全是 2，未知区域在中间游动。指针 `low` 指向下一个 0 应放的位置，`mid` 扫描未知区域，`high` 指向下一个 2 应放的位置。

Listing 13.2 颜色分类：三指针维护四段区域

```
void sort_colors(std::vector<int>& nums)
{
    constexpr int RED = 0;
    constexpr int WHITE = 1;
    constexpr int BLUE = 2;

    int low = 0;
    int mid = 0;
    int high = static_cast<int>(nums.size()) - 1;

    while (mid <= high) {
        if (nums[mid] == RED) {
            std::swap(nums[low], nums[mid]);
            ++low;
            ++mid;
        } else if (nums[mid] == WHITE) {
            ++mid;
        } else if (nums[mid] == BLUE) {
            std::swap(nums[mid], nums[high]);
            --high;
        }
    }
}
```

```

    }
}

```

这题最容易错在交换 2 之后是否移动 `mid`。答案是不能移动，因为从 `high` 换过来的元素还没有检查，可能是 0、1 或 2。交换 0 后可以移动 `mid`，因为 `low` 左边已经处理完，换到 `mid` 的元素来自已确认的 1 区或当前位置本身。这个差别正是不变量的体现：`[0, low)` 是 0，`[low, mid)` 是 1，`(high, n)` 是 2，`[mid, high]` 未知。

合并区间展示排序如何把二维关系变成线性扫描。暴力方法是反复找任意两个相交区间合并，直到不能合并为止；每次合并后都可能重新扫描全部区间，复杂度很高。按起点排序后，相交关系只需要和当前结果的最后一个区间比较。因为后续区间起点只会越来越大，如果它不能和最后区间相交，就不可能和更早已经结束的区间相交。

Listing 13.3 合并区间：排序后只看结果尾部

```

std::vector<std::vector<int>> merge_intervals(
    std::vector<std::vector<int>> intervals)
{
    if (intervals.empty()) {
        return {};
    }

    std::sort(intervals.begin(), intervals.end(),
        [](const std::vector<int>& lhs, const std::vector<int>& rhs) {
            return lhs[0] < rhs[0];
        });

    std::vector<std::vector<int>> merged;
    merged.push_back(intervals.front());

    for (int i = 1; i < static_cast<int>(intervals.size()); ++i) {
        std::vector<int>& last = merged.back();
        const std::vector<int>& current = intervals[i];
        if (current[0] <= last[1]) {
            last[1] = std::max(last[1], current[1]);
        } else {
            merged.push_back(current);
        }
    }

    return merged;
}

```

这里参数按值传入，是因为函数内部排序会改变区间顺序。如果调用者不需要保留原顺序，传引用也可以。复杂度是排序 $O(n \log n)$ 加扫描 $O(n)$ 。易错点是相交条件：如果区间是闭区间，`current[0] <= last[1]` 表示相交或接触；如果题目定义半开区间，就要重新审视等号是否成立。区问题一定要先明确端点语义。

插入区间与合并区间类似，但输入已经有序且不重叠。暴力方法可以把新区间放进去重新排序再合并，复杂度仍可接受，但没有利用已有条件。线性做法分三段：先加入所有结束在新区间之前的区间；再把所有和新区间相交的区间合并进新区间；最后加入剩余区间。这种写法体现了区间扫描的分段思维。

Listing 13.4 插入区间：前段、中段、后段

```

std::vector<std::vector<int>> insert_interval(
    const std::vector<std::vector<int>>& intervals,
    std::vector<int> new_interval)
{
    std::vector<std::vector<int>> answer;
    int index = 0;
    const int n = static_cast<int>(intervals.size());

    while (index < n && intervals[index][1] < new_interval[0]) {
        answer.push_back(intervals[index]);
        ++index;
    }

    while (index < n && intervals[index][0] <= new_interval[1]) {

```

```

        new_interval[0] = std::min(new_interval[0], intervals[index][0]);
        new_interval[1] = std::max(new_interval[1], intervals[index][1]);
        ++index;
    }
    answer.push_back(new_interval);

    while (index < n) {
        answer.push_back(intervals[index]);
        ++index;
    }

    return answer;
}

```

这题的优化条件是“原区间已经按起点排序且互不重叠”。如果这个条件不存在，就必须先排序或换模型。第一段的判断是 `intervals[index][1] < new_interval[0]`，说明当前区间完全在新区间左边；第二段的判断是 `intervals[index][0] <= new_interval[1]`，说明存在重叠。两个等号同样取决于闭区间语义。面试中把三段说清楚，代码通常很自然。

无重叠区间是区间贪心题。题目要求删除最少区间，使剩余区间互不重叠。等价地，可以保留最多数量的不重叠区间。暴力方法是枚举所有区间子集，检查是否互不重叠，复杂度指数级。贪心优化依据是：每次选择结束最早的区间，会给后面留下最大的空间。按结束时间排序，能选就选，不能选就跳过。

Listing 13.5 无重叠区间：按结束时间贪心保留最多区间

```

int erase_overlap_intervals(std::vector<std::vector<int>> intervals)
{
    if (intervals.empty()) {
        return 0;
    }

    std::sort(intervals.begin(), intervals.end(),
        [](const std::vector<int>& lhs, const std::vector<int>& rhs) {
            return lhs[1] < rhs[1];
        });

    int kept = 0;
    int current_end = std::numeric_limits<int>::min();
    for (const std::vector<int>& interval : intervals) {
        if (interval[0] >= current_end) {
            ++kept;
            current_end = interval[1];
        }
    }

    return static_cast<int>(intervals.size()) - kept;
}

```

这个贪心的交换论证是：假设某个最优解第一步选择了一个结束更晚的区间，而存在另一个兼容区间结束更早。把最优解里的第一个区间替换成结束更早的区间，不会减少后续可选空间，因此不会让答案变差。反复替换，就得到按结束时间贪心的解。区间贪心题不要只说“排序后贪心”，必须说明排序键为什么是结束时间而不是开始时间。

数组中的第 K 大元素可以排序解决，时间 $O(n \log n)$ 。如果只需要第 K 大，不需要完整有序，快速选择更合适。它借鉴快速排序的分区思想：选择一个枢轴，把大于枢轴的放左边，小于枢轴的放右边；如果枢轴最终位置正好是 $k - 1$ ，答案找到；如果位置偏左或偏右，只在一侧继续。平均复杂度 $O(n)$ 。

Listing 13.6 第 K 大元素：迭代快速选择

```

int find_kth_largest_quickselect(std::vector<int> nums, int k)
{
    const int target = k - 1;
    int left = 0;
    int right = static_cast<int>(nums.size()) - 1;

    while (left <= right) {
        const int pivot = nums[right];

```

```

int store = left;
for (int i = left; i < right; ++i) {
    if (nums[i] > pivot) {
        std::swap(nums[store], nums[i]);
        ++store;
    }
}
std::swap(nums[store], nums[right]);

if (store == target) {
    return nums[store];
}
if (store < target) {
    left = store + 1;
} else {
    right = store - 1;
}
}

return nums[target];
}

```

这里按值传参是刻意的，因为快速选择会重排数组。如果题目允许修改输入，可以传引用减少一次复制。代码使用最后一个元素作枢轴，最坏情况下会退化到 $O(n^2)$ ；工业实现通常会随机化枢轴或使用更稳健的选择算法。面试中要主动说出这个风险。快速选择的正确性来自分区后的位置含义：左侧元素都比枢轴大，因此枢轴排名已经确定，不需要管左右两侧内部是否有序。

排序和选择题经常和去重绑定。三数之和先排序，是为了让固定一个数后，剩余两数可以用双指针线性查找；合并区间先排序，是为了让所有可能相交的区间在扫描时相邻；第 K 大使用分区，是为了只确定目标排名而不完整排序。三者共同点是：排序或分区不是目的，而是让后续操作只看局部信息。

归并排序在刷题里经常不是为了排序本身，而是为了在“合并两个有序段”的过程中统计跨区间关系。逆序对就是典型例子。暴力方法枚举所有 $i < j$ ，检查 $nums[i] > nums[j]$ ，复杂度 $O(n^2)$ 。归并排序把数组分成左右两半，递归意义上左右内部逆序对已经统计完；合并时，只需要统计左半元素和右半元素形成的跨区间逆序对。由于左右两边已经有序，若 $left[i] > right[j]$ ，那么从 i 到左半末尾的所有元素都大于 $right[j]$ 。

Listing 13.7 逆序对：归并过程中统计跨区间关系

```

long long count_reverse_pairs(std::vector<int> nums)
{
    std::vector<int> buffer(nums.size());
    long long answer = 0;

    for (int width = 1; width < static_cast<int>(nums.size()); width *= 2) {
        for (int left = 0; left < static_cast<int>(nums.size()); left += 2 * width) {
            const int mid = std::min(left + width, static_cast<int>(nums.size()));
            const int right = std::min(left + 2 * width, static_cast<int>(nums.size()));

            int i = left;
            int j = mid;
            int out = left;
            while (i < mid && j < right) {
                if (nums[i] <= nums[j]) {
                    buffer[out++] = nums[i++];
                } else {
                    answer += mid - i;
                    buffer[out++] = nums[j++];
                }
            }
            while (i < mid) {
                buffer[out++] = nums[i++];
            }
            while (j < right) {
                buffer[out++] = nums[j++];
            }
            for (int k = left; k < right; ++k) {
                nums[k] = buffer[k];
            }
        }
    }
}

```

```

    }
}

return answer;
}

```

这里使用自底向上的迭代归并，避免递归。`width` 表示当前已经有序的段长度，每轮把相邻两段合并成更长有序段。统计逆序对的关键行是 `answer += mid - i`：当右侧当前元素更小，左侧从 `i` 到 `mid - 1` 的所有元素都会和它形成逆序对。这个推理依赖左半段已经有序。归并类统计题的复盘要问：跨区间关系能不能在两个有序段合并时一次性数出来？如果能，通常可以从平方复杂度降到 $O(n \log n)$ 。

会议室问题把区间排序和堆结合起来。会议室 I 问一个人能否参加所有会议，只要按开始时间排序，检查相邻会议是否重叠即可。会议室 II 问至少需要多少会议室，就不能只看相邻会议，因为多个会议可能同时进行。暴力方法对每个会议扫描当前所有房间，找能复用的房间；优化方法用小顶堆保存每个房间当前会议的结束时间。新会议开始时，如果最早结束的房间已经空出来，就复用它；否则需要新房间。

Listing 13.8 会议室 II：小顶堆保存房间结束时间

```

int min_meeting_rooms(std::vector<std::vector<int>> intervals)
{
    if (intervals.empty()) {
        return 0;
    }

    std::sort(intervals.begin(), intervals.end(),
              [](const std::vector<int>& lhs, const std::vector<int>& rhs) {
                  return lhs[0] < rhs[0];
              });

    std::priority_queue<int, std::vector<int>, std::greater<int>> endings;
    for (const std::vector<int>& meeting : intervals) {
        if (!endings.empty() && endings.top() <= meeting[0]) {
            endings.pop();
        }
        endings.push(meeting[1]);
    }

    return static_cast<int>(endings.size());
}

```

堆顶表示当前所有占用房间中最早结束的会议。如果它都晚于新会议开始，那么其他房间结束更晚，也不能复用；如果它不晚于新会议开始，就弹出并把该房间的新结束时间压入。复杂度是排序 $O(n \log n)$ 加每个会议一次堆操作。注意这里一次只弹出一个结束时间就够了，因为每个新会议只需要一个房间；如果题目问某个时间点同时进行会议数量，可能会弹出所有已结束会议。

差分数组是区间更新题的另一种排序替代思路。如果题目反复对区间加值，再询问每个位置最终值，暴力对每次更新逐个位置加，复杂度 $O(qn)$ 。差分数组保存相邻元素变化量：对区间 `[left, right]` 加 `value`，只需要 `diff[left] += value`，`diff[right + 1] -= value`。最后做一次前缀和还原所有位置。

Listing 13.9 航班预订统计：差分数组处理区间加法

```

std::vector<int> corp_flight_bookings(const std::vector<std::vector<int>>& bookings,
                                      int n)
{
    std::vector<int> diff(n + 1, 0);
    for (const std::vector<int>& booking : bookings) {
        const int left = booking[0] - 1;
        const int right = booking[1] - 1;
        const int seats = booking[2];
        diff[left] += seats;
        if (right + 1 < n) {
            diff[right + 1] -= seats;
        }
    }
}

```

```

std::vector<int> answer(n, 0);
int current = 0;
for (int i = 0; i < n; ++i) {
    current += diff[i];
    answer[i] = current;
}
return answer;
}

```

差分正确性来自“影响从 left 开始，在 right 之后结束”。它把区间内每个点都加的操作，变成两个边界事件。扫描线也有类似思想：把区间起点看成 +1 事件，终点看成 -1 事件，排序后扫描事件就能得到当前重叠数量。差分适合坐标连续且范围可开数组的场景；扫描线适合坐标很大或稀疏，需要排序事件的场景。

计数排序和桶排序利用值域信息。若数组长度很大但值域很小，比较排序的 $O(n \log n)$ 下界不再是必须承受的，因为我们不靠两两比较区分所有排列，而是统计每个值出现次数。颜色分类可以看成值域只有 3 的计数排序变体；前 K 高频元素也可以在统计频次后按频次分桶。桶排序的风险是空间由值域或频次范围决定，不能在值域巨大时盲目开数组。

Listing 13.10 前 K 高频元素：桶按频次收集

```

std::vector<int> top_k_frequent_bucket(const std::vector<int>& nums, int k)
{
    std::unordered_map<int, int> frequency;
    frequency.reserve(nums.size());
    for (const int x : nums) {
        ++frequency[x];
    }

    std::vector<std::vector<int>> buckets(nums.size() + 1);
    for (const auto& [value, count] : frequency) {
        buckets[count].push_back(value);
    }

    std::vector<int> answer;
    for (int count = static_cast<int>(buckets.size()) - 1;
         count >= 0 && static_cast<int>(answer.size()) < k;
         --count) {
        for (const int value : buckets[count]) {
            answer.push_back(value);
            if (static_cast<int>(answer.size()) == k) {
                break;
            }
        }
    }
    return answer;
}

```

这个版本的时间复杂度是 $O(n)$ ，空间复杂度也是 $O(n)$ ，因为最大频次不超过数组长度。它比堆更快吗？理论上是线性的，但常数和内存占用更高，且输出顺序不固定。实际面试可以先讲堆，因为堆适用范围更广；再补桶排序作为值域或频次范围明确时的优化。高质量答案不是只追求最低复杂度，而是解释方案适用条件。

相对排序数组展示“值域有限时不一定要比较排序”。题目给两个数组，第二个数组给出部分元素的相对顺序，要求第一个数组中这些元素按第二个数组顺序排列，其余元素升序排列。暴力做法可以写自定义比较器：若两个元素都在第二个数组中，就比较它们的排名；若只有一个在第二个数组中，它排前面；否则按值升序。这个做法清晰，但每次比较都要查询排名，复杂度仍是排序的 $O(n \log n)$ 。如果值域较小，可以计数后按规则输出，复杂度接近线性。

Listing 13.11 相对排序：计数后按给定顺序输出

```

std::vector<int> relative_sort_array(const std::vector<int>& arr1,
                                    const std::vector<int>& arr2)
{
    constexpr int MAX_VALUE = 1000;
    std::array<int, MAX_VALUE + 1> count{};
    for (const int x : arr1) {
        ++count[x];
    }
}

```

```

    }

    std::vector<int> answer;
    answer.reserve(arr1.size());
    for (const int x : arr2) {
        while (count[x] > 0) {
            answer.push_back(x);
            --count[x];
        }
    }
    for (int value = 0; value <= MAX_VALUE; ++value) {
        while (count[value] > 0) {
            answer.push_back(value);
            --count[value];
        }
    }
    return answer;
}

```

这段代码的前提是题目给定值域不超过 1000，所以 `MAX_VALUE` 是题目约束，不是随便拍脑袋。若值域很大，就不能开这么大的数组，应改用哈希表统计，再对剩余元素排序。这个题训练的是“比较排序下界的适用范围”：当元素值域和输出顺序被额外约束时，我们可以不通过比较确定所有相对顺序。

按奇偶排序、按自定义规则排序、把负数放前面等题，看起来简单，其实都在考“稳定性”。如果题目要求保持同类元素的原始相对顺序，就需要稳定划分，可以新开数组，或者用 `stable_partition`。如果不要稳定，就可以用左右指针原地交换。原地不稳定划分的复杂度低、空间少，但会改变相对顺序；稳定划分保留顺序，但通常需要额外空间或更复杂的移动。面试中主动问清稳定性，是比直接写代码更成熟的表现。

Listing 13.12 稳定划分：保持非零元素相对顺序

```

std::vector<int> stable_partition_by_sign(const std::vector<int>& nums)
{
    std::vector<int> answer;
    answer.reserve(nums.size());
    for (const int x : nums) {
        if (x < 0) {
            answer.push_back(x);
        }
    }
    for (const int x : nums) {
        if (x >= 0) {
            answer.push_back(x);
        }
    }
    return answer;
}

```

这不是最省空间的写法，但语义非常清晰。若题目必须原地且稳定，问题会变难，因为把一个负数移动到前面可能要整体搬移中间元素。很多面试题默认不要求稳定，允许左右指针交换；但如果业务语义中原始顺序有意义，例如日志时间顺序、用户输入顺序，就不能随便破坏。排序和划分题一定要把“允许改变什么”说清楚。

数组中的逆序对除了归并计数，还可以用树状数组。归并排序从“跨左右两半的逆序对”出发；树状数组从“扫描到当前数时，之前有多少数比它大”出发。由于数值可能很大或为负，需要先离散化，把值映射到排名。扫描数组时，查询已经出现过的元素总数减去小于等于当前排名的数量，就是以当前元素为右端的逆序对数量；然后把当前排名出现次数加一。

Listing 13.13 逆序对：离散化加树状数组

```

class FenwickTree {
public:
    explicit FenwickTree(int size) : tree_(size + 1, 0) {}

    void add(int index, int delta)
    {
        for (int i = index; i < static_cast<int>(this->tree_.size());

```

```

        i += i & -i) {
            this->tree_[i] += delta;
        }
    }

    int sum(int index) const
    {
        int result = 0;
        for (int i = index; i > 0; i -= i & -i) {
            result += this->tree_[i];
        }
        return result;
    }

private:
    std::vector<int> tree_;
};

long long reverse_pairs_by_fenwick(const std::vector<int>& nums)
{
    std::vector<int> values = nums;
    std::sort(values.begin(), values.end());
    values.erase(std::unique(values.begin(), values.end()), values.end());

    FenwickTree tree(static_cast<int>(values.size()));
    long long answer = 0;
    for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
        const int rank = static_cast<int>(
            std::lower_bound(values.begin(), values.end(), nums[i]) -
            values.begin()) + 1;
        const int seen = i;
        const int not_greater = tree.sum(rank);
        answer += seen - not_greater;
        tree.add(rank, 1);
    }
    return answer;
}

```

树状数组适合动态维护前缀计数。它的下标从 1 开始， $i \& -i$ 表示当前节点覆盖区间的长度。这个结构比归并计数更抽象，但能迁移到很多“动态排名、区间频次、前缀和修改查询”问题。面试中如果只考逆序对，归并排序更容易解释；如果题目有多次更新或在线查询，树状数组更有优势。这里再次体现：排序不是唯一制造顺序的方法，离散化加索引树也能制造排名结构。

扫描线是区间问题的重要模型。区间重叠数量、天际线、会议室、航班预订，都可以看成一系列事件按坐标发生。区间开始是一个加事件，区间结束是一个减事件。排序事件后从左到右扫描，当前累计值就是当前位置的活跃区间数量。对于闭区间和半开区间，开始和结束在同一坐标时的处理顺序不同；例如会议室通常认为一个会议在时间 t 结束，另一个会议在时间 t 开始可以复用同一房间，所以结束事件应该先于开始事件处理。

Listing 13.14 会议室数量：扫描线事件排序

```

int min_meeting_rooms_by_events(const std::vector<std::vector<int>>& intervals)
{
    std::vector<std::pair<int, int>> events;
    events.reserve(intervals.size() * 2);
    for (const std::vector<int>& interval : intervals) {
        events.emplace_back(interval[0], 1);
        events.emplace_back(interval[1], -1);
    }

    std::sort(events.begin(), events.end(),
        [](const auto& lhs, const auto& rhs) {
            if (lhs.first != rhs.first) {
                return lhs.first < rhs.first;
            }
            return lhs.second < rhs.second;
        });

    int active = 0;
    int answer = 0;
    for (const auto [time, delta] : events) {

```

```

        (void)time;
        active += delta;
        answer = std::max(answer, active);
    }
    return answer;
}

```

这里 -1 事件在同一时间排在 $+1$ 前面，所以会议结束先释放房间。若题目是闭区间覆盖点，结束和开始的同点处理可能要反过来。扫描线的难点不是代码，而是事件语义：一个事件表示影响从哪里开始，在哪里结束，边界是否包含。差分数组是坐标连续且范围可开数组时的扫描线；事件排序是坐标稀疏或很大时的扫描线。

用最少箭引爆气球是区间贪心。每个气球是一个水平区间，一支箭射在某个坐标可以引爆所有覆盖该坐标的气球。暴力想法是尝试每个可能射击位置，但坐标范围巨大。排序后贪心：按右端点升序排列，第一支箭射在第一个气球的右端点，这个选择最不容易影响后续，因为它尽可能靠右，同时仍能引爆当前气球。之后扫描气球，若当前气球起点大于当前箭位置，说明必须新射一箭。

Listing 13.15 最少箭引爆气球：按右端点贪心

```

int find_min_arrow_shots(std::vector<std::vector<int>> points)
{
    if (points.empty()) {
        return 0;
    }
    std::sort(points.begin(), points.end(),
        [](const std::vector<int>& lhs, const std::vector<int>& rhs) {
            return lhs[1] < rhs[1];
        });

    int arrows = 1;
    int arrow_position = points[0][1];
    for (int i = 1; i < static_cast<int>(points.size()); ++i) {
        if (points[i][0] <= arrow_position) {
            continue;
        }
        ++arrows;
        arrow_position = points[i][1];
    }
    return arrows;
}

```

正确性可以用交换论证说明：对于最早结束的气球，任何可行解都必须有一支箭射在它的区间内。把这支箭移动到该气球右端点，不会让它失去对已覆盖气球的覆盖能力，反而给后续区间留下更大机会。因此选择最早右端点是安全的。这个思路也适用于无重叠区间中“保留最多区间”：按结束时间选择最早结束的区间，给后面留下空间。

区间列表交集是双指针题。两个区间列表各自有序且互不重叠。暴力两两比较复杂度 $O(mn)$ 。双指针优化来自有序性：当前两个区间的交集左端是较大起点，右端是较小终点；若左端不大于右端，就有交集。然后谁的终点更早，谁就不可能再和对方后续区间相交，移动谁的指针。

Listing 13.16 区间列表交集：移动较早结束的一方

```

std::vector<std::vector<int>> interval_intersection(
    const std::vector<std::vector<int>>& first,
    const std::vector<std::vector<int>>& second)
{
    std::vector<std::vector<int>> answer;
    int i = 0;
    int j = 0;
    while (i < static_cast<int>(first.size()) &&
        j < static_cast<int>(second.size())) {
        const int left = std::max(first[i][0], second[j][0]);
        const int right = std::min(first[i][1], second[j][1]);
        if (left <= right) {
            answer.push_back({left, right});
        }

        if (first[i][1] < second[j][1]) {
            ++i;
        }
    }
}

```

```

    } else {
        ++j;
    }
}
return answer;
}

```

这题的推理比代码重要：终点较早的区间在当前比较后已经没有机会和另一个当前区间或它后面的区间形成新的交集，因为后续区间起点只会更大。因此可以安全丢弃它。这个“丢弃谁”的逻辑和合并区间、插入区间、会议室复用都属于同一类端点单调推理。

日志重排和版本号比较这类题看起来不是传统算法，但同样考排序规则。日志重排要求字母日志排在数字日志前，字母日志按内容排序，内容相同按标识符排序，数字日志保持原始相对顺序。若直接写比较器处理数字日志之间返回 `false`，还需要使用稳定排序才能保持它们原顺序；或者把字母日志单独取出排序，再追加数字日志。这里的关键是理解 `std::sort` 不稳定，不能指望相等元素原顺序不变。

常见误区

排序比较器有一个底线：不能让两个元素互相都“小于”等关系成立，也不能让相等元素比较返回真。比较器依赖外部可变状态、随机数、递增计数器，都会破坏排序算法假设。若题目要求保持相对顺序，请使用稳定排序或显式分组，不要假设普通排序“碰巧没改顺序”。

区间和排序题的测试要覆盖边界语义。合并区间要测相邻是否合并、完全包含、乱序输入；会议室要测一个会议结束另一个会议开始；最少箭要测端点相同的大坐标；快速选择要测重复值、K 为 1、K 为数组长度；计数排序要测值域边界。很多排序题代码看起来一行 `sort` 很短，但真正的错误都藏在比较器和边界定义里。

排序区间实验：第一，把会议室问题分别用堆和扫描线写一遍，用同一组边界用例比较答案。第二，把无重叠区间、最少箭、会议安排放在一起复盘，写出它们为什么都偏好按右端点排序。第三，给相对排序构造值域很大但元素很少的用例，比较计数数组和哈希统计的空间差异。第四，故意写一个 `a <= b` 的错误比较器，观察小数据可能不报错但语义已经未定义。

排序题复盘模板

先写清楚排序键，再说明排序后获得了什么性质：相同值相邻、端点单调、排名确定、候选可淘汰，还是可以双指针。然后说明是否需要保留原下标，比较器是否满足严格弱序，排序是否改变输入。最后比较完整排序、堆、快速选择、计数排序等方案的适用范围。能说清这些，排序题才不是“先 `sort` 一下试试”。

第 14 章

堆、Top K 和贪心证明

堆和贪心经常一起出现，但它们不是同一个概念。堆是一种数据结构，负责快速取出当前最大或最小候选；贪心是一种算法策略，声称每一步做局部最优选择不会破坏全局最优。堆题的关键是“当前最该处理的候选是谁”；贪心题的关键是“为什么现在做这个选择以后不会后悔”。面试中只写出 `priority_queue` 远远不够，还要讲清候选含义、过期规则、比较器方向和正确性理由。

C++ 的 `std::priority_queue` 默认是大顶堆。要写小顶堆，需要使用 `std::greater<T>`。如果元素是结构体，需要自定义比较器。比较器的方向经常让人困惑：`priority_queue` 会把“优先级最大”的元素放在 `top`，比较器返回 `true` 表示左侧优先级低于右侧。因此写自定义比较器时，最好用“谁应该排后面”来理解。

Listing 14.1 `priority_queue` 的比较器方向

```
struct Entry {
    int value = 0;
    int frequency = 0;
};

struct LowerFrequencyFirst {
    bool operator()(const Entry& lhs, const Entry& rhs) const
    {
        return lhs.frequency > rhs.frequency;
    }
};

using MinHeap = std::priority_queue<Entry,
                                   std::vector<Entry>,
                                   LowerFrequencyFirst>;
```

这段比较器让频次更小的元素优先出现在堆顶，因为当 `lhs.frequency > rhs.frequency` 时，`lhs` 被认为优先级更低。很多人会在这里写反，导致小顶堆变成大顶堆。简单类型能用 `std::greater<int>` 时就用标准库；复杂类型必须写清比较器语义，并用小样例验证堆顶是谁。

前 K 个高频元素是哈希表和堆的组合题。暴力做法是统计频次后把所有不同元素按频次排序，复杂度 $O(m \log m)$ ， m 是不同元素个数。如果只要前 K 个，不需要完整排序，可以维护一个大小为 K 的小顶堆。堆顶是当前前 K 高频候选里频次最低的元素；新元素频次如果不超过堆顶，就不可能进入前 K；如果超过，就替换堆顶。

Listing 14.2 前 K 个高频元素：频次表加大小为 K 的小顶堆

```
std::vector<int> top_k_frequent(const std::vector<int>& nums, int k)
{
    std::unordered_map<int, int> frequency;
    frequency.reserve(nums.size());
    for (const int x : nums) {
        ++frequency[x];
    }

    using Entry = std::pair<int, int>; // frequency, value
    std::priority_queue<Entry, std::vector<Entry>, std::greater<Entry>> heap;

    for (const auto& [value, count] : frequency) {
        heap.emplace(count, value);
        if (static_cast<int>(heap.size()) > k) {
            heap.pop();
        }
    }

    std::vector<int> answer;
    answer.reserve(k);
    while (!heap.empty()) {
```

```

        answer.push_back(heap.top().second);
        heap.pop();
    }
    return answer;
}

```

这里用 `pair<count, value>` 是为了让 `std::greater` 按频次小顶堆工作；如果频次相同，会按值比较，但题目通常不要求固定顺序。复杂度是统计 $O(n)$ ，堆维护 $O(m \log k)$ 。如果 K 接近 m ，完整排序可能更简单；如果值域有限，也可以用桶排序按频次分桶，达到线性复杂度。面试时要比较这三种方案，而不是只给一种。

数据流中位数展示双堆模型。一个大顶堆保存较小一半，一个小顶堆保存较大一半。两个堆的大小差不超过 1，并且大顶堆堆顶不大于小顶堆堆顶。这样总数为奇数时，中位数是元素更多那一侧的堆顶；总数为偶数时，中位数是两个堆顶平均值。暴力方法每次插入后排序或线性插入，查询中位数容易但插入代价高。

Listing 14.3 数据流中位数：双堆维护两半

```

class MedianFinder {
public:
    void add_num(int num)
    {
        if (this->lower_.empty() || num <= this->lower_.top()) {
            this->lower_.push(num);
        } else {
            this->upper_.push(num);
        }
        this->rebalance();
    }

    double find_median() const
    {
        if (this->lower_.size() == this->upper_.size()) {
            return (static_cast<double>(this->lower_.top()) +
                    static_cast<double>(this->upper_.top())) / 2.0;
        }
        return this->lower_.size() > this->upper_.size()
            ? static_cast<double>(this->lower_.top())
            : static_cast<double>(this->upper_.top());
    }

private:
    void rebalance()
    {
        if (this->lower_.size() > this->upper_.size() + 1) {
            this->upper_.push(this->lower_.top());
            this->lower_.pop();
        } else if (this->upper_.size() > this->lower_.size() + 1) {
            this->lower_.push(this->upper_.top());
            this->upper_.pop();
        }
    }

    std::priority_queue<int> lower_;
    std::priority_queue<int, std::vector<int>, std::greater<int>> upper_;
};

```

这段代码把重平衡逻辑拆成私有函数，避免 `add_num` 过长。成员访问使用 `this->`，符合 C++ 代码规范。复杂度是插入 $O(\log n)$ ，查询 $O(1)$ 。易错点有两个：偶数个元素时要转成 `double` 避免整数溢出和整数除法；重平衡只保证大小差，不要忘记插入时先按堆顶决定放入哪一侧，否则大小平衡但顺序不平衡。

合并 K 个升序链表是堆在链表上的应用。暴力方法可以每次扫描 K 个链表头，找最小节点接入结果，若总节点数为 N ，复杂度 $O(NK)$ 。堆优化是把每条链表当前头节点放入小顶堆，每次弹出最小节点，再把它后继放入堆。堆里最多 K 个节点，所以复杂度 $O(N \log K)$ 。

Listing 14.4 合并 K 个升序链表：堆保存每条链当前头节点

```

ListNode* merge_k_lists(std::vector<ListNode*> lists)

```

```

{
    struct CompareNode {
        bool operator()(const ListNode* lhs, const ListNode* rhs) const
        {
            return lhs->value > rhs->value;
        }
    };

    std::priority_queue<ListNode*, std::vector<ListNode*>, CompareNode> heap;
    for (ListNode* node : lists) {
        if (node != nullptr) {
            heap.push(node);
        }
    }

    ListNode dummy;
    ListNode* tail = &dummy;
    while (!heap.empty()) {
        ListNode* node = heap.top();
        heap.pop();
        if (node->next != nullptr) {
            heap.push(node->next);
        }
        tail->next = node;
        tail = tail->next;
    }
    tail->next = nullptr;
    return dummy.next;
}

```

这里比较器把值更大的节点放到后面，因此堆顶是值最小节点。弹出节点后先把后继放入堆，再接入结果，两个顺序都可以，但最后要确保 `tail->next = nullptr`，避免结果尾部还连着旧链表的残余路径产生误解。这个算法没有创建新节点，复用输入节点；如果不允许修改输入，需要新建节点。面试表达要强调堆里保存的是“每条链表尚未合并部分的最小候选”。

跳跃游戏是贪心的入门题。题目问能否从下标 0 跳到最后。暴力搜索会从每个位置尝试所有可跳长度，存在大量重复状态。DP 可以记录每个位置是否可达，复杂度 $O(n^2)$ 。贪心更简单：扫描过程中维护当前能到达的最远下标 `farthest`。如果扫描到的位置已经超过 `farthest`，说明这个位置不可达；否则用它继续更新最远范围。

Listing 14.5 跳跃游戏：维护当前可达最远位置

```

bool can_jump(const std::vector<int>& nums)
{
    int farthest = 0;
    for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
        if (i > farthest) {
            return false;
        }
        farthest = std::max(farthest, i + nums[i]);
        if (farthest >= static_cast<int>(nums.size()) - 1) {
            return true;
        }
    }
    return true;
}

```

这个贪心的正确性来自覆盖区间：只要当前位置 `i` 可达，那么从它出发能扩展的所有位置都被 `farthest` 覆盖；我们不关心具体从哪条路径到达，只关心可达区域的右边界。每次扫描到边界内的新位置，都可能把边界向右推。若某个位置在边界之外，任何之前位置都无法到达它，后面的位置更不可能被访问。

跳跃游戏 II 要求最少跳跃次数。暴力 BFS 可以把每个下标当节点，每条可跳边连接到后续位置，求无权最短路，但边数最坏 $O(n^2)$ 。贪心做法把一跳能到达的范围看成 BFS 的一层：`current_end` 是当前跳数能覆盖的最右位置，扫描这一层内所有点，计算下一跳能到达的最远位置 `farthest`；当扫描到 `current_end`，说明这一层结束，必须增加一次跳跃。

Listing 14.6 跳跃游戏 II：按层扩展可达区间

```

int jump(const std::vector<int>& nums)
{
    if (nums.size() <= 1) {
        return 0;
    }

    int jumps = 0;
    int current_end = 0;
    int farthest = 0;

    for (int i = 0; i < static_cast<int>(nums.size()) - 1; ++i) {
        farthest = std::max(farthest, i + nums[i]);
        if (i == current_end) {
            ++jumps;
            current_end = farthest;
        }
    }

    return jumps;
}

```

这段代码本质上是压缩版 BFS。每次增加 `jumps`，代表进入下一层；`current_end` 以内的位置都属于当前层。为什么循环只到倒数第二个位置？因为一旦已经能到达最后，就不需要从最后一个位置再跳。常见错误是在每个位置都加跳数，或者没有区分当前层边界和下一层最远边界。

贪心证明通常有两种表达：交换论证和领先性论证。区间调度用交换论证：把最优解中结束更晚的选择换成结束更早的选择，不会变差。跳跃游戏用领先性论证：贪心维护的可达最远边界始终不落落后于任何其他方案在相同跳数下能到达的边界。面试中给出哪种证明不重要，重要的是不要把“看起来对”当成证明。

任务调度器是堆、计数和构造思维的交叉题。题目给一组任务和冷却时间 `n`，相同任务之间必须至少间隔 `n` 个单位时间，问完成所有任务的最短时间。暴力模拟可以每个时间点扫描所有任务，选择当前可执行且剩余次数最多的任务；如果任务种类多，扫描会重复。堆优化是把可执行任务按剩余次数放进大顶堆，把冷却中的任务放进队列，时间推进时把冷却结束的任务放回堆。

Listing 14.7 任务调度器：堆处理可执行任务，队列处理冷却

```

int least_interval(const std::vector<char>& tasks, int n)
{
    std::array<int, 26> count{};
    for (const char task : tasks) {
        ++count[task - 'A'];
    }

    std::priority_queue<int> available;
    for (const int c : count) {
        if (c > 0) {
            available.push(c);
        }
    }

    std::queue<std::pair<int, int>> cooling; // ready_time, remaining
    int time = 0;
    while (!available.empty() || !cooling.empty()) {
        ++time;
        if (!available.empty()) {
            int remaining = available.top();
            available.pop();
            --remaining;
            if (remaining > 0) {
                cooling.emplace(time + n, remaining);
            }
        }

        if (!cooling.empty() && cooling.front().first == time) {
            available.push(cooling.front().second);
            cooling.pop();
        }
    }
}

```

```

    return time;
}

```

这个模拟版本容易解释，但还有数学构造解。设最高频任务出现 `max_count` 次，有 `max_kinds` 种任务都达到最高频。最短时间至少是 $(\text{max_count} - 1) * (n + 1) + \text{max_kinds}$ ，因为最高频任务形成 `max_count - 1` 个完整间隔块，最后一组放所有最高频任务。答案还不能小于任务总数，所以取二者最大。模拟适合理解过程，公式适合直接求最短长度。面试中给出一种后，最好能说明另一种的直觉。

加油站是贪心的区间累积题。题目给每个站的油量和到下一站的消耗，问是否存在起点能绕一圈。暴力方法从每个站出发模拟一圈，复杂度 $O(n^2)$ 。贪心依据是：如果从 `start` 出发到 `i` 之间某处油量变成负数，那么 `start` 到 `i` 之间任何站都不可能作为合法起点。因为从这些中间站出发，只会少掉 `start` 到该站之前积累的油量，不会更好。

Listing 14.8 加油站：失败区整体跳过

```

int can_complete_circuit(const std::vector<int>& gas,
                        const std::vector<int>& cost)
{
    int total = 0;
    int tank = 0;
    int start = 0;

    for (int i = 0; i < static_cast<int>(gas.size()); ++i) {
        const int delta = gas[i] - cost[i];
        total += delta;
        tank += delta;
        if (tank < 0) {
            start = i + 1;
            tank = 0;
        }
    }

    return total >= 0 ? start : -1;
}

```

`total < 0` 表示全局油量不足，任何起点都不可能成功。若全局油量足够，局部失败时把起点跳到 `i + 1` 是安全的。这个跳跃是贪心正确性的核心，不是经验优化。加油站题的复盘要把“失败区整体排除”说清楚，否则代码看起来像玄学。

划分字母区间展示“最后出现位置”如何制造贪心边界。题目要求把字符串划分成尽可能多的片段，使每个字母最多出现在一个片段中。暴力方法枚举所有切分并检查字符集合是否交叉，复杂度 $O(n^2)$ 。优化思路是先记录每个字符最后出现的位置。扫描时，当前片段右边界必须至少覆盖已经遇到的所有字符的最后位置；当扫描下标到达这个右边界时，当前片段可以安全切开。

Listing 14.9 划分字母区间：扫描当前片段最远边界

```

std::vector<int> partition_labels(const std::string& s)
{
    std::array<int, 26> last{};
    for (int i = 0; i < static_cast<int>(s.size()); ++i) {
        last[s[i] - 'a'] = i;
    }

    std::vector<int> answer;
    int start = 0;
    int end = 0;
    for (int i = 0; i < static_cast<int>(s.size()); ++i) {
        end = std::max(end, last[s[i] - 'a']);
        if (i == end) {
            answer.push_back(end - start + 1);
            start = i + 1;
        }
    }
    return answer;
}

```

这个贪心的边界语义非常清楚：当前片段中出现过的所有字符，它们的最后出现位置都不超过

end。当 $i == \text{end}$ ，说明这些字符不会在后面再次出现，因此切开不会违反题意；如果提前切开，至少有一个字符会跨片段。类似思想也出现在字符串覆盖、最小区间和扫描线问题中：先知道每个对象的最后影响范围，再扫描形成最小合法边界。

IPO 项目选择题把贪心和双堆结合起来。给定初始资本、最多做 k 个项目、每个项目需要资本和带来利润，问最大资本。暴力方法每轮扫描所有项目，找当前资本能做且利润最高的项目，复杂度 $O(kn)$ 。优化方法是先按资本要求排序，用指针把当前资本可负担的项目加入利润大顶堆，每轮从堆顶选择利润最高项目。

Listing 14.10 IPO：按资本解锁项目，用利润堆选择

```
int find_maximized_capital(int k,
                           int capital,
                           std::vector<int> profits,
                           std::vector<int> required)
{
    std::vector<std::pair<int, int>> projects;
    projects.reserve(profits.size());
    for (int i = 0; i < static_cast<int>(profits.size()); ++i) {
        projects.emplace_back(required[i], profits[i]);
    }
    std::sort(projects.begin(), projects.end());

    std::priority_queue<int> available_profits;
    int index = 0;
    for (int round = 0; round < k; ++round) {
        while (index < static_cast<int>(projects.size()) &&
               projects[index].first <= capital) {
            available_profits.push(projects[index].second);
            ++index;
        }
        if (available_profits.empty()) {
            break;
        }
        capital += available_profits.top();
        available_profits.pop();
    }

    return capital;
}
```

这个问题的局部最优选择是：在当前能做的项目中，选择利润最高的。为什么不会后悔？因为做项目只会增加资本，不会消耗资本；利润越高，下一轮可选项目集合只会更大，不会更小。这个性质非常关键。如果项目会消耗资源，或者项目之间有依赖，简单贪心就可能失败。堆这里只是让“当前可选项目中利润最高”查询更快。

贪心失败的例子同样重要。零钱兑换若面额是 1, 3, 4，目标 6，贪心先选 4，再选 1、1，共 3 枚；最优是 3、3，共 2 枚。这个反例说明“每次选最大面额”不一定全局最优。判断贪心是否成立，不能只看样例，要找交换论证、单调边界、领先性或 matroid 这类结构。没有证明时，优先退回 DP 或搜索。

重构字符串是堆处理“当前最紧急约束”的例子。题目要求重新排列字符串，使相邻字符不同。暴力回溯可以尝试所有排列，但重复字符多时搜索空间巨大。贪心想法是每次优先使用剩余次数最多的字符，因为它最难安排；但不能连续使用同一个字符，所以每轮从大顶堆取出两个频次最高的不同字符，依次放入答案，再把剩余频次放回堆。若某个字符出现次数超过 $(n+1)/2$ ，必然无法重构。

Listing 14.11 重构字符串：每次取两个最高频字符

```
std::string reorganize_string(const std::string& s)
{
    std::array<int, 26> count{};
    for (const char ch : s) {
        ++count[ch - 'a'];
    }

    using Entry = std::pair<int, char>;
    std::priority_queue<Entry> heap;
```

```

for (int i = 0; i < static_cast<int>(count.size()); ++i) {
    if (count[i] > 0) {
        heap.emplace(count[i], static_cast<char>('a' + i));
    }
}

std::string answer;
answer.reserve(s.size());
while (heap.size() >= 2) {
    auto [first_count, first_char] = heap.top();
    heap.pop();
    auto [second_count, second_char] = heap.top();
    heap.pop();

    answer.push_back(first_char);
    answer.push_back(second_char);
    if (--first_count > 0) {
        heap.emplace(first_count, first_char);
    }
    if (--second_count > 0) {
        heap.emplace(second_count, second_char);
    }
}

if (!heap.empty()) {
    auto [remaining_count, remaining_char] = heap.top();
    if (remaining_count > 1) {
        return "";
    }
    if (!answer.empty() && answer.back() == remaining_char) {
        return "";
    }
    answer.push_back(remaining_char);
}

return answer;
}

```

这个贪心的直觉是“高频字符最危险，越晚处理越可能无处可放”。每次取两个不同字符，可以保证刚放入的两个字符相邻不同，也避免同一个最高频字符连续出现。更简洁的写法是每次取一个字符，并把上一轮用过但还剩次数的字符延迟一轮再放回堆；两种写法本质相同。正确性还可以用必要条件解释：最高频字符需要插入到其他字符形成的空隙里，空隙数量最多是其他字符数加一，因此最高频不能超过 $(n+1)/2$ 。

任务调度器和重构字符串类似，但它要求相同任务之间至少间隔 n 个单位时间，可以插入空闲。模拟做法是每轮取最多 $n+1$ 个不同任务执行，剩余任务放回堆；如果任务还没做完而本轮不足 $n+1$ ，就需要补空闲。数学做法更快：设最高频为 maxFreq ，拥有最高频的任务种类数为 maxCount ，最短时间至少是 $(\text{maxFreq} - 1) * (n + 1) + \text{maxCount}$ ，再和任务总数取最大。

Listing 14.12 任务调度器：按最高频任务计算骨架长度

```

int least_interval(const std::vector<char>& tasks, int cooldown)
{
    std::array<int, 26> count{};
    for (const char task : tasks) {
        ++count[task - 'A'];
    }

    int max_frequency = 0;
    int max_count = 0;
    for (const int frequency : count) {
        if (frequency > max_frequency) {
            max_frequency = frequency;
            max_count = 1;
        } else if (frequency == max_frequency) {
            ++max_count;
        }
    }

    const int skeleton =
        (max_frequency - 1) * (cooldown + 1) + max_count;
}

```

```
return std::max(static_cast<int>(tasks.size()), skeleton);
}
```

公式的含义是：最高频任务形成 `max_frequency - 1` 个完整间隔块，每块长度至少 `cooldown + 1`，最后再放所有最高频任务的尾部。若其他任务足够多，它们可以填满空隙，答案就是任务总数；若不够，就必须空闲，答案由骨架决定。这个题很好地区分了“堆模拟”和“数学贪心”：堆模拟更通用，公式更简洁但依赖对结构的深理解。

课程表 III 是“按截止时间排序 + 最大堆反悔”的经典题。每门课有持续时间和截止时间，要求最多能上多少门。暴力搜索选课集合是指数级。贪心先按截止时间升序处理课程，尽量把当前课程加入计划；如果加入后总时间超过当前截止时间，就从已选课程中删除耗时最长的一门。为什么删最长？因为课程数量减少一门时，删除耗时最长能让总时间降得最多，给后续课程留下最大空间。

Listing 14.13 课程表 III：超过截止时删除最长课程

```
int schedule_course(std::vector<std::vector<int>> courses)
{
    std::sort(courses.begin(), courses.end(),
        [](const std::vector<int>& lhs, const std::vector<int>& rhs) {
            return lhs[1] < rhs[1];
        });

    std::priority_queue<int> selected_durations;
    int total_time = 0;
    for (const std::vector<int>& course : courses) {
        const int duration = course[0];
        const int deadline = course[1];
        selected_durations.push(duration);
        total_time += duration;

        if (total_time > deadline) {
            total_time -= selected_durations.top();
            selected_durations.pop();
        }
    }

    return static_cast<int>(selected_durations.size());
}
```

这个算法的堆不是为了取下一门课，而是为了在当前已选集合中执行“反悔”。按截止时间排序保证当处理当前课程时，只需要满足当前及更早截止时间；如果超时，必须删掉一门已选课程。删最长课程不会减少已选课程数量以外的可行性，且让总耗时最小。这个证明属于交换论证：如果某个可行方案删的是较短课程而保留较长课程，交换为保留较短、删除较长，课程数量不变，总时间不增。

无重叠区间也能从“反悔”角度理解。按起点排序扫描时，若当前区间和上一个保留区间重叠，就必须删除其中一个。为了给后续留下更多空间，应保留结束更早的那个，删除结束更晚的那个。按终点排序的版本更直接：每次选择结束最早且与已选区间不重叠的区间，最大化保留数量；答案也可以用总数减去最多保留数量。

Listing 14.14 无重叠区间：保留结束更早的区间

```
int erase_overlap_intervals(std::vector<std::vector<int>> intervals)
{
    if (intervals.empty()) {
        return 0;
    }
    std::sort(intervals.begin(), intervals.end(),
        [](const std::vector<int>& lhs, const std::vector<int>& rhs) {
            return lhs[0] < rhs[0];
        });

    int removed = 0;
    int current_end = intervals[0][1];
    for (int i = 1; i < static_cast<int>(intervals.size()); ++i) {
        if (intervals[i][0] >= current_end) {
            current_end = intervals[i][1];
            continue;
        }
    }
    return intervals.size() - removed;
}
```

```

    }
    ++removed;
    current_end = std::min(current_end, intervals[i][1]);
}
return removed;
}

```

当发生重叠时，`current_end = min(current_end, intervals[i][1])` 表示保留结束更早的区间。这个写法不显式记录保留哪个区间，但保留了对后续唯一重要的信息：当前已保留区间的最小结束时间。贪心题经常不需要保留完整历史，只保留能决定未来可行性的摘要状态。若题目要求输出具体删除哪些区间，就需要额外记录选择。

根据身高重建队列是排序和贪心的组合。每个人由 `height` 和 `k` 表示，`k` 是前面身高大于等于自己的人数。若从矮到高放，后面更高的人插入会影响矮个子的 `k`；若从高到矮放，已经放入的人都不矮于当前人，当前人的 `k` 就等于插入位置。排序规则是身高降序，身高相同则 `k` 升序，然后按 `k` 插入结果数组。

Listing 14.15 根据身高重建队列：高个先放

```

std::vector<std::vector<int>> reconstruct_queue(
    std::vector<std::vector<int>> people)
{
    std::sort(people.begin(), people.end(),
        [](const std::vector<int>& lhs, const std::vector<int>& rhs) {
            if (lhs[0] != rhs[0]) {
                return lhs[0] > rhs[0];
            }
            return lhs[1] < rhs[1];
        });

    std::vector<std::vector<int>> queue;
    queue.reserve(people.size());
    for (const std::vector<int>& person : people) {
        queue.insert(queue.begin() + person[1], person);
    }
    return queue;
}

```

这段代码的插入是 $O(n)$ ，整体 $O(n^2)$ ，但力扣规模通常可接受。若规模很大，可以用树状数组或线段树找第 `k` 个空位。正确性来自处理顺序：当插入当前人时，队列中已有的人都比他高或等高，因此插入下标正好决定了他前面符合条件的人数；后续更矮的人插入不会影响他的 `k`。同高的人按 `k` 升序处理，避免同高较大 `k` 先插入造成位置不稳定。

合并石头、戳气球、最优二叉搜索树这类题看上去也有“每次选局部最小或最大”的诱惑，但通常不能简单贪心。以合并石头为例，如果每次合并当前最小的相邻两堆，不一定得到全局最优，因为相邻约束会改变未来可合并结构。没有相邻约束的哈夫曼合并可以每次取最小两项，这是因为任意两项都可合并，且最优前缀树有明确交换性质；有相邻约束后，问题变成区间 DP。这个对比非常重要：同样是“合并代价”，约束不同，算法类型就不同。

柠檬水找零是简单贪心，但很适合训练“优先保留灵活资源”。顾客支付 5、10、20，饮料价格 5。收到 10 需要找一个 5；收到 20 时，优先找 10+5，而不是三个 5，因为 10 只能在找 20 时使用，5 更灵活，后续找 10 和 20 都需要。这个证明不是复杂数学，而是资源灵活性：越通用的资源越应该保留。若无法找零则返回 `false`。

Listing 14.16 柠檬水找零：优先使用较不灵活的 10 元

```

bool lemonade_change(const std::vector<int>& bills)
{
    int five = 0;
    int ten = 0;
    for (const int bill : bills) {
        if (bill == 5) {
            ++five;
        } else if (bill == 10) {
            if (five == 0) {
                return false;
            }
        }
    }
}

```

```

        --five;
        ++ten;
    } else {
        if (ten > 0 && five > 0) {
            --ten;
            --five;
        } else if (five >= 3) {
            five -= 3;
        } else {
            return false;
        }
    }
}
return true;
}

```

这个题也能说明贪心证明可以很朴素。不是所有贪心都需要长篇交换论证，有时只要明确资源的支配关系：一张 5 元可以参与两类找零，一张 10 元只能参与 20 元找零；在能完成当前找零的前提下，消耗更不灵活的资源不会让未来更差。若题目币值变化，这个策略可能失效，因此不能把它泛化成“永远优先用大面额”。

最小加油次数是堆和贪心的又一个“反悔”模型。汽车沿路经过加油站，每个站有位置和油量，目标是用最少加油次数到达终点。暴力搜索在哪些站加油是指数级。贪心扫描所有可达加油站，把它们的油量放入大顶堆；当当前油量无法到达下一个站或终点时，就从已经路过且可选择的站中取油量最大的一个加油。虽然看起来像回头加油，但等价于在之前经过时选择了那个站。

Listing 14.17 最小加油次数：不可达时从已过站点中选最大油量

```

int min_refuel_stops(int target,
                    int start_fuel,
                    std::vector<std::vector<int>> stations)
{
    stations.push_back({target, 0});
    std::priority_queue<int> fuels;
    int fuel = start_fuel;
    int previous_position = 0;
    int stops = 0;

    for (const std::vector<int>& station : stations) {
        const int position = station[0];
        const int available_fuel = station[1];
        fuel -= position - previous_position;
        while (fuel < 0 && !fuels.empty()) {
            fuel += fuels.top();
            fuels.pop();
            ++stops;
        }
        if (fuel < 0) {
            return -1;
        }
        fuels.push(available_fuel);
        previous_position = position;
    }

    return stops;
}

```

这个算法的正确性来自延迟决策：在还没被迫加油之前，不决定具体在哪个已过站点加；一旦必须加油，为了让未来最容易可达，就选择已过站点中油量最大的。选择更小油量不会减少加油次数，反而可能更早再次陷入不可达。堆中保存的是已经经过但尚未使用的加油机会。这个模型和 IPO、课程表 III 都有相似结构：先收集可用候选，必要时选最有利的一个。

贪心题的讲解应该包含失败测试。跳跃游戏 II 如果每一步只跳到当前能跳到的最远下标，可能不是最少跳数；正确做法是按当前层覆盖范围推进，选择下一层最远覆盖。课程表 III 如果只按持续时间短排序，会错过截止时间约束；如果只按截止时间排序但不反悔，也会被长课程拖垮。重构字符串如果每次只取最高频字符但不延迟上一字符，就可能产生相邻重复。把这些失败原因讲出来，答案才可信。

堆与贪心实验：第一，为零钱兑换构造 1,3,4 和目标 6，写出贪心失败过程，再用 DP 修正。第二，给课程表 III 构造一门很长但截止早的课程，观察最大堆如何反悔。第三，用任务调度器分别实现堆模拟和公式法，比较输出一致性。第四，把最小加油次数中的大顶堆改成小顶堆，找一个用例说明为什么会增加加油次数或失败。

堆和贪心题复盘模板

堆题先说明堆里保存什么候选，堆顶为什么是下一步最该处理的对象，候选何时入堆、何时出堆、何时过期。贪心题先给暴力或 DP，再说明局部选择为什么不会影响全局最优，最好用交换论证或领先性论证。遇到 Top K，主动比较排序、大小为 K 的堆、快速选择和桶排序；遇到区间覆盖，主动说明排序键和边界含义。

第 15 章

图论进阶：并查集、最短路和状态图

图论进阶题的难点通常不在 BFS 或 DFS 的基本写法，而在选择正确的图模型。无向连通性适合并查集；无权最短路适合 BFS；边权非负的最短路适合 Dijkstra；存在状态压缩或多维状态时，要先定义状态节点，再决定边和代价。很多题目表面不是图，例如密码锁、单词变化、网格体力消耗、课程安排、账户合并，只要存在“状态”和“一步转移”，就可以用图论语言重写。

并查集解决的是动态连通性。它不回答路径是什么，也不回答最短距离是多少；它只维护“两个元素当前是否属于同一集合”。暴力做法在每次合并后用 DFS 或 BFS 检查连通性，会重复扫描大量边。并查集把每个集合表示成一棵树，用根节点作为代表。路径压缩让查询时把沿途节点直接挂到根上，按大小合并让小树挂到大树下，两者结合后，均摊复杂度接近常数。

Listing 15.1 并查集：迭代 find、路径压缩和按大小合并

```
class DisjointSetUnion {
public:
    explicit DisjointSetUnion(int n)
        : parent_(n), size_(n, 1)
    {
        std::iota(this->parent_.begin(), this->parent_.end(), 0);
    }

    int find(int x)
    {
        int root = x;
        while (this->parent_[root] != root) {
            root = this->parent_[root];
        }
        while (this->parent_[x] != x) {
            const int next = this->parent_[x];
            this->parent_[x] = root;
            x = next;
        }
        return root;
    }

    bool unite(int a, int b)
    {
        int root_a = this->find(a);
        int root_b = this->find(b);
        if (root_a == root_b) {
            return false;
        }
        if (this->size_[root_a] < this->size_[root_b]) {
            std::swap(root_a, root_b);
        }
        this->parent_[root_b] = root_a;
        this->size_[root_a] += this->size_[root_b];
        return true;
    }

private:
    std::vector<int> parent_;
    std::vector<int> size_;
};
```

这里的 `find` 是迭代写法，避免递归深度问题。第一次循环找根，第二次循环压缩路径。`unite` 返回 `false` 表示两个元素本来已经连通，合并会形成环。这个返回值在很多题里正好就是答案判断条件。成员访问使用 `this->`，让读者清楚哪些状态属于并查集对象。

冗余连接是并查集的标准例题。题目给一棵树额外加一条边，要求找出导致环的边。暴力方法是每加入一条边，就在已有图中检查这两个端点是否已经连通；如果已经连通，再加边就会成环。并查集正好维护这个关系：扫描边时，如果 `unite(u, v)` 返回 `false`，说明这条边连接了同一集合内

的两个点，它就是冗余边。

Listing 15.2 冗余连接：合并失败的边就是成环边

```
std::vector<int> find_redundant_connection(
    const std::vector<std::vector<int>>& edges)
{
    DisjointSetUnion dsu(static_cast<int>(edges.size() + 1));
    for (const std::vector<int>& edge : edges) {
        const int u = edge[0];
        const int v = edge[1];
        if (!dsu.unite(u, v)) {
            return edge;
        }
    }
    return {};
}
```

这题要注意节点编号通常从 1 开始，所以并查集大小开到 `edges.size() + 1`。如果题目节点编号不连续，就需要哈希映射到连续编号。复杂度近似 $O(n)$ 。面试表达中要把树的性质说出来：一棵 n 个节点的树有 $n-1$ 条边且无环；额外加入一条边后，第一次发现两个端点已经连通的边，就是让无环结构变成有环结构的边。

省份数量也是连通块问题。输入是邻接矩阵，`isConnected[i][j] == 1` 表示城市 i 和 j 直接连接。DFS/BFS 可以做，并查集也可以做。暴力思路可能是从每个城市出发搜索所有可达城市并统计，但会重复访问。并查集做法扫描矩阵上三角，把直接连接的城市合并，最后统计根节点个数。

Listing 15.3 省份数量：邻接矩阵合并连通城市

```
int find_circle_num(const std::vector<std::vector<int>>& is_connected)
{
    const int n = static_cast<int>(is_connected.size());
    DisjointSetUnion dsu(n);

    for (int i = 0; i < n; ++i) {
        for (int j = i + 1; j < n; ++j) {
            if (is_connected[i][j] == 1) {
                dsu.unite(i, j);
            }
        }
    }

    int provinces = 0;
    for (int city = 0; city < n; ++city) {
        if (dsu.find(city) == city) {
            ++provinces;
        }
    }
    return provinces;
}
```

邻接矩阵是对称的，因此只扫描上三角即可。这里并查集的优势不是绝对性能，而是模型清晰：省份就是连通分量，直接连接就是合并操作。若题目还要求列出每个省份包含哪些城市，则并查集统计完根后还要按根分组；若题目要求最短路径，并查集就不够，因为它丢掉了路径长度信息。

Dijkstra 处理非负边权最短路。无权图 BFS 成立，是因为每条边代价相同，按层访问保证第一次到达最短；带权图中，一条边可能代价很大，普通 BFS 不再可靠。Dijkstra 的贪心点是：每次从尚未确定的节点中取当前距离最小者，它的最短距离已经确定。这个结论依赖所有边权非负，因为后续绕路不可能通过负边把距离降得更低。

网络延迟时间是 Dijkstra 的典型题。节点是服务器，有向边带传播时间。题目问从起点发信号，到所有节点都收到信号需要多久，也就是起点到所有节点最短路的最大值。暴力方法可以反复松弛所有边，类似 Bellman-Ford，复杂度 $O(nm)$ 。由于边权非负，用堆优化 Dijkstra 更合适。

Listing 15.4 网络延迟时间：堆优化 Dijkstra

```
int network_delay_time(const std::vector<std::vector<int>>& times,
    int n,
    int source)
```

```

{
    using Edge = std::pair<int, int>; // neighbor, weight
    std::vector<std::vector<Edge>> graph(n + 1);
    for (const std::vector<int>& edge : times) {
        graph[edge[0]].emplace_back(edge[1], edge[2]);
    }

    constexpr int INF = 1'000'000'000;
    std::vector<int> distance(n + 1, INF);
    distance[source] = 0;

    using State = std::pair<int, int>; // distance, node
    std::priority_queue<State, std::vector<State>, std::greater<State>> heap;
    heap.emplace(0, source);

    while (!heap.empty()) {
        const auto [dist, node] = heap.top();
        heap.pop();
        if (dist != distance[node]) {
            continue;
        }

        for (const auto [next, weight] : graph[node]) {
            if (dist + weight < distance[next]) {
                distance[next] = dist + weight;
                heap.emplace(distance[next], next);
            }
        }
    }

    int answer = 0;
    for (int node = 1; node <= n; ++node) {
        if (distance[node] == INF) {
            return -1;
        }
        answer = std::max(answer, distance[node]);
    }
    return answer;
}

```

这段代码没有使用显式 `visited`，而是用 `dist != distance[node]` 跳过堆中的过期状态。因为 C++ 标准堆不支持 decrease-key，更新距离时直接把新状态再压入堆，旧状态留在堆里，弹出时识别并跳过。复杂度是 $O((n+m) \log m)$ 或常写为 $O(m \log n)$ 。如果存在负边，Dijkstra 不适用，应考虑 Bellman-Ford 或 SPFA 的风险分析。

最小体力消耗路径看似网格题，实质是最短路变体。路径代价不是边权之和，而是路径上最大高度差的最小可能值。状态是格子，边是上下左右相邻格子，走一条边的代价是高度差。到达某个格子的最优体力定义为：从起点到它的路径中，最大边权尽可能小。转移时，新代价是 $\max(\text{当前路径体力}, \text{新边高度差})$ 。这个代价仍满足 Dijkstra 的非负和单调扩展性质。

Listing 15.5 最小体力路径：以最大边权作为路径代价

```

int minimum_effort_path(const std::vector<std::vector<int>>& heights)
{
    constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
        std::pair{1, 0}, std::pair{-1, 0},
        std::pair{0, 1}, std::pair{0, -1}
    };
    const int rows = static_cast<int>(heights.size());
    const int cols = static_cast<int>(heights[0].size());
    constexpr int INF = 1'000'000'000;

    std::vector<std::vector<int>> effort(rows, std::vector<int>(cols, INF));
    effort[0][0] = 0;

    using State = std::tuple<int, int, int>; // effort, row, col
    std::priority_queue<State, std::vector<State>, std::greater<State>> heap;
    heap.emplace(0, 0, 0);

    while (!heap.empty()) {
        const auto [cost, row, col] = heap.top();

```

```

    heap.pop();
    if (row == rows - 1 && col == cols - 1) {
        return cost;
    }
    if (cost != effort[row][col]) {
        continue;
    }

    for (const auto [dr, dc] : DIRECTIONS) {
        const int next_row = row + dr;
        const int next_col = col + dc;
        const bool out_of_bounds =
            next_row < 0 || next_row >= rows ||
            next_col < 0 || next_col >= cols;
        if (out_of_bounds) {
            continue;
        }

        const int edge_cost =
            std::abs(heights[row][col] - heights[next_row][next_col]);
        const int next_cost = std::max(cost, edge_cost);
        if (next_cost < effort[next_row][next_col]) {
            effort[next_row][next_col] = next_cost;
            heap.emplace(next_cost, next_row, next_col);
        }
    }
}

return 0;
}

```

这题也可以二分答案加 BFS：猜一个最大允许体力，看能否只走高度差不超过该值的边到终点。二分答案复杂度是 $O(mn \log H)$ ，Dijkstra 是 $O(mn \log(mn))$ 。两种方法都合理，面试时可以先给更统一的 Dijkstra，再说明二分可行的原因是“体力上限越大，可达性越强”，存在单调性。

状态图题还包括打开转盘锁、单词接龙、蛇梯棋等。共同点是输入没有直接给边，需要你根据规则生成邻居。生成邻居时要注意两件事：第一，状态编码要能高效去重；第二，访问标记要在入队时设置，避免同一状态被多个父状态重复加入。无权最短路使用 BFS，第一次到达目标即为最短；带权状态图则要换 Dijkstra 或 A*，不能强行 BFS。

真正写图论题之前，还要先决定图如何存储。邻接矩阵适合节点数量较小、需要频繁判断任意两点是否有边的场景，空间是 $O(n^2)$ ；邻接表适合稀疏图，空间是 $O(n + m)$ ；边数组适合 Kruskal、Bellman-Ford 这类按边扫描的算法；网格图通常不需要显式建图，用坐标和方向数组现场生成邻居即可。很多低质量答案只说“建图”，却没有说明为什么这样建。面试中应该先看输入规模：如果 n 可以到十万，邻接矩阵基本不可能；如果 m 远小于 n^2 ，邻接表更自然；如果题目要求按边权排序，边数组会比邻接表更直接。

邻接表也有工程差异。`vector<vector<pair<int,int>>>` 写法直观，但每个内层 `vector` 都是独立对象，极端情况下会产生很多小分配；竞赛中常见的前向星把所有边放进几个连续数组，通过 `head` 和 `next` 串起来，局部性更好，但代码可读性低。刷题面试通常以正确性和表达清晰为第一优先，选 `vector` 邻接表即可；当题目数据极大或需要讲性能时，可以补充连续边数组的思路。这个判断体现的是工程能力：知道什么时候写简单代码，什么时候需要控制内存布局。

Listing 15.6 边数组转邻接表：保留输入语义并便于最短路

```

struct WeightedEdge {
    int from;
    int to;
    int weight;
};

std::vector<std::vector<std::pair<int, int>>> build_directed_graph(
    int node_count,
    const std::vector<WeightedEdge>& edges)
{
    std::vector<std::vector<std::pair<int, int>>> graph(node_count);
    for (const WeightedEdge& edge : edges) {
        graph[edge.from].emplace_back(edge.to, edge.weight);
    }
}

```

```

return graph;
}

```

这段代码看似简单，但它体现了一个重要习惯：输入边和算法图结构分开。`WeightedEdge` 保留题目给出的边语义，邻接表服务于后续算法。若后面同时要跑 Kruskal 和 Dijkstra，就不必从邻接表反推出边数组。很多工程 bug 来自过早把所有信息塞进一种结构，后面算法一换就需要反复转换。

BFS 的本质是按距离层扩展。普通队列能保证这一点，因为所有边权都是 1。访问标记必须在入队时设置，而不是出队时设置；如果出队时才标记，一个节点可能被同一层的多个父节点重复入队，轻则浪费时间，重则在记录父节点或计数时产生错误。多源 BFS 只是把多个起点同时放入队列，把它们的初始距离都设为 0。腐烂的橘子、地图分析、离所有建筑最近的空地等题，都可以用这个模型理解。

Listing 15.7 多源 BFS：所有起点同时入队

```

int shortest_distance_from_sources(
    const std::vector<std::vector<int>>& grid,
    const std::vector<std::pair<int, int>>& sources)
{
    constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
        std::pair{1, 0}, std::pair{-1, 0},
        std::pair{0, 1}, std::pair{0, -1}
    };
    const int rows = static_cast<int>(grid.size());
    const int cols = static_cast<int>(grid[0].size());

    std::vector<std::vector<int>> distance(rows, std::vector<int>(cols, -1));
    std::queue<std::pair<int, int>> queue;
    for (const auto [row, col] : sources) {
        distance[row][col] = 0;
        queue.emplace(row, col);
    }

    int farthest = 0;
    while (!queue.empty()) {
        const auto [row, col] = queue.front();
        queue.pop();
        farthest = std::max(farthest, distance[row][col]);

        for (const auto [dr, dc] : DIRECTIONS) {
            const int next_row = row + dr;
            const int next_col = col + dc;
            const bool out_of_bounds =
                next_row < 0 || next_row >= rows ||
                next_col < 0 || next_col >= cols;
            if (out_of_bounds || grid[next_row][next_col] == 1 ||
                distance[next_row][next_col] != -1) {
                continue;
            }
            distance[next_row][next_col] = distance[row][col] + 1;
            queue.emplace(next_row, next_col);
        }
    }

    return farthest;
}

```

这段模板里，`distance == -1` 同时承担“尚未访问”和“距离未知”的含义。若题目需要允许障碍、空地、起点不同类型，就不要直接修改原数组，单独开 `distance` 会更安全。复杂度是 $O(mn)$ ，因为每个格子最多入队一次，每条网格边最多检查常数次。面试时要强调：多源 BFS 不是从每个源分别 BFS，而是把所有源放在第 0 层同时扩散；这样第一次到达某个点的源，就是离它最近的一批源之一。

0-1 BFS 是普通 BFS 和 Dijkstra 之间的桥梁。当边权只可能是 0 或 1 时，使用优先队列当然正确，但 `deque` 可以把复杂度降到线性。权重为 0 的转移放到队首，权重为 1 的转移放到队尾。这样队列中的状态仍然按照非降距离处理，只是用双端队列代替了堆。最典型的题是网格中每个格子给出推荐方向，顺着方向走代价为 0，改方向代价为 1，求从左上到右下的最小修改次数。

Listing 15.8 0-1 BFS：零代价边入队首，一代价边入队尾

```

int min_cost_valid_path(const std::vector<std::vector<int>>& grid)
{
    constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
        std::pair{0, 1}, std::pair{0, -1},
        std::pair{1, 0}, std::pair{-1, 0}
    };
    constexpr int INF = 1'000'000'000;
    const int rows = static_cast<int>(grid.size());
    const int cols = static_cast<int>(grid[0].size());

    std::vector<std::vector<int>> distance(rows, std::vector<int>(cols, INF));
    std::deque<std::pair<int, int>> deque;
    distance[0][0] = 0;
    deque.emplace_front(0, 0);

    while (!deque.empty()) {
        const auto [row, col] = deque.front();
        deque.pop_front();

        for (int dir = 0; dir < static_cast<int>(DIRECTIONS.size()); ++dir) {
            const auto [dr, dc] = DIRECTIONS[dir];
            const int next_row = row + dr;
            const int next_col = col + dc;
            const bool out_of_bounds =
                next_row < 0 || next_row >= rows ||
                next_col < 0 || next_col >= cols;
            if (out_of_bounds) {
                continue;
            }

            const int extra_cost = (grid[row][col] == dir + 1) ? 0 : 1;
            const int next_distance = distance[row][col] + extra_cost;
            if (next_distance >= distance[next_row][next_col]) {
                continue;
            }

            distance[next_row][next_col] = next_distance;
            if (extra_cost == 0) {
                deque.emplace_front(next_row, next_col);
            } else {
                deque.emplace_back(next_row, next_col);
            }
        }
    }

    return distance[rows - 1][cols - 1];
}

```

0-1 BFS 的正确性可以这样解释：普通 BFS 每次扩展当前最小层；Dijkstra 每次从堆里取最小距离；当新增边代价只有 0 或 1 时，当前状态的后继要么仍在当前距离层，要么进入下一距离层。放队首和队尾正好维护了这个层次。易错点有三个：第一，不能用普通 `queue`，因为 0 代价边应该优先处理；第二，不能只用 `visited`，因为某个状态可能先以较大代价到达，后来被较小代价改进；第三，方向编号要和题目约定对齐，示例里题目方向从 1 开始，所以比较时使用 `dir + 1`。

拓扑排序处理有向无环图。它回答的不是最短路，而是依赖顺序：如果一件事必须在另一件事之前完成，就可以画一条有向边。课程表就是标准题。暴力想法是不断找没有前置依赖的课程，删除它，再更新其他课程；拓扑排序把这个过程用入度数组和队列高效实现。若最后处理课程数小于总课程数，说明存在环，环内课程互相等待，无法完成。

Listing 15.9 课程表：Kahn 拓扑排序判断是否有环

```

bool can_finish(int course_count,
                const std::vector<std::vector<int>>& prerequisites)
{
    std::vector<std::vector<int>> graph(course_count);
    std::vector<int> indegree(course_count, 0);

    for (const std::vector<int>& relation : prerequisites) {
        const int course = relation[0];

```

```

    const int dependency = relation[1];
    graph[dependency].push_back(course);
    ++indegree[course];
}

std::queue<int> ready;
for (int course = 0; course < course_count; ++course) {
    if (indegree[course] == 0) {
        ready.push(course);
    }
}

int processed = 0;
while (!ready.empty()) {
    const int course = ready.front();
    ready.pop();
    ++processed;

    for (const int next : graph[course]) {
        --indegree[next];
        if (indegree[next] == 0) {
            ready.push(next);
        }
    }
}

return processed == course_count;
}

```

这题最容易写反边。题目常写成 $[a, b]$ 表示想学 a 必须先学 b ，所以边应该是 $b \rightarrow a$ 。入度表示还有多少依赖没有满足。队列中放的是当前已经没有前置依赖的课程。正确性来自不变量：队列中的每门课入度为 0，已经可以安排；处理它等价于完成这门课，因此它指向的后继少一个未满足依赖。若图中有环，环上每个点至少依赖环内另一个点，永远不会全部入队。

拓扑排序不只判断能不能完成，还能在 DAG 上做动态规划。只要图无环，就可以按拓扑顺序保证前驱状态已经计算完，再更新后继。比如有向无环图中求从起点到每个点的最长路径，不能用 Dijkstra，因为目标是最长而不是最短；也不能在有环图上直接做，因为正权环会让最长路径没有上界。DAG 的拓扑顺序让“阶段”变得明确。

Listing 15.10 DAG 最长路径：拓扑序上的动态规划

```

std::vector<int> longest_path_in_dag(
    int node_count,
    const std::vector<std::tuple<int, int, int>>& edges,
    int source)
{
    std::vector<std::vector<std::pair<int, int>>> graph(node_count);
    std::vector<int> indegree(node_count, 0);
    for (const auto [from, to, weight] : edges) {
        graph[from].emplace_back(to, weight);
        ++indegree[to];
    }

    std::queue<int> ready;
    for (int node = 0; node < node_count; ++node) {
        if (indegree[node] == 0) {
            ready.push(node);
        }
    }

    constexpr int NEG_INF = -1'000'000'000;
    std::vector<int> distance(node_count, NEG_INF);
    distance[source] = 0;

    while (!ready.empty()) {
        const int node = ready.front();
        ready.pop();
        if (distance[node] != NEG_INF) {
            for (const auto [next, weight] : graph[node]) {
                distance[next] = std::max(distance[next], distance[node] + weight);
            }
        }
    }
}

```

```

    }
    for (const auto [next, weight] : graph[node]) {
        (void)weight;
        --indegree[next];
        if (indegree[next] == 0) {
            ready.push(next);
        }
    }
}
return distance;
}

```

上面代码故意把“松弛后继”和“减少入度”分开写，便于读者看清两个动作。实际工程中可以合并循环，但教材里先强调概念更重要。DAG DP 的状态是 `distance[v]`，含义是从 `source` 到 `v` 的最大路径和；转移来自所有前驱。拓扑序保证当某个节点出队时，它的所有前驱已经处理完，状态不再被未来节点回头修改。这个结构和一维 DP 很像：所谓“阶段顺序”，在图上就是拓扑顺序。

如果图中存在负边，Dijkstra 的贪心基础会崩塌。因为一个当前距离较大的节点，未来可能通过负边把另一个已经确定的节点拉得更小。Bellman-Ford 的想法更朴素：一条最短路径如果最多包含 k 条边，那么做 k 轮按边松弛就能得到它。经典最短路最多需要 $n - 1$ 条边，因为无负环的最短简单路径不会重复经过节点。力扣中“最多经过 K 站中转”的机票题，本质是限制边数的 Bellman-Ford。

Listing 15.11 K 站中转内最便宜航班：按边数分层松弛

```

int find_cheapest_price(int n,
                        const std::vector<std::vector<int>>& flights,
                        int source,
                        int target,
                        int max_stops)
{
    constexpr int INF = 1'000'000'000;
    std::vector<int> previous(n, INF);
    previous[source] = 0;

    for (int step = 0; step <= max_stops; ++step) {
        std::vector<int> current = previous;
        for (const std::vector<int>& flight : flights) {
            const int from = flight[0];
            const int to = flight[1];
            const int price = flight[2];
            if (previous[from] == INF) {
                continue;
            }
            current[to] = std::min(current[to], previous[from] + price);
        }
        previous = std::move(current);
    }

    return previous[target] == INF ? -1 : previous[target];
}

```

这里必须使用 `previous` 和 `current` 两层数组，不能在同一个数组里原地更新。原因是第 `step` 轮只允许使用最多 `step + 1` 条边，如果原地更新，一轮内刚更新出来的状态又被后面的边继续使用，就会偷偷使用更多边。这个问题和 0-1 背包的倒序遍历本质类似：转移要读取上一阶段，不能让当前阶段污染自己。复杂度是 $O((K + 1)m)$ ，空间 $O(n)$ 。

最小生成树回答的是“把所有点连起来的最低总成本”，它不是最短路。最短路关心从某个起点到其他点的路径长度；最小生成树关心选出一组边，让全图连通且总权重最小。Kruskal 的策略是按边权从小到大扫描，能连接两个不同连通块就选，否则跳过。并查集用来判断一条边是否会成环。这个贪心正确性来自割性质：任意一个割上，权重最小的跨割边一定存在某棵最小生成树包含它。

Listing 15.12 Kruskal：按边权连接不同连通块

```

struct UndirectedEdge {
    int u;
    int v;
    int weight;
};

```

```

int minimum_spanning_tree_cost(int node_count,
                               std::vector<UndirectedEdge> edges)
{
    std::sort(edges.begin(), edges.end(),
              [](const UndirectedEdge& lhs, const UndirectedEdge& rhs) {
                  return lhs.weight < rhs.weight;
              });

    DisjointSetUnion dsu(node_count);
    int total_cost = 0;
    int used_edges = 0;

    for (const UndirectedEdge& edge : edges) {
        if (!dsu.unite(edge.u, edge.v)) {
            continue;
        }
        total_cost += edge.weight;
        ++used_edges;
        if (used_edges == node_count - 1) {
            return total_cost;
        }
    }

    return -1;
}

```

面试中区分 Kruskal 和 Prim 的方式很简单：Kruskal 站在“边”的视角，排序后用并查集选边；Prim 站在“点”的视角，从一个连通块开始，每次选连接外部点的最小边。边比较少、已经容易生成边数组时，Kruskal 很顺；完全图且边很多时，Prim 可以避免显式存储所有边。连接所有点的最小费用中，点与点之间距离是曼哈顿距离，朴素建完全图有 $O(n^2)$ 条边， n 不大时可以接受；如果规模上去，就要研究几何图优化，而不是盲目套模板。

并查集还能和二分答案结合。最小体力路径除了 Dijkstra，也可以二分一个体力上限 `limit`，只把高度差不超过 `limit` 的相邻格子合并，最后看起点和终点是否连通。若某个 `limit` 可行，那么更大的上限一定可行；若不可行，更小的上限一定不可行。这就是二分的单调性。这个方法的优点是解释直观，缺点是每次检查都要重新扫描网格。

Listing 15.13 最小体力路径：二分答案加并查集

```

bool can_reach_with_effort(const std::vector<std::vector<int>>& heights,
                          int limit)
{
    const int rows = static_cast<int>(heights.size());
    const int cols = static_cast<int>(heights[0].size());
    DisjointSetUnion dsu(rows * cols);

    auto id = [cols](int row, int col) {
        return row * cols + col;
    };

    for (int row = 0; row < rows; ++row) {
        for (int col = 0; col < cols; ++col) {
            if (row + 1 < rows &&
                std::abs(heights[row][col] - heights[row + 1][col]) <= limit) {
                dsu.unite(id(row, col), id(row + 1, col));
            }
            if (col + 1 < cols &&
                std::abs(heights[row][col] - heights[row][col + 1]) <= limit) {
                dsu.unite(id(row, col), id(row, col + 1));
            }
        }
    }

    return dsu.find(0) == dsu.find(rows * cols - 1);
}

int minimum_effort_path_by_binary_search(
    const std::vector<std::vector<int>>& heights)
{
    int low = 0;

```

```

int high = 1'000'000;
while (low < high) {
    const int mid = low + (high - low) / 2;
    if (can_reach_with_effort(heights, mid)) {
        high = mid;
    } else {
        low = mid + 1;
    }
}
return low;
}

```

这段代码只合并向下和向右的边，是因为无向网格中每条相邻边检查一次即可。复杂度是 $O(mn \log H \cdot \alpha(mn))$ ，其中 H 是高度差范围。和 Dijkstra 相比，二分并查集依赖答案单调性；Dijkstra 依赖路径代价的单调扩展性质。两种方法各有适用场景。面试时能同时说出这两种建模，说明你不是在背题，而是在识别“最小化最大值”的结构。

账户合并是并查集的另一个高频场景。题目给出若干账户，每个账户包含姓名和邮箱；只要两个账户共享邮箱，就属于同一个人。暴力方法是两两比较账户邮箱集合，复杂度可能达到 $O(n^2s)$ ，其中 s 是邮箱数量。更好的模型是：邮箱是节点，同一个账户里的所有邮箱属于同一集合。扫描账户时，把第一个邮箱和后续邮箱合并；最后按根收集邮箱，再排序输出。

Listing 15.14 账户合并：把邮箱映射为并查集节点

```

std::vector<std::vector<std::string>> accounts_merge(
    const std::vector<std::vector<std::string>>& accounts)
{
    std::unordered_map<std::string, int> email_id;
    std::unordered_map<std::string, std::string> email_name;

    for (const std::vector<std::string>& account : accounts) {
        const std::string& name = account[0];
        for (int i = 1; i < static_cast<int>(account.size()); ++i) {
            const std::string& email = account[i];
            if (!email_id.contains(email)) {
                const int id = static_cast<int>(email_id.size());
                email_id.emplace(email, id);
            }
            email_name[email] = name;
        }
    }

    DisjointSetUnion dsu(static_cast<int>(email_id.size()));
    for (const std::vector<std::string>& account : accounts) {
        const int first_id = email_id[account[1]];
        for (int i = 2; i < static_cast<int>(account.size()); ++i) {
            dsu.unite(first_id, email_id[account[i]]);
        }
    }

    std::unordered_map<int, std::vector<std::string>> groups;
    for (const auto& [email, id] : email_id) {
        groups[dsu.find(id)].push_back(email);
    }

    std::vector<std::vector<std::string>> answer;
    answer.reserve(groups.size());
    for (auto& [root, emails] : groups) {
        (void)root;
        std::sort(emails.begin(), emails.end());
        std::vector<std::string> merged;
        merged.reserve(emails.size() + 1);
        merged.push_back(email_name[emails.front()]);
        merged.insert(merged.end(), emails.begin(), emails.end());
        answer.push_back(std::move(merged));
    }
    return answer;
}

```

这里的关键不是并查集代码，而是节点选择。若把账户编号作为节点，也能合并共享邮箱的账户，但需要用邮箱反查已经出现过的账户编号；若把邮箱作为节点，则最后天然得到邮箱分组。由于

输出要求邮箱有序，每个组还要排序。姓名通常不用于判断合并，只用于输出；如果真实业务中同一邮箱对应多个姓名，就要定义冲突规则，不能默认题目条件成立。刷题题解可以借题目保证，工程代码必须把数据一致性说清楚。

单词接龙是状态图建模的代表。每个单词是节点，若两个单词只差一个字符，就有一条无权边。暴力建图需要两两比较单词，复杂度高。更好的做法是用通配模式建立中间索引，例如 `hot` 可以生成 `*ot`、`h*t`、`ho*`。所有拥有同一通配模式的单词互为相邻候选。搜索时从当前单词生成模式，再找到候选单词。由于边权相同，用 BFS，第一次到达目标就是最短转换长度。

Listing 15.15 单词接龙：通配模式建索引后 BFS

```
int ladder_length(const std::string& begin_word,
                 const std::string& end_word,
                 const std::vector<std::string& word_list)
{
    std::unordered_set<std::string> dictionary(word_list.begin(), word_list.end());
    if (!dictionary.contains(end_word)) {
        return 0;
    }
    dictionary.insert(begin_word);

    std::unordered_map<std::string, std::vector<std::string>> buckets;
    for (const std::string& word : dictionary) {
        for (int i = 0; i < static_cast<int>(word.size()); ++i) {
            std::string pattern = word;
            pattern[i] = '*';
            buckets[pattern].push_back(word);
        }
    }

    std::queue<std::string> queue;
    std::unordered_map<std::string, int> distance;
    queue.push(begin_word);
    distance[begin_word] = 1;

    while (!queue.empty()) {
        const std::string word = queue.front();
        queue.pop();
        if (word == end_word) {
            return distance[word];
        }

        for (int i = 0; i < static_cast<int>(word.size()); ++i) {
            std::string pattern = word;
            pattern[i] = '*';
            for (const std::string& next : buckets[pattern]) {
                if (distance.contains(next)) {
                    continue;
                }
                distance[next] = distance[word] + 1;
                queue.push(next);
            }
        }
    }

    return 0;
}
```

这段代码为了表达清楚保留了字符串拷贝。面试里可以进一步优化：每个模式桶被访问后清空，避免不同路径反复扫描同一桶；也可以使用双向 BFS，从起点集合和终点集合中较小的一侧扩展，降低搜索宽度。暴力为什么慢？因为单词数量为 n 、长度为 L 时，两两比较是 $O(n^2L)$ ；通配索引构建约为 $O(nL^2)$ ，搜索时主要消耗在模式生成和桶扫描。若使用 `string_view` 或预编码单词，工程上还能减少临时字符串分配，但面试中要先保证模型清楚。

双向 BFS 的优化依据是搜索树宽度。单向 BFS 深度为 d 、平均分支因子为 b 时，可能访问约 b^d 个状态；双向 BFS 从两端各扩展约 $d/2$ 层，理想情况下访问量接近 $2b^{d/2}$ 。这不是改变最坏复杂度的魔法，而是利用了许多状态图的指数扩展特征。写双向 BFS 时，永远扩展当前较小的一侧，并在生成新状态时检查它是否已经出现在另一侧。如果相遇，当前步数就是答案。

Listing 15.16 打开转盘锁：双向 BFS 的状态集合扩展

```

int open_lock(const std::vector<std::string>& deadends,
              const std::string& target)
{
    const std::unordered_set<std::string> dead(deadends.begin(), deadends.end());
    const std::string start = "0000";
    if (dead.contains(start)) {
        return -1;
    }
    if (target == start) {
        return 0;
    }

    auto neighbors = [](const std::string& state) {
        std::vector<std::string> result;
        result.reserve(8);
        for (int i = 0; i < static_cast<int>(state.size()); ++i) {
            for (const int delta : {-1, 1}) {
                std::string next = state;
                const int digit = state[i] - '0';
                next[i] = static_cast<char>('0' + (digit + delta + 10) % 10);
                result.push_back(std::move(next));
            }
        }
        return result;
    };

    std::unordered_set<std::string> front{start};
    std::unordered_set<std::string> back{target};
    std::unordered_set<std::string> visited{start, target};
    int steps = 0;

    while (!front.empty() && !back.empty()) {
        if (front.size() > back.size()) {
            std::swap(front, back);
        }
        std::unordered_set<std::string> next_front;
        ++steps;

        for (const std::string& state : front) {
            for (const std::string& next : neighbors(state)) {
                if (dead.contains(next) || visited.contains(next)) {
                    continue;
                }
                if (back.contains(next)) {
                    return steps;
                }
                visited.insert(next);
                next_front.insert(next);
            }
        }
        front = std::move(next_front);
    }

    return -1;
}

```

转盘锁的状态空间最多一万个，因此普通 BFS 也足够；这里展示双向 BFS，是为了训练状态集合扩展的写法。注意 `visited` 同时包含两端访问过的状态，避免重复回头。若先检查 `visited` 再检查 `back`，可能会错过相遇状态，所以上面代码对新状态的处理顺序是：死点跳过，已访问跳过，相遇返回，再加入下一层。也可以把相遇检查放在已访问之前，并单独维护两侧访问集；关键是语义一致。

图题中还有一类“边不是原题里的动作，而是你定义出来的关系”。比如岛屿数量把相邻陆地视为同一连通块；方程除法把变量视为节点、比例视为带权有向边；除法求值时， $a / b = 2$ 可以建立 $a \rightarrow b$ 权重 2 和 $b \rightarrow a$ 权重 $1/2$ ，查询 x / y 就是从 x 到 y 的路径乘积。这个模型不适合普通最短路，因为路径权重不是相加，而是相乘；但 DFS/BFS 沿路径累计乘积即可。若查询很多，还可以用带权并查集维护每个节点到根的比例。

带权并查集比普通并查集难，因为集合关系不仅是“是否连通”，还要维护相对权值。以方程除

法为例，`weight[x]` 可以表示 `x / parent[x]`。路径压缩时，需要同时更新 `x / root`。合并 `x` 和 `y` 且已知 `x / y = value` 时，若令 `root_x` 挂到 `root_y`，就要计算 `root_x / root_y` 的值。这个推导容易错，因此面试中如果时间紧，先给 BFS 图解法更稳；只有查询量大或题目明确动态合并时，再写带权并查集。

网格图的另一个高频变化是“状态不只包含位置”。最短路径消除障碍物中，状态应为 `(row, col, remaining)`，表示走到某个格子时还剩多少次消除机会。若只用 `visited[row][col]`，会错误地剪掉更优状态：同样到达一个格子，剩余消除次数越多，后续能力越强。正确的访问标记可以是三维布尔数组，也可以记录到达每个格子时见过的最大剩余次数；当新状态剩余次数不超过历史最好时，才可以剪掉。

Listing 15.17 障碍消除最短路：位置加资源才是完整状态

```
int shortest_path_with_obstacle_elimination(
    const std::vector<std::vector<int>>& grid,
    int eliminations)
{
    constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
        std::pair{1, 0}, std::pair{-1, 0},
        std::pair{0, 1}, std::pair{0, -1}
    };
    const int rows = static_cast<int>(grid.size());
    const int cols = static_cast<int>(grid[0].size());

    struct State {
        int row;
        int col;
        int remaining;
        int steps;
    };

    std::vector<std::vector<int>> best_remaining(
        rows, std::vector<int>(cols, -1));
    std::queue<State> queue;
    queue.push(State{0, 0, eliminations, 0});
    best_remaining[0][0] = eliminations;

    while (!queue.empty()) {
        const State state = queue.front();
        queue.pop();
        if (state.row == rows - 1 && state.col == cols - 1) {
            return state.steps;
        }

        for (const auto [dr, dc] : DIRECTIONS) {
            const int next_row = state.row + dr;
            const int next_col = state.col + dc;
            const bool out_of_bounds =
                next_row < 0 || next_row >= rows ||
                next_col < 0 || next_col >= cols;
            if (out_of_bounds) {
                continue;
            }
            const int next_remaining =
                state.remaining - grid[next_row][next_col];
            if (next_remaining < 0 ||
                next_remaining <= best_remaining[next_row][next_col]) {
                continue;
            }
            best_remaining[next_row][next_col] = next_remaining;
            queue.push(State{next_row, next_col, next_remaining, state.steps + 1});
        }
    }

    return -1;
}
```

这个题非常适合训练“状态支配”思想。到达同一格子时，步数由 BFS 层保证是非降的；如果某个新状态剩余消除次数更少或相等，那么它在未来不会比历史状态更好，可以剪掉。若新状态剩余次数更多，即使步数不更短，也可能穿过更多障碍，不能剪。状态支配比简单 `visited` 更细，是

很多高阶 BFS 的核心。

深入理解

从源码和工程角度看，图算法的性能经常被内存访问模式决定。邻接表遍历时，外层 `vector` 的每个桶指向独立存储；节点度数分布很不均匀时，热点节点的邻接边会形成大块连续扫描，而大量低度节点会带来分散访问。前向星或 CSR 风格结构把所有边压进连续数组，遍历时更接近顺序读。Linux 内核大量使用侵入式链表、红黑树、基数树或 XArray 这类结构，是因为内核对象本身已经存在，数据结构常常不拥有对象，只负责把对象链接进某种索引。刷题不需要照搬内核写法，但要理解背后的原则：结构选择服务于访问模式、生命周期和更新频率。

实验建议：选三道题做横向对比。第一道用 Dijkstra 和二分并查集分别解决最小体力路径，记录两种建模的状态、转移和复杂度。第二道用单向 BFS 与双向 BFS 解决打开转盘锁，统计访问状态数量。第三道把课程表从“能否完成”改成“给出一种学习顺序”，再改成“每学期最多学 k 门课时最少需要几学期”，观察拓扑排序、BFS 和状态压缩之间的边界。实验报告不要求长，但必须写清楚为什么一种模型能替代另一种模型，什么时候不能替代。

图论进阶选型

只问是否连通或连通块数量，优先考虑并查集或 BFS/DFS；问无权最短步数，优先 BFS；问非负权最短距离，优先 Dijkstra；问能否在某个阈值下连通，可以考虑二分答案加 BFS/并查集；问所有边反复松弛且可能有负边，才考虑 Bellman-Ford。先选模型，再写代码。

第 16 章

动态规划进阶、容器原理和源码阅读

刷题到中后期，很多错误不再来自不会写循环，而来自状态定义不稳定、容器成本估计错误、以及对标准库行为理解不够。动态规划进阶要把问题拆成“选择、容量、阶段、状态”；C++ 容器原理要知道数据如何存储、什么时候扩容、迭代器何时失效；源码阅读要能从接口行为反推实现约束。面试中如果能同时讲算法和容器成本，答案会明显更可靠。

0-1 背包是 DP 进阶的基础模型。给定若干物品，每个物品只能选一次，有重量和价值，背包容量有限，求最大价值。暴力方法是枚举每个物品选或不选，复杂度 $O(2^n)$ 。状态可以定义为 `dp[i][cap]`：只考虑前 `i` 个物品、容量为 `cap` 时的最大价值。第 `i` 个物品要么不选，要么在容量足够时选择。

Listing 16.1 0-1 背包：一维滚动数组

```
int knapsack_01(const std::vector<int>& weights,
               const std::vector<int>& values,
               int capacity)
{
    std::vector<int> dp(capacity + 1, 0);

    for (int i = 0; i < static_cast<int>(weights.size()); ++i) {
        for (int cap = capacity; cap >= weights[i]; --cap) {
            dp[cap] = std::max(dp[cap], dp[cap - weights[i]] + values[i]);
        }
    }

    return dp[capacity];
}
```

一维滚动数组必须倒序遍历容量，因为每个物品只能使用一次。若正序遍历，`dp[cap - weights[i]]` 可能已经包含当前物品，等价于重复选择。这个遍历方向就是 0-1 背包和完全背包最重要的区别。面试中不要只背公式，要说清楚倒序是为了让当前物品的转移读取上一轮状态。

分割等和子集是 0-1 背包的判定版本。题目问数组能否分成两个和相等的子集。总和若为奇数，直接不可能；否则问题变成能否从数组中选一些数，使和为 `sum / 2`。每个数只能选一次，所以是 0-1 背包。状态 `dp[cap]` 表示是否能凑出容量 `cap`。

Listing 16.2 分割等和子集：0-1 背包可达性

```
bool can_partition(const std::vector<int>& nums)
{
    const int total = std::accumulate(nums.begin(), nums.end(), 0);
    if (total % 2 != 0) {
        return false;
    }

    const int target = total / 2;
    std::vector<char> dp(target + 1, false);
    dp[0] = true;

    for (const int x : nums) {
        for (int cap = target; cap >= x; --cap) {
            dp[cap] = dp[cap] || dp[cap - x];
        }
    }

    return dp[target];
}
```

这题的关键不是代码，而是把“分成两个相等子集”转成“选择若干元素凑出总和一半”。如果数组元素都是正数，背包容量有限，DP 很自然；如果存在负数，容量模型就不直接适用，需要偏移或换思路。复杂度是 $O(n \cdot \text{target})$ ，其中 `target` 与数值大小有关，所以这是伪多项式复杂度，不是

严格输入长度多项式。

目标和可以转成背包计数。给每个数加正号或负号，使表达式等于 `target`。设正数集合和为 P ，负数集合和为 N ，总和 S 。则 $P - N = \text{target}$ ， $P + N = S$ ，推出 $P = (S + \text{target}) / 2$ 。问题变成从数组中选若干数，使和为 P 的方案数。暴力枚举符号是 $O(2^n)$ ；DP 保存每个和的方案数。

Listing 16.3 目标和：转换为 0-1 背包计数

```
int find_target_sum_ways(const std::vector<int>& nums, int target)
{
    const int total = std::accumulate(nums.begin(), nums.end(), 0);
    if (std::abs(target) > total || (total + target) % 2 != 0) {
        return 0;
    }

    const int positive_sum = (total + target) / 2;
    std::vector<int> dp(positive_sum + 1, 0);
    dp[0] = 1;

    for (const int x : nums) {
        for (int sum = positive_sum; sum >= x; --sum) {
            dp[sum] += dp[sum - x];
        }
    }

    return dp[positive_sum];
}
```

这题要特别注意 0。若数组里有 0，给它加正号或负号都不改变和，但方案数应翻倍。上述 DP 能自然处理：当 $x == 0$ 时， $\text{dp}[\text{sum}] += \text{dp}[\text{sum}]$ ，所有已有方案翻倍。很多手写特殊逻辑反而容易错。计数型 DP 要关注整型范围，力扣原题结果通常能放进 `int`，但工程中应考虑 `long long`。

零钱兑换 II 是完全背包计数。硬币可以无限使用，问凑成金额的组合数。注意是组合数，不是排列数。若外层遍历金额、内层遍历硬币，会把不同顺序当作不同方案；若外层遍历硬币、内层正序遍历金额，每种硬币按固定顺序加入，就只统计组合。

Listing 16.4 零钱兑换 II：完全背包组合计数

```
int change(int amount, const std::vector<int>& coins)
{
    std::vector<int> dp(amount + 1, 0);
    dp[0] = 1;

    for (const int coin : coins) {
        for (int value = coin; value <= amount; ++value) {
            dp[value] += dp[value - coin];
        }
    }

    return dp[amount];
}
```

为什么容量正序？因为当前硬币可以重复使用， $\text{dp}[\text{value} - \text{coin}]$ 应该允许已经使用当前硬币。为什么硬币在外层？因为组合不关心顺序，我们规定按硬币种类逐步开放选择。这个题非常适合区分 0-1 背包、完全背包、组合计数和排列计数。只要循环顺序解释不清，背包题就还没有真正掌握。

股票题是状态机 DP。买卖股票的最佳时机 I 只允许一次交易，最简单做法是扫描时维护历史最低买入价和当前最大利润。暴力枚举买入日和卖出日是 $O(n^2)$ ；优化来自“卖出日固定时，最优买入日一定是它之前价格最低的一天”。

Listing 16.5 买卖股票 I：维护历史最低价格

```
int max_profit_one_transaction(const std::vector<int>& prices)
{
    int min_price = std::numeric_limits<int>::max();
    int answer = 0;

    for (const int price : prices) {
```

```

        min_price = std::min(min_price, price);
        answer = std::max(answer, price - min_price);
    }

    return answer;
}

```

含冷冻期的股票题不能只维护最低价，因为今天能不能买取决于前一天是否卖出。状态机更清楚：**hold** 表示今天结束时持有股票的最大收益；**sold** 表示今天刚卖出；**rest** 表示今天结束时不持有且不在刚卖出的状态。转移时，买入只能从 **rest** 来，卖出只能从 **hold** 来，休息可以从昨天的 **rest** 或 **sold** 来。

Listing 16.6 含冷冻期股票：三状态 DP

```

int max_profit_with_cooldown(const std::vector<int>& prices)
{
    if (prices.empty()) {
        return 0;
    }

    int hold = -prices[0];
    int sold = 0;
    int rest = 0;

    for (int i = 1; i < static_cast<int>(prices.size()); ++i) {
        const int previous_hold = hold;
        const int previous_sold = sold;
        const int previous_rest = rest;

        hold = std::max(previous_hold, previous_rest - prices[i]);
        sold = previous_hold + prices[i];
        rest = std::max(previous_rest, previous_sold);
    }

    return std::max(sold, rest);
}

```

这段代码的关键是先保存昨天状态，避免同一天内状态互相污染。返回值不能是 **hold**，因为结束时仍持有股票不代表已经实现收益。股票 DP 的复盘模板是：每天结束时有哪些状态，状态之间允许哪些动作，动作是否受冷冻期、手续费、交易次数限制。把限制放进状态机，公式就会自然出现。

动态规划最容易被学成公式库，但公式不是起点。起点永远是暴力搜索：有哪些选择，搜索树的节点表示什么，哪些节点会重复出现。以最长递增子序列为例，暴力做法是对每个元素选择“选或不选”，同时维护上一个被选元素，复杂度指数级。重复子问题来自“处理到某个位置、上一个选择约束相同”时，后续选择空间完全一样。第一种 DP 定义可以是 **dp[i]**：以 **nums[i]** 结尾的最长递增子序列长度。这样每个状态都问一个局部稳定的问题：最后一个元素固定为 **i**，前一个元素只能从 **0..i-1** 中找，并且值要更小。

Listing 16.7 最长递增子序列：二次 DP 和路径还原

```

std::vector<int> reconstruct_lis(const std::vector<int>& nums)
{
    const int n = static_cast<int>(nums.size());
    if (n == 0) {
        return {};
    }

    std::vector<int> dp(n, 1);
    std::vector<int> parent(n, -1);
    int best_index = 0;

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < i; ++j) {
            if (nums[j] >= nums[i] || dp[j] + 1 <= dp[i]) {
                continue;
            }
            dp[i] = dp[j] + 1;
        }
    }
}

```

```

        parent[i] = j;
    }
    if (dp[i] > dp[best_index]) {
        best_index = i;
    }
}

std::vector<int> sequence;
for (int index = best_index; index != -1; index = parent[index]) {
    sequence.push_back(nums[index]);
}
std::reverse(sequence.begin(), sequence.end());
return sequence;
}

```

这段代码不是最优复杂度，但它最适合讲清楚状态。`parent[i]` 记录让 `dp[i]` 变大的前驱下标，所以最后可以从最优结尾倒着还原路径。正确性来自一个简单不变量：外层处理到 `i` 时，所有 `j < i` 的 `dp[j]` 已经是 `j` 结尾的最优答案；若 `nums[j] < nums[i]`，把 `i` 接到以 `j` 结尾的最优序列后面，一定得到一个合法递增序列。枚举所有可能前驱后取最大值，就不会漏掉以 `i` 结尾的最优解。复杂度 $O(n^2)$ ，空间 $O(n)$ 。

最长递增子序列还可以优化到 $O(n \log n)$ 。优化依据不是简单二分，而是维护一个更抽象的数组 `tails`：长度为 `len + 1` 的递增子序列，结尾最小值是多少。结尾越小，未来接上新元素的机会越大。扫描每个数 `x`，用二分找到第一个大于等于 `x` 的位置并替换它；如果找不到，就把 `x` 追加到末尾。`tails` 不一定是一条真实子序列，但它的长度等于 LIS 长度。这个区别必须讲清楚，否则读者会误以为 `tails` 本身就是答案路径。

Listing 16.8 最长递增子序列：tails 数组和二分优化

```

int length_of_lis(const std::vector<int>& nums)
{
    std::vector<int> tails;
    tails.reserve(nums.size());

    for (const int x : nums) {
        auto it = std::lower_bound(tails.begin(), tails.end(), x);
        if (it == tails.end()) {
            tails.push_back(x);
        } else {
            *it = x;
        }
    }

    return static_cast<int>(tails.size());
}

```

为什么替换不会破坏答案？因为替换位置 `pos` 的含义是：存在长度为 `pos + 1` 的递增子序列以 `x` 结尾，并且 `x` 不大于原来的结尾。更小的结尾不会让未来变差，只会让未来更容易扩展。若题目要求严格递增，使用 `lower_bound`；若要求非递减，使用 `upper_bound`。这是面试里常见追问：二分函数的选择取决于相等元素是否允许接在后面。

网格路径 DP 是二维状态的入门。不同路径问从左上到右下只能向右或向下走有多少条路径。暴力递归会从每个格子继续向右和向下，重复计算大量子矩形。状态 `dp[row][col]` 表示到达当前格子的路径数，它只依赖上方和左方。由于每个状态只依赖上一行和当前行左侧，可以压缩成一维数组。这里的遍历顺序不是随便的：从左到右更新时，`dp[col]` 仍表示上一行同列，`dp[col - 1]` 已经表示当前行左侧。

Listing 16.9 不同路径：一维滚动数组

```

int unique_paths(int rows, int cols)
{
    std::vector<int> dp(cols, 1);
    for (int row = 1; row < rows; ++row) {
        for (int col = 1; col < cols; ++col) {
            dp[col] += dp[col - 1];
        }
    }
}

```

```
    return dp[cols - 1];
}
```

最小路径和与不同路径结构相同，只是“方案数相加”变成“代价取最小”。状态 `dp[col]` 表示处理到当前行时，到达该列的最小路径和。第一行只能从左边来，第一列只能从上边来，其余位置取上方和左方较小者。边界处理是这类题的主要错误来源。一个稳妥写法是先初始化第一行，再逐行处理第一列和内部格子；不要在双循环里塞很多特殊判断，让主转移被边界噪声淹没。

Listing 16.10 最小路径和：边界初始化后主转移

```
int min_path_sum(const std::vector<std::vector<int>>& grid)
{
    const int rows = static_cast<int>(grid.size());
    const int cols = static_cast<int>(grid[0].size());
    std::vector<int> dp(cols, 0);

    dp[0] = grid[0][0];
    for (int col = 1; col < cols; ++col) {
        dp[col] = dp[col - 1] + grid[0][col];
    }

    for (int row = 1; row < rows; ++row) {
        dp[0] += grid[row][0];
        for (int col = 1; col < cols; ++col) {
            dp[col] = std::min(dp[col], dp[col - 1]) + grid[row][col];
        }
    }

    return dp[cols - 1];
}
```

编辑距离是字符串 DP 的核心题。给定两个单词，允许插入、删除、替换，求把一个单词变成另一个单词的最少操作数。暴力递归从两个字符串末尾比较：若字符相同，两个指针都左移；若不同，可以删除一个字符、插入一个字符、替换一个字符。暴力慢是因为同一对前缀长度会被多条路径反复访问。状态 `dp[i][j]` 表示 `word1` 的前 `i` 个字符变成 `word2` 的前 `j` 个字符的最少操作数。空串边界很重要：`dp[i][0] = i`, `dp[0][j] = j`。

Listing 16.11 编辑距离：两行滚动数组

```
int min_distance(const std::string& word1, const std::string& word2)
{
    const int n = static_cast<int>(word1.size());
    const int m = static_cast<int>(word2.size());

    std::vector<int> previous(m + 1, 0);
    std::vector<int> current(m + 1, 0);
    for (int j = 0; j <= m; ++j) {
        previous[j] = j;
    }

    for (int i = 1; i <= n; ++i) {
        current[0] = i;
        for (int j = 1; j <= m; ++j) {
            if (word1[i - 1] == word2[j - 1]) {
                current[j] = previous[j - 1];
                continue;
            }
            const int remove_cost = previous[j] + 1;
            const int insert_cost = current[j - 1] + 1;
            const int replace_cost = previous[j - 1] + 1;
            current[j] = std::min({remove_cost, insert_cost, replace_cost});
        }
        std::swap(previous, current);
    }

    return previous[m];
}
```

三个操作的含义要能对上表格位置。删除 `word1[i-1]` 后，问题变成 `word1` 前 `i-1` 个字符到

word2 前 j 个字符，所以来自 `previous[j]`；插入 `word2[j-1]` 后，等价于先把 `word1` 前 i 个字符变成 `word2` 前 $j-1$ 个字符，所以来自 `current[j-1]`；替换则来自 `previous[j-1]`。两行滚动数组的空间是 $O(m)$ ，时间是 $O(nm)$ 。如果题目要求还原操作序列，就不能只保留两行，需要记录决策或重新回溯。

最长公共子序列和编辑距离很接近。状态 `dp[i][j]` 表示两个前缀的 LCS 长度。若末尾字符相同，就从 `dp[i-1][j-1] + 1` 转移；否则取删掉任一末尾后的最大值。它和最长公共子串不同：子序列允许不连续，子串必须连续。这个概念差异决定了转移差异。最长公共子串在字符不同时必须归零，LCS 在字符不同时取上方或左方最大值。面试里一旦题目出现“可以删除若干字符但保持相对顺序”，就要想到子序列。

Listing 16.12 最长公共子序列：一维数组和左上角缓存

```
int longest_common_subsequence(const std::string& text1,
                               const std::string& text2)
{
    const int n = static_cast<int>(text1.size());
    const int m = static_cast<int>(text2.size());
    std::vector<int> dp(m + 1, 0);

    for (int i = 1; i <= n; ++i) {
        int diagonal = 0;
        for (int j = 1; j <= m; ++j) {
            const int old_up = dp[j];
            if (text1[i - 1] == text2[j - 1]) {
                dp[j] = diagonal + 1;
            } else {
                dp[j] = std::max(dp[j], dp[j - 1]);
            }
            diagonal = old_up;
        }
    }

    return dp[m];
}
```

一维 LCS 的难点在于保存左上角。更新前的 `dp[j]` 是上一行同列，`dp[j-1]` 是当前行左侧，`diagonal` 是上一行左上角。每次循环结束前把旧的 `dp[j]` 放进 `diagonal`，供下一列使用。若你觉得这个写法难以保证正确，可以先写二维表；面试时清楚解释二维转移比写一个容易错的一维优化更重要。空间优化应该建立在状态依赖已经完全理解之后。

单词拆分是“前缀可达性”DP。题目问字符串能否被字典中的单词拼接出来。暴力方法枚举第一段单词，然后递归处理剩余后缀；同一个起点会被反复递归。状态 `dp[i]` 表示前 i 个字符能否被拆分。转移枚举最后一个单词的起点 j ：若 `dp[j]` 为真，且 `s[j..i)` 在字典中，则 `dp[i]` 为真。这里要注意 `substr` 的成本，它会创建新字符串。数据量不大时可接受；数据量大时，应限制最大单词长度，或用字典树、字符串视图、滚动哈希减少复制。

Listing 16.13 单词拆分：限制最大单词长度降低无效枚举

```
bool word_break(const std::string& s,
                const std::vector<std::string>& word_dict)
{
    std::unordered_set<std::string> words(word_dict.begin(), word_dict.end());
    int max_word_length = 0;
    for (const std::string& word : word_dict) {
        max_word_length = std::max(max_word_length,
                                    static_cast<int>(word.size()));
    }

    const int n = static_cast<int>(s.size());
    std::vector<char> dp(n + 1, false);
    dp[0] = true;

    for (int end = 1; end <= n; ++end) {
        const int begin_limit = std::max(0, end - max_word_length);
        for (int begin = end - 1; begin >= begin_limit; --begin) {
            if (!dp[begin]) {
                continue;
            }
            if (words.find(s.substr(begin, end - begin)) != words.end()) {
                dp[end] = true;
            }
        }
    }

    return dp[n];
}
```

```

    }
    if (words.contains(s.substr(begin, end - begin))) {
        dp[end] = true;
        break;
    }
}
}
return dp[n];
}

```

这个优化不是改变问题模型，而是减少不可能成为单词的长度。若字典中最长单词长度为 L ，枚举起点时只需要看最近 L 个位置。复杂度粗略为 $O(nL)$ 次查询，但每次 `substr` 仍有拷贝成本。面试表达可以分层：先给清楚的 DP，再说明字符串切片成本，最后提出可选优化。不要一上来写复杂字典树，把主线埋掉。

区间 DP 的状态通常是“处理一个连续区间的最优值”。它与普通线性 DP 的区别是：最后一步可能把区间切成两半，或者选择区间内某个点作为最后处理对象。戳气球是代表题。直接思考“先戳哪个气球”很混乱，因为戳掉一个气球会改变邻居；换成“最后戳哪个气球”就稳定了。设 $dp[left][right]$ 表示开区间 $(left, right)$ 内的气球全部戳掉能获得的最大硬币数。若最后戳 mid ，此时它左右邻居一定是 $left$ 和 $right$ ，收益为 $nums[left] * nums[mid] * nums[right]$ ，左右两边互不影响。

Listing 16.14 戳气球：选择最后一个被戳的气球

```

int max_coins(const std::vector<int>& nums)
{
    std::vector<int> values;
    values.reserve(nums.size() + 2);
    values.push_back(1);
    values.insert(values.end(), nums.begin(), nums.end());
    values.push_back(1);

    const int n = static_cast<int>(values.size());
    std::vector<std::vector<int>> dp(n, std::vector<int>(n, 0));

    for (int length = 2; length < n; ++length) {
        for (int left = 0; left + length < n; ++left) {
            const int right = left + length;
            for (int mid = left + 1; mid < right; ++mid) {
                const int score = dp[left][mid] + dp[mid][right] +
                    values[left] * values[mid] * values[right];
                dp[left][right] = std::max(dp[left][right], score);
            }
        }
    }

    return dp[0][n - 1];
}

```

区间 DP 的遍历顺序按区间长度从小到大，因为大区间依赖小区间。这里的边界 1 是虚拟气球，不会被戳，只是让左右边界收益统一。正确性要抓住“最后一步”：如果 mid 是最后被戳的气球，那么 $left..mid$ 和 $mid..right$ 两个子问题已经在它之前完成，且互相独立；枚举所有可能的最后气球，取最大值。复杂度是 $O(n^3)$ ，空间 $O(n^2)$ 。若面试官追问能否优化，要先说明一般区间 DP 不一定有四边形不等式或决策单调性，不能乱套优化。

状态压缩 DP 把集合放进整数二进制位。适用条件通常是元素数量不大，例如 $n \leq 20$ 。每一位表示某个任务、节点或物品是否已经被选择。经典问题是旅行商或访问所有节点的最短路径。若图无权，访问所有节点的最短路径可以用 BFS，把状态定义为 $(node, mask)$ ：当前在 $node$ ，已经访问的节点集合是 $mask$ 。这其实是状态压缩和图 BFS 的结合，而不是传统表格 DP。

Listing 16.15 访问所有节点的最短路径：节点加集合状态 BFS

```

int shortest_path_length(const std::vector<std::vector<int>>& graph)
{
    const int n = static_cast<int>(graph.size());
    const int full_mask = (1 << n) - 1;
}

```

```

std::queue<std::pair<int, int>> queue;
std::vector<std::vector<int>> distance(n, std::vector<int>(1 << n, -1));

for (int node = 0; node < n; ++node) {
    const int mask = 1 << node;
    distance[node][mask] = 0;
    queue.emplace(node, mask);
}

while (!queue.empty()) {
    const auto [node, mask] = queue.front();
    queue.pop();
    if (mask == full_mask) {
        return distance[node][mask];
    }

    for (const int next : graph[node]) {
        const int next_mask = mask | (1 << next);
        if (distance[next][next_mask] != -1) {
            continue;
        }
        distance[next][next_mask] = distance[node][mask] + 1;
        queue.emplace(next, next_mask);
    }
}

return 0;
}

```

这题如果只把节点作为访问标记，会出错。到达同一个节点时，已访问集合不同，未来能力不同。**(node, mask)** 才是完整状态。多源初始化也很关键：路径可以从任意节点开始，所以所有单节点状态都在第 0 层。复杂度是 $O(n2^n + m2^n)$ ，只适合小 n 。状态压缩题要先估算 2^n 是否可承受，再决定是否使用；看到集合并不意味着一定能压缩，如果 n 是一百，二进制集合就不现实。

另一类状态压缩是子集枚举。比如把数组分成若干组、每组和相等。可以用 `dp[mask]` 表示选择集合 `mask` 后，当前桶中已经放入的对目标桶容量取模是多少；若 `dp[mask] == -1` 表示不可达。每次尝试加入一个未选元素，只要当前桶不超容量，就更新新集合。这个写法比回溯更像 DP，因为每个集合状态只计算一次。

Listing 16.16 划分为 K 个等和子集：集合状态 DP

```

bool can_partition_k_subsets(std::vector<int> nums, int k)
{
    const int total = std::accumulate(nums.begin(), nums.end(), 0);
    if (total % k != 0) {
        return false;
    }
    const int target = total / k;
    std::sort(nums.begin(), nums.end(), std::greater<int>());
    if (!nums.empty() && nums.front() > target) {
        return false;
    }

    const int n = static_cast<int>(nums.size());
    const int state_count = 1 << n;
    std::vector<int> dp(state_count, -1);
    dp[0] = 0;

    for (int mask = 0; mask < state_count; ++mask) {
        if (dp[mask] == -1) {
            continue;
        }
        for (int i = 0; i < n; ++i) {
            if ((mask & (1 << i)) != 0) {
                continue;
            }
            const int next_sum = dp[mask] + nums[i];
            if (next_sum > target) {
                continue;
            }

```

```

        const int next_mask = mask | (1 << i);
        dp[next_mask] = next_sum % target;
    }
}

return dp[state_count - 1] == 0;
}

```

这个 DP 的状态值不是总和，而是当前桶的剩余进度。每当一个桶刚好装满，对 `target` 取模后的进度回到 0，表示开始装下一个桶。排序不是正确性必需，但大的数先处理能更早剪掉不可能状态。复杂度是 $O(n2^n)$ 。与回溯相比，状态 DP 的优势是避免不同选择顺序到达同一集合时重复搜索；缺点是内存随 2^n 增长。

单调队列优化 DP 是从“转移枚举范围”中挤出复杂度。以带限制的最大子序列和为例，题目要求选择一个非空子序列，相邻被选元素下标差最多为 k ，求最大和。朴素 DP 定义 `dp[i]` 为必须选择 `nums[i]` 且以它结尾的最大和，则

$$dp[i] = nums[i] + \max(0, dp[j]) \quad i - k \leq j < i.$$

如果每个 i 都扫描前 k 个位置，复杂度 $O(nk)$ 。优化来自窗口最大值：我们只需要最近 k 个 `dp[j]` 的最大值，用单调队列维护即可。

Listing 16.17 受限子序列和：单调队列优化转移最大值

```

int constrained_subset_sum(const std::vector<int>& nums, int k)
{
    std::deque<int> deque;
    std::vector<int> dp(nums.size(), 0);
    int answer = std::numeric_limits<int>::min();

    for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
        while (!deque.empty() && deque.front() < i - k) {
            deque.pop_front();
        }

        dp[i] = nums[i];
        if (!deque.empty()) {
            dp[i] = std::max(dp[i], nums[i] + dp[deque.front()]);
        }
        answer = std::max(answer, dp[i]);

        while (!deque.empty() && dp[deque.back()] <= dp[i]) {
            deque.pop_back();
        }
        if (dp[i] > 0) {
            deque.push_back(i);
        }
    }

    return answer;
}

```

队列中保存下标，而不是保存值，因为要判断是否过期。队首永远是当前窗口内 `dp` 最大的位置。只把正的 `dp[i]` 放入队列，是因为负贡献不如从当前元素重新开始。正确性来自两个不变量：队列下标从前到后递增，保证能判断窗口；对应 `dp` 值从前到后递减，保证队首最大。每个下标最多进队一次、出队一次，所以总复杂度 $O(n)$ 。

动态规划和最短路之间有很深的联系。许多 DP 可以看成在有向无环图上找路径：状态是节点，转移是边，遍历顺序是拓扑序。若状态图有环但边权非负，可能变成 Dijkstra；若边权全为 1，可能变成 BFS；若存在负边但没有负环，可能变成 Bellman-Ford。这样理解后，题型边界会更清楚：不是所有“求最优”都叫 DP，也不是所有“状态转移”都要填表。你要先问状态图是否有自然阶段顺序，是否允许回头，转移代价是否单调。

常见误区

DP 常见误区有四个。第一，只背公式，不说明状态含义，导致换一个题面就失效。第二，滚动数组时忘记转移读的是上一阶段还是当前阶段。第三，把组合数和排列数混在一起，循环顺序写反。第四，看到最值就写 DP，却没有检查状态数量是否可承受。每次写完 DP，都要用一句话解释 `dp` 的含义，用一句话解释遍历顺序，用一句话解释为什么不会漏解或重复计数。

C++ 容器原理是刷题代码质量的底座。`std::vector` 是连续数组，支持随机访问，扩容时会搬迁元素，导致指针、引用和迭代器失效。`std::deque` 分段连续，支持两端高效插入删除，但随机访问局部性通常不如 `vector`。`std::list` 节点稳定，但缓存局部性差，刷题中很少是首选。`std::unordered_map` 平均常数查询，但 rehash 会让迭代器失效，且不维护顺序。`std::map` 有序但每次操作是 $O(\log n)$ 。

阅读标准库源码时，不需要一开始钻进所有模板细节。先看三层：公开接口、核心数据成员、关键操作路径。以 `vector` 为例，大多数实现内部会维护三个指针或等价状态：开始位置、当前结束位置、容量结束位置。`size()` 是当前结束减开始，`capacity()` 是容量结束减开始，`push_back` 在容量足够时原地构造，在容量不足时申请更大内存并移动旧元素。理解这三根指针，就能解释很多迭代器失效规则。

Listing 16.18 `vector` 使用习惯：reserve、resize 和引用失效

```
void collect_values(const std::vector<int>& input)
{
    std::vector<int> values;
    values.reserve(input.size());

    for (const int x : input) {
        values.push_back(x * 2);
    }

    const int* data = values.data();
    values.push_back(42);
    // 如果 push_back 触发扩容，data 已经失效，不能继续解引用。
}
```

`reserve` 和 `resize` 的区别必须非常熟。`reserve` 只保证容量，不改变大小；`resize` 改变大小并构造元素。DP 数组需要可下标访问，应使用 `vector<int> dp(n + 1)` 或 `resize`；收集答案时知道最多数量，可以先 `reserve` 再 `push_back`。把二者混用，是 C++ 刷题里非常常见的越界来源。

`unordered_map` 的源码阅读重点是桶数组、哈希函数、负载因子和 rehash。插入元素时，容器通过哈希值定位桶，再在桶内查找等价 key。平均常数复杂度依赖哈希分布；当元素数量相对桶数过多，会触发 rehash，重新分配桶并把元素放到新桶中。刷题时使用 `reserve` 可以减少 rehash 次数，尤其是两数之和、前缀和计数、频次统计这类一次性插入较多 key 的题。

Listing 16.19 `unordered_map` 查询：contains 与 find 的取舍

```
int count_frequency(const std::vector<int>& nums, int target)
{
    std::unordered_map<int, int> frequency;
    frequency.reserve(nums.size());

    for (const int x : nums) {
        ++frequency[x];
    }

    const auto found = frequency.find(target);
    if (found == frequency.end()) {
        return 0;
    }
    return found->second;
}
```

如果只判断存在，C++20 的 `contains` 很清楚；如果判断后还要取值，用 `find` 避免查两次。不要用 `operator[]` 做纯查询，因为 key 不存在时会插入默认值，改变容器状态。这个细节在前缀和

计数里尤其重要：有些查询应该只读历史状态，不能悄悄插入一个 0。

源码阅读也能帮助理解算法复杂度的真实成本。字符串 `substr` 会创建新字符串，放在双层循环里可能把 $O(n^2)$ 变成接近 $O(n^3)$ 的实际成本；`vector<vector<int>>` 的每一行是独立分配，矩阵遍历仍然可用，但内存不一定完全连续；`priority_queue` 没有删除任意元素和 `decrease-key`，所以 Dijkstra 常用“压入新状态，弹出时跳过过期状态”的写法。理解容器能力边界，才能解释为什么代码这样写。

`std::deque` 的名字容易误导初学者。它不是双向链表，而是分段连续数组。典型实现会维护一个“map”，其中每个槽指向一段固定大小的缓冲区；首尾插入删除大多只移动段内指针，必要时分配新段。它支持随机访问，但随机访问需要先定位段再定位段内偏移，局部性通常不如 `vector`。这解释了为什么滑动窗口最大值适合用 `deque` 存下标，而普通顺序 DP 数组不应该为了“可能两端操作”随手换成 `deque`。

Listing 16.20 deque 在单调队列中的正确用法

```
std::vector<int> max_sliding_window(const std::vector<int>& nums, int k)
{
    std::deque<int> deque;
    std::vector<int> answer;
    answer.reserve(nums.size());

    for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
        while (!deque.empty() && deque.front() <= i - k) {
            deque.pop_front();
        }
        while (!deque.empty() && nums[deque.back()] <= nums[i]) {
            deque.pop_back();
        }
        deque.push_back(i);
        if (i + 1 >= k) {
            answer.push_back(nums[deque.front()]);
        }
    }

    return answer;
}
```

这里 `deque` 保存下标而不是值，原因和单调队列 DP 一样：需要判断过期位置。下标还能处理重复值，若只存值，窗口移出时很难知道队首值是否对应离开的元素。源码层面看，`pop_front` 和 `push_back` 都是 `deque` 的强项；若用 `vector` 在头部删除，就会移动大量元素；若用 `list`，虽然删除快，但无法高效访问队尾并且缓存局部性差。容器选择和算法不变量应该匹配。

`std::string` 在现代实现中通常具有小字符串优化。短字符串可以直接存放在对象内部，不必单独堆分配；长字符串才指向堆内存。这个事实能解释为什么偶尔创建短字符串不一定灾难，但不能因此在热循环里无限 `substr`。标准并不要求具体的小字符串容量，不同标准库实现也不同，因此工程代码不能依赖“小于多少字符一定不分配”。源码阅读要区分“标准保证的语义”和“某个实现的优化”。

Listing 16.21 字符串扫描：用下标边界避免大量 `substr`

```
bool has_word_at(const std::string& text,
                int begin,
                const std::string& word)
{
    if (begin + static_cast<int>(word.size()) >
        static_cast<int>(text.size())) {
        return false;
    }
    for (int i = 0; i < static_cast<int>(word.size()); ++i) {
        if (text[begin + i] != word[i]) {
            return false;
        }
    }
    return true;
}
```

这段代码展示的是思想：当你只需要比较一段文本时，不一定要构造新字符串。C++20 可以使

用 `std::string_view` 表示非拥有切片，但要注意生命周期；视图不能比原字符串活得更久。刷题中 `substr` 更短、更不易错，数据量允许时可以使用；工程中如果它处在双层或三层循环内，就应该估算分配和拷贝成本。

`std::priority_queue` 是容器适配器，默认底层容器通常是 `vector`，并用堆算法维护最大堆。它只暴露 `top`、`push`、`pop`，不支持遍历、不支持删除任意元素、不支持 `decrease-key`。这些限制不是缺陷，而是接口选择：标准库给的是简单、稳定、低成本的堆抽象。Dijkstra 中距离变小后重新入堆，是顺应这个接口；双堆中位数中延迟删除，也是因为堆不能直接删中间元素。

Listing 16.22 双堆延迟删除：滑动窗口中位数的核心结构

```
class DualHeap {
public:
    explicit DualHeap(int window_size) : window_size_(window_size) {}

    void insert(int value)
    {
        if (this->small_.empty() || value <= this->small_.top()) {
            this->small_.push(value);
            ++this->small_size_;
        } else {
            this->large_.push(value);
            ++this->large_size_;
        }
        this->rebalance();
    }

    void erase(int value)
    {
        ++this->delayed_[value];
        if (value <= this->small_.top()) {
            --this->small_size_;
            if (value == this->small_.top()) {
                this->prune_small();
            }
        } else {
            --this->large_size_;
            if (!this->large_.empty() && value == this->large_.top()) {
                this->prune_large();
            }
        }
        this->rebalance();
    }

    double median() const
    {
        if (this->window_size_ % 2 == 1) {
            return this->small_.top();
        }
        return (static_cast<double>(this->small_.top()) +
                static_cast<double>(this->large_.top())) / 2.0;
    }

private:
    void rebalance()
    {
        if (this->small_size_ > this->large_size_ + 1) {
            this->large_.push(this->small_.top());
            this->small_.pop();
            --this->small_size_;
            ++this->large_size_;
            this->prune_small();
        } else if (this->small_size_ < this->large_size_) {
            this->small_.push(this->large_.top());
            this->large_.pop();
            ++this->small_size_;
            --this->large_size_;
            this->prune_large();
        }
    }

    void prune_small()
    {
        while (this->small_.top() < this->large_.top() && this->delayed_[this->small_.top()] > 0) {
            this->small_.pop();
            --this->small_size_;
            this->delayed_[this->small_.top()]--;
        }
    }

    void prune_large()
    {
        while (this->small_.top() < this->large_.top() && this->delayed_[this->large_.top()] > 0) {
            this->large_.pop();
            --this->large_size_;
            this->delayed_[this->large_.top()]--;
        }
    }
};
```

```

{
    while (!this->small_.empty()) {
        const int value = this->small_.top();
        auto found = this->delayed_.find(value);
        if (found == this->delayed_.end()) {
            return;
        }
        if (--found->second == 0) {
            this->delayed_.erase(found);
        }
        this->small_.pop();
    }
}

void prune_large()
{
    while (!this->large_.empty()) {
        const int value = this->large_.top();
        auto found = this->delayed_.find(value);
        if (found == this->delayed_.end()) {
            return;
        }
        if (--found->second == 0) {
            this->delayed_.erase(found);
        }
        this->large_.pop();
    }
}

int window_size_;
std::priority_queue<int> small_;
std::priority_queue<int, std::vector<int>, std::greater<int>> large_;
std::unordered_map<int, int> delayed_;
int small_size_ = 0;
int large_size_ = 0;
};

```

这段结构比普通数据流中位数复杂，因为窗口滑动时需要删除离开的元素。既然堆不能删除任意位置，就把待删除值记在 `delayed_` 中；只有当它出现在堆顶时才真正弹出。`small_size_` 和 `large_size_` 记录的是有效元素数量，不是堆内部实际元素数量。成员访问使用 `this->` 是为了明确哪些变量属于对象状态。复杂度均摊 $O(\log k)$ 。如果用 `multiset`，删除任意元素更直接，但常数和迭代器维护方式不同；两种方案都要能解释取舍。

`std::map` 和 `std::set` 通常由红黑树实现，保证有序遍历和 $O(\log n)$ 插入、删除、查找。红黑树是一种近似平衡二叉搜索树，插入删除后通过旋转和染色维持高度对数级。面试刷题里，使用 `map` 的信号通常是是需要有序键、前驱后继、区间边界或按顺序输出。若只需要存在性或计数，`unordered_map` 平均更快；若哈希可能被攻击、需要稳定顺序或需要 `lower_bound`，就选有序树。

Listing 16.23 有序映射：用 `upper_bound` 找右侧区间

```

int count_covered_points(const std::map<int, int>& intervals, int point)
{
    const auto right = intervals.upper_bound(point);
    if (right == intervals.begin()) {
        return 0;
    }
    const auto candidate = std::prev(right);
    return candidate->second >= point ? 1 : 0;
}

```

`upper_bound(point)` 返回第一个起点大于 `point` 的区间，前一个区间才可能覆盖 `point`。这类写法是有序容器的典型优势。注意 `std::prev(right)` 之前必须判断 `right != begin()`，否则会越界。源码阅读中看到树迭代器时，要记住它不是连续内存指针，自增自减会沿树结构寻找后继前驱，复杂度通常是均摊常数或对数级细节由实现保证，但缓存局部性不如数组。

`unordered_map` 的平均常数复杂度不是无条件的。它依赖哈希函数把键分散到桶里，也依赖负载因子保持在合理范围。若大量键落入同一桶，查找会退化。C++ 标准没有要求具体冲突处理方式必须是链表还是开放寻址，常见实现多使用桶加节点。节点式实现的好处是 rehash 时可以重挂节点，

坏处是每个元素单独分配，局部性不如连续表。刷题时使用 `reserve`，不只是为了速度，也是为了减少 rehash 导致的迭代器失效风险。

Listing 16.24 前缀和计数：查询和更新顺序不能反

```
int subarray_sum(const std::vector<int>& nums, int target)
{
    std::unordered_map<int, int> count;
    count.reserve(nums.size() * 2);
    count[0] = 1;

    int prefix = 0;
    int answer = 0;
    for (const int x : nums) {
        prefix += x;
        const auto found = count.find(prefix - target);
        if (found != count.end()) {
            answer += found->second;
        }
        ++count[prefix];
    }
    return answer;
}
```

这里必须先查询历史前缀，再加入当前前缀。若先更新，`target == 0` 时会把空子数组错误计入。这个例子说明容器 API 和算法不变量是绑在一起的：`count` 的语义是“当前下标之前出现过的 前缀和次数”，不是所有前缀和次数。`find` 用于查询，`operator[]` 用于确认要插入或累加的位置；二者角色不同。

源码阅读可以按固定路线进行。第一步读标准语义：这个容器承诺什么复杂度，哪些操作会让迭代器失效。第二步看实现的核心成员：`vector` 的三个指针，`string` 的大小容量和小字符串缓冲，`deque` 的段表，`unordered_map` 的桶数组和节点，`map` 的树根和节点颜色。第三步跟一条热路径：`push_back` 如何判断容量，`rehash` 如何搬迁，`erase` 如何维护链接，`lower_bound` 如何沿树下降。第四步回到题目，解释为什么某个 API 成本可接受或不可接受。

深入理解

阅读 `libstdc++` 或 `libc++` 源码时，不要被模板细节吓住。可以先从头文件里的 `public` 函数名定位到内部辅助函数，再沿着分配、构造、移动、销毁四类操作看。比如 `vector::push_back` 最核心的问题只有两个：容量够不够，元素如何构造；容量不够时，才进入重新分配和移动旧元素的路径。再比如 `unordered_map::operator[]` 的关键语义是“没有 key 就插入默认值”，源码里会表现为查找失败后构造节点。把这些行为和题目中的不变量联系起来，源码阅读才会变成能力，而不是机械翻文件。

实验建议：第一，自己写一个小程序，创建 `vector<int>`，连续 `push_back` 并打印 `size`、`capacity` 和 `data()` 地址，观察扩容时机和地址变化。第二，写前缀和计数题，把“先查询后更新”故意改反，用 `target = 0` 的用例验证错误。第三，比较 `substr` 和下标比较在长字符串 DP 中的运行时间。第四，阅读本机标准库中 `vector`、`deque`、`unordered_map` 的头文件，只记录核心数据成员和迭代器失效规则，不要求理解每个模板分支。

DP 和容器源码阅读复盘

DP 先问状态是否稳定，转移是否只依赖已计算状态，遍历顺序是否正确；容器先问数据是否连续、插入删除位置在哪里、查询是否需要顺序、迭代器何时失效。源码阅读不追求背实现细节，而是用实现模型解释复杂度、失效规则和 API 选择。

力扣刷题手册

刷题不要按编号顺序推进，要按题型推进。每道题都记录题型、输入规模、暴力解、优化来源、不变量、使用的数据结构、复杂度、错因和一周后是否能独立重写。

数组和字符串主线：1 两数之和、26 删除有序数组重复项、27 移除元素、88 合并两个有序数组、53 最大子数组和、283 移动零、344 反转字符串。

双指针和窗口主线：3 无重复字符的最长子串、11 盛最多水的容器、15 三数之和、42 接雨水、76 最小覆盖子串、209 长度最小的子数组、438 找到字符串中所有字母异位词。

哈希和前缀和主线：49 字母异位词分组、128 最长连续序列、202 快乐数、560 和为 K 的子数组、523 连续的子数组和、525 连续数组。

栈和堆主线：20 有效括号、155 最小栈、739 每日温度、84 柱状图中最大的矩形、215 数组中的第 K 个最大元素、347 前 K 个高频元素、295 数据流的中位数。

二分主线：35 搜索插入位置、34 在排序数组中查找元素的第一个和最后一个位置、33 搜索旋转排序数组、162 寻找峰值、875 爱吃香蕉的珂珂、410 分割数组的最大值。

树和图主线：102 二叉树层序遍历、104 最大深度、98 验证二叉搜索树、236 最近公共祖先、200 岛屿数量、994 腐烂的橘子、207 课程表、210 课程表 II、127 单词接龙。

回溯主线：46 全排列、47 全排列 II、78 子集、39 组合总和、22 括号生成、79 单词搜索、51 N 皇后。

动态规划主线：70 爬楼梯、198 打家劫舍、322 零钱兑换、300 最长递增子序列、139 单词拆分、1143 最长公共子序列、72 编辑距离。

面试专项主线：19 删除链表倒数第 N 个节点、21 合并两个有序链表、25 K 个一组翻转链表、84 柱状图中最大的矩形、239 滑动窗口最大值、56 合并区间、75 颜色分类、215 数组中的第 K 个最大元素、347 前 K 个高频元素、295 数据流的中位数、55 跳跃游戏、45 跳跃游戏 II、547 省份数量、684 冗余连接、743 网络延迟时间、1631 最小体力消耗路径、416 分割等和子集、494 目标和、518 零钱兑换 II、121 买卖股票的最佳时机、309 最佳买卖股票时机含冷冻期。

下面的题目卡片不是为了替代正文，而是为了复习时快速恢复推理链。每一题都要先自己说出暴力解，再说优化依据；如果只能背出最终代码，就把它当作没有掌握。

1 两数之和。题目要求在数组中找到两个数，使它们的和等于目标值。暴力方法是双层循环枚举两个下标，复杂度 $O(n^2)$ 。优化来自“当前数需要的另一个数是确定的”：扫描数组时，用哈希表保存已经见过的值到下标的映射，处理 x 时查询 $target - x$ 是否出现过。必须先查再插入当前数，避免同一个元素被使用两次。复杂度 $O(n)$ ，空间 $O(n)$ 。若数组已排序，可以用双指针，但原题通常要求返回原始下标，哈希更直接。

26 删除有序数组重复项。有序是核心条件。暴力可以新建数组保存不重复元素，再复制回原数组；原地优化用慢指针表示下一个唯一元素应该写入的位置，快指针扫描所有元素。因为重复元素相邻，只要当前值和前一个不同，就写到慢指针位置。正确性不变量是： $[0, slow)$ 始终保存已经处理前缀中的唯一元素且顺序不变。复杂度 $O(n)$ ，额外空间 $O(1)$ 。易错点是空数组和返回的是新长度，不是数组本身。

27 移除元素。题目要求原地移除所有等于给定值的元素，元素相对顺序通常不重要或不强制。保持顺序的做法是快慢指针：快指针扫描，遇到不等于目标的值就写到慢指针位置。若不要求顺序，还可以用左右指针，把等于目标的左侧元素和右端未处理元素交换，从而减少写入次数。面试中要先确认题目是否要求稳定顺序。复杂度 $O(n)$ ，空间 $O(1)$ 。

88 合并两个有序数组。暴力可以把两个数组合并后排序，但没有利用有序条件。由于第一个数组尾部有空位，最优写法从后往前填充：比较两个数组当前最大值，把较大者放到最终位置。这样不会覆盖第一个数组中尚未比较的元素。若从前往后写，会破坏还没读取的值。循环结束后，如果第二个数组还有剩余，需要继续复制；第一个数组剩余则天然已经在正确位置。复杂度 $O(m+n)$ 。

53 最大子数组和。暴力枚举所有子数组并求和，直接做是 $O(n^3)$ ，用前缀和可降到 $O(n^2)$ 。进一步优化是 Kadane 算法：令 $current$ 表示必须以当前位置结尾的最大子数组和，那么它要么接在

前一个最大后缀后面，要么从当前元素重新开始。转移是 $\text{current} = \max(x, \text{current} + x)$ 。答案是所有 current 的最大值。易错点是全负数组，初始答案不能设成 0。

283 移动零。题目要求把零移动到末尾，同时保持非零元素相对顺序。暴力可以不断删除零再追加零，但移动成本高。双指针做法先把所有非零元素按原顺序写到前面，再把剩余位置填 0。也可以扫描时遇到非零就和慢指针交换，这样一次扫描完成。正确性不变量是：慢指针左侧都是已经稳定放好的非零元素。复杂度 $O(n)$ ，空间 $O(1)$ 。

344 反转字符串。这是双指针基本题。左右指针分别从两端向中间移动，每次交换字符。暴力新建字符串再倒序复制也正确，但题目通常要求原地修改。正确性来自每次交换后，两端各有一个位置已经放到最终位置。复杂度 $O(n)$ ，空间 $O(1)$ 。它看似简单，但能训练边界：空串、单字符、偶数长度、奇数长度都要自然通过。

3 无重复字符的最长子串。暴力枚举所有子串并检查是否重复，复杂度高。滑动窗口优化来自单调性：当右端扩展导致重复时，左端只能右移，不能左移。用哈希表或数组记录字符上次出现位置，扫描右端时把左边界更新到重复字符上次位置之后。窗口始终保持无重复。复杂度 $O(n)$ 。易错点是左边界只能取更大值，不能被窗口外的旧位置拉回去。

11 盛最多水的容器。暴力枚举两条线，复杂度 $O(n^2)$ 。双指针优化的依据是面积由较短边决定。左右指针形成当前宽度，若移动较高边，宽度变小且短板不变或更差，不可能得到更大面积；因此每次移动较短边，才有机会提高短板。这个排除论证是本题核心。复杂度 $O(n)$ 。不要把它和接雨水混淆：本题只选两条边，接雨水计算每个位置贡献。

15 三数之和。暴力三层循环是 $O(n^3)$ 。排序后固定第一个数，剩下两个数用双指针找和为相反数的组合。排序制造了单调性：和太小就左指针右移，和太大就右指针左移。去重要分两层：固定数相同要跳过；找到一组解后，左右指针也要跳过相同值。复杂度 $O(n^2)$ 。易错点是不能用哈希随便拼结果而不处理重复。

42 接雨水。暴力对每个位置分别向左向右找最高柱，复杂度 $O(n^2)$ 。前后缀最大数组能把每个位置的左右最高值预处理出来，时间 $O(n)$ 、空间 $O(n)$ 。双指针进一步维护 left_max 和 right_max ，每次结算较低的一侧，因为较低侧的水位已经由本侧最大值决定。也可用单调栈在遇到右边界时结算凹槽。面试中要说明每种解法的状态含义。

76 最小覆盖子串。暴力枚举所有子串再检查是否覆盖目标字符，复杂度很高。滑动窗口成立的原因是：右端扩展只会让覆盖条件更容易满足，左端收缩只会让窗口更短但可能破坏覆盖。用两个计数表维护需求和窗口内字符数量，并用 formed 记录已经满足的字符种类数。每当窗口满足覆盖，就尽量左移缩短。易错点是字符重复需求，例如目标串有两个 A 时，窗口中一个 A 不够。

209 长度最小的子数组。数组元素为正时是滑动窗口的关键。暴力枚举起点终点，复杂度 $O(n^2)$ 。因为元素为正，右端扩展会让和变大，左端收缩会让和变小，窗口具有单调性。扫描右端累加，当和达到目标时，不断左移并更新最短长度。若数组存在负数，这个方法会失效，需要前缀和加单调队列等更复杂方法。复杂度 $O(n)$ 。

438 找到字符串中所有字母异位词。暴力枚举每个长度为 p 的子串并排序比较，复杂度高。固定长度窗口可以维护字符频次，右进左出。若字符集是小写字母，用长度 26 的数组比哈希表更快。窗口长度达到 $p.\text{size}()$ 后检查频次数组是否相等。更优化的写法维护差异计数，避免每次比较 26 个位置，但面试中先写清楚版本更稳。复杂度 $O(n)$ 或 $O(26n)$ 。

49 字母异位词分组。暴力两两判断是否异位，复杂度高。优化是为每个字符串构造规范 key：排序后的字符串，或者 26 个字符计数组成的 key。异位词会得到相同 key，放入同一哈希桶。排序 key 的复杂度是每个词 $O(L \log L)$ ，计数 key 是 $O(L)$ 但实现稍繁。易错点是 key 必须无歧义，例如计数拼接要加分隔符，避免 1,11 和 11,1 混淆。

128 最长连续序列。排序后扫描连续段是 $O(n \log n)$ 。哈希优化是把所有数放入集合，只从“没有前驱”的数开始扩展连续段。若 $x - 1$ 存在，说明 x 不是序列起点，跳过它，避免重复扫描。每个数最多作为某个连续段的一部分被访问常数次数，因此平均 $O(n)$ 。易错点是重复元素，哈希集合天然去重。

202 快乐数。反复把数字替换为各位平方和，若最终到 1 则快乐。关键是检测循环。可以用哈希集合记录出现过的数字；如果再次出现同一个数字，就会无限循环。也可以用快慢指针，把转换函数看成链表的 next ，若存在环且环入口不是 1，就不是快乐数。暴力一直算不加检测会死循环。由于数字平方和会很快下降到有限范围，复杂度很小。

560 和为 K 的子数组。暴力枚举所有子数组并求和，用前缀和可到 $O(n^2)$ 。进一步优化是扫描

前缀和 `prefix`，如果之前出现过 `prefix - k`，那么这些位置到当前下标之间的子数组和为 `k`。哈希表保存历史前缀和出现次数。必须先查询再更新当前前缀，避免把空区间算进去。复杂度 $O(n)$ ，可以处理负数，这是它和滑动窗口的重要区别。

523 连续的子数组和。题目要求长度至少为 2 的子数组和是 `k` 的倍数。根据同余性质，若两个前缀和对 `k` 取模相同，它们之间的区间和能被 `k` 整除。哈希表保存某个余数第一次出现的位置，尽量保留最早位置以满足长度条件。初始化余数 0 的位置为 -1，表示从开头开始的区间。易错点是 `k` 的取值和长度至少为 2。

525 连续数组。把 0 看成 -1，1 看成 +1，问题变成找和为 0 的最长子数组。扫描前缀和，若同一个前缀和值再次出现，两次出现之间的区间和为 0，说明 0 和 1 数量相等。哈希表保存每个前缀和第一次出现的位置，因为要求最长区间。复杂度 $O(n)$ 。这题训练的是“把计数相等转换成前缀和相等”。

20 有效括号。栈保存尚未匹配的左括号。遇到左括号入栈，遇到右括号时检查栈顶是否是对应左括号。若栈空或类型不匹配，立即失败；扫描结束后栈必须为空。暴力反复删除相邻匹配括号也能做，但复杂度差且不自然。栈模型对应“最近打开的括号必须最先关闭”。易错点是只检查匹配过程却忘记最后栈非空。

155 最小栈。普通栈无法常数时间返回最小值，因为最小值可能埋在中间。同步最小栈保存每个深度下的历史最小值，压入元素时同时压入当前最小，弹出时同步弹出。也可以只在新值不大于当前最小时压入辅助栈，但弹出时要处理重复最小值。核心不变量是：辅助结构栈顶永远对应当前数据栈的最小值。所有操作 $O(1)$ 。

739 每日温度。暴力对每一天向后找第一个更高温度，复杂度 $O(n^2)$ 。单调栈保存还没找到更高温度的下标，且对应温度从栈底到栈顶单调递减。扫描到新温度时，它会解决所有比它低的栈顶日子，答案就是下标差。每个下标最多入栈出栈一次，复杂度 $O(n)$ 。易错点是栈里存下标，不是只存温度。

84 柱状图中最大的矩形。暴力枚举每根柱子作为最矮柱向左右扩展，复杂度高。单调栈维护递增高度下标；当遇到更矮柱子时，栈顶柱子的右边界确定，左边界是弹出后新的栈顶。为统一边界，可在左右加入高度 0 的哨兵。每根柱子弹出时计算一次以它为最矮高度的最大矩形。复杂度 $O(n)$ 。关键是解释宽度，不要只背代码。

215 数组中的第 K 个最大元素。完整排序是 $O(n \log n)$ 。维护大小为 K 的小顶堆是 $O(n \log k)$ ，堆顶就是当前第 K 大。快速选择平均 $O(n)$ ，通过 partition 每次只递归或迭代进入目标所在一侧。面试中最稳的是先讲堆，再补快速选择。若写快速选择，要随机化 pivot 或说明最坏复杂度 $O(n^2)$ 的风险。

347 前 K 个高频元素。先用哈希表统计频次。方案一是按频次完整排序，复杂度 $O(m \log m)$ 。方案二是大小为 K 的小顶堆，复杂度 $O(m \log k)$ 。方案三是桶排序，频次最大不超过 `n`，把元素按频次放桶后从高频桶向低频桶收集，复杂度 $O(n)$ 但空间更高。要根据 K 和数据规模比较方案。

295 数据流的中位数。如果每次插入后排序，插入成本高。双堆把数据分成较小一半和较大一半：大顶堆保存较小一半，小顶堆保存较大一半。保持两个堆大小差不超过 1，且左堆堆顶不大于右堆堆顶。中位数由堆顶得到。插入复杂度 $O(\log n)$ ，查询 $O(1)$ 。易错点是偶数个元素时求平均要防溢出。

35 搜索插入位置。本质是找第一个大于等于目标值的位置，也就是 `lower_bound`。二分不变量可以定义为答案在 `[left, right]` 或 `[left, right)` 中，选择一种写法坚持到底。若中点值小于目标，答案一定在右侧；否则答案在当前位置或左侧。复杂度 $O(\log n)$ 。不要在等于时提前返回后又忘记插入位置语义，使用 `lower_bound` 模型最统一。

34 在排序数组中查找元素的第一个和最后一个位置。分别找第一个大于等于 `target` 的位置和第一个大于 `target` 的位置，后者减一就是最后位置。这样比在找到一个 `target` 后向两边扩展更稳定，最坏仍为 $O(\log n)$ 。若第一个位置越界或值不是 `target`，说明不存在。关键是把“范围查找”拆成两个边界查找。

33 搜索旋转排序数组。数组由有序数组旋转而来，每次二分至少有一半是有序的。比较 `nums[left]` 和 `nums[mid]` 判断左半是否有序；若目标落在有序半边范围内，就收缩到那边，否则去另一边。暴力线性扫描是 $O(n)$ 。二分复杂度 $O(\log n)$ 。若存在重复元素，判断有序半边会变复杂，可能退化，需要额外处理。

162 寻找峰值。峰值是比相邻元素大的位置。二分依据是斜率：如果 `nums[mid] < nums[mid +`

1], 右侧一定存在峰值; 否则左侧含 `mid` 一定存在峰值。这个结论来自数组边界可视为负无穷, 以及上升段最终要么继续到边界形成峰, 要么下降形成峰。复杂度 $O(\log n)$ 。不要试图找全局最大, 题目只要任意峰值。

875 爱吃香蕉的珂珂。速度越快, 吃完所需时间越少, 存在单调性。二分速度 `speed`, 检查以该速度能否在 `h` 小时内吃完。检查函数中每堆耗时是向上取整, 可以写成 $(pile + speed - 1) / speed$ 。最小可行速度用左闭右闭或左闭右开二分都可以。复杂度 $O(n \log M)$ 。本题训练“答案空间二分”, 不是在数组下标上二分。

410 分割数组的最大值。给数组分成 `k` 段, 最小化各段和的最大值。暴力枚举切分位置复杂度 high, 也可用 DP。更常见是二分答案: 猜最大段和上限 `limit`, 贪心扫描数组, 尽量往当前段放, 超过上限就开新段, 统计所需段数。如果段数不超过 `k`, 说明上限可行, 可以尝试更小。下界是最大元素, 上界是总和。复杂度 $O(n \log S)$ 。

102 二叉树层序遍历。队列保存当前层待处理节点。每轮先记录队列大小, 这个大小就是当前层节点数; 弹出这些节点并把子节点加入队列。暴力递归按深度收集也可以, 但队列更直接体现层次。复杂度 $O(n)$ 。易错点是如果不记录层大小, 容易把下一层节点混进当前层结果。

104 二叉树最大深度。递归定义很自然: 树深度等于左右子树最大深度加一。但为了避免深树递归栈问题, 也可以用层序遍历迭代计算层数。面试中如果题目环境不限制树深, 递归可读性高; 工程中要意识到递归深度风险。复杂度 $O(n)$ 。这题核心是深度定义和空树返回 0。

98 验证二叉搜索树。不能只检查每个节点大于左孩子、小于右孩子, 因为子树中更深的节点也受祖先边界约束。方法一是中序遍历应严格递增; 方法二是迭代栈遍历时维护上一个访问值; 方法三是给每个节点传入合法上下界。若允许重复值, 比较条件要根据题目定义调整。复杂度 $O(n)$ 。

236 最近公共祖先。若是普通二叉树, 可以记录每个节点父指针, 再把一个节点的祖先放入集合, 向上查另一个节点; 也可以递归后序判断左右子树是否包含目标。若是二叉搜索树, 可利用值的大小关系从根向下走: 两个目标都小于当前节点就去左边, 都大于就去右边, 否则当前节点就是分叉点。复习时要区分普通树和搜索树条件。

200 岛屿数量。网格中的陆地相邻即连通块。暴力从每个陆地反复搜索会重复计数。正确做法是扫描网格, 遇到未访问陆地就启动 BFS/DFS, 把整座岛标记访问, 岛屿数加一。也可用并查集合并相邻陆地。复杂度 $O(mn)$ 。易错点是访问标记必须在入队时设置, 避免同一格子重复入队。

994 腐烂的橘子。多源 BFS。所有初始腐烂橘子同时作为第 0 分钟入队, 按层扩散到相邻新鲜橘子。最后若还有新鲜橘子未腐烂, 返回 -1; 否则返回最大层数。不要从每个腐烂橘子分别 BFS, 那会重复扩散且难以处理最短时间。复杂度 $O(mn)$ 。核心是“同时扩散”。

207 课程表。有向图判环。Kahn 拓扑排序用入度数组找当前没有前置依赖的课程, 处理后减少后继入度。若最终处理数量等于课程数, 则无环; 否则存在环。边方向要特别注意: `[a,b]` 表示学 `a` 前要学 `b`, 因此边是 `b` 指向 `a`。复杂度 $O(n + m)$ 。

210 课程表 II。在课程表判环基础上, 把出队顺序记录下来就是一种可行学习顺序。若最后记录长度小于课程数, 返回空数组。它和 207 的区别只是要输出拓扑序。若有多个入度为 0 的课程, 选择任意一个都可; 若题目要求字典序最小, 就把队列换成小顶堆。

127 单词接龙。单词是状态, 一次改变一个字符形成边, 边权相同, 所以 BFS 求最短转换长度。暴力建图两两比较单词复杂度 high; 通配模式如 `h*t` 可以把相邻候选聚到同一桶。若数据较大, 双向 BFS 从起点和终点同时扩展能显著减少访问状态数。易错点是起始单词不一定在字典中, 结束单词必须在字典中。

46 全排列。回溯构造路径, 每一层选择一个尚未使用的元素。状态包括当前路径和 `used` 数组。暴力本质就是枚举所有排列, 复杂度 $O(n!)$, 无法从渐进意义上优化, 只能通过剪枝或生成方式减少常数。回溯不变量是: 路径中元素互不重复, 深度等于已选择数量。到深度 `n` 时收集答案。

47 全排列 II。输入有重复元素, 需要去重。先排序, 让相同元素相邻。每层枚举候选时, 如果当前元素和前一个相同, 且前一个相同元素在本层还没有被使用, 则跳过当前元素, 避免同一层选择相同值造成重复排列。去重条件要和 `used` 数组配合, 不能简单跳过所有相等值, 否则会漏解。

78 子集。每个元素都有选或不选两种选择, 答案数是 $O(2^n)$ 。可以用回溯: 每进入一个状态就收集当前路径, 然后从当前位置之后继续选择元素。也可以用位掩码枚举所有集合。回溯写法更直观, 位掩码适合讲状态压缩。子集题不要求固定长度, 因此每个节点都是一个合法答案。

39 组合总和。元素可以重复使用, 组合不区分顺序。回溯时传入起始下标 `start`, 下一层仍可从当前下标开始, 因为可以重复选; 但不能回到更小下标, 否则会产生排列重复。剪枝条件是当前

和超过目标。若候选数组排序，还可以在超过目标时提前 break。核心是“允许重复使用，但保持非降选择顺序”。

22 括号生成。状态是已经放入的左括号数和右括号数。任何前缀中右括号数不能超过左括号数，左括号数不能超过 n 。暴力生成所有长度为 $2n$ 的括号串再检查会浪费大量非法前缀；回溯在构造过程中剪掉非法前缀。正确性来自所有合法括号串都满足这两个约束，所有满足约束且长度为 $2n$ 的串都合法。

79 单词搜索。网格回溯。状态包括当前位置、匹配到单词的下标和访问标记。每一步只能走上下左右，且同一格不能重复使用。暴力从每个格子作为起点尝试匹配；剪枝包括首字符不匹配直接跳过，越界、已访问、字符不等立即返回。复杂度最坏指数级，但实际由剪枝控制。易错点是回溯后恢复访问标记。

51 N 皇后。每行放一个皇后，递归或迭代枚举行。合法性由列、主对角线、副对角线三个集合判断。主对角线可用 $row - col$ 标识，副对角线用 $row + col$ 标识。暴力在棋盘任意格放皇后组合巨大；按行放置把每行一个皇后的约束提前固定。答案数本来可能很多，重点是剪枝和状态恢复。

70 爬楼梯。暴力递归会重复计算同一层数的方案数。状态 $dp[i]$ 表示到达第 i 阶的方法数，最后一步来自 $i-1$ 或 $i-2$ ，所以是斐波那契型转移。可以用两个变量滚动。复杂度 $O(n)$ ，空间 $O(1)$ 。这是理解“重叠子问题”的入门题。

198 打家劫舍。每间房可偷或不偷，相邻不能同时偷。状态 $dp[i]$ 表示考虑前 i 间房的最大收益。第 i 间不偷，收益是 $dp[i-1]$ ；偷，则收益是 $dp[i-2] + nums[i]$ 。取最大。可用两个变量滚动。若题目变成环形房屋，第一间和最后一间互斥，需要拆成不含首、不含尾两个线性问题。

322 零钱兑换。求最少硬币数，是完全背包最值问题。状态 $dp[amount]$ 表示凑出金额 $amount$ 的最少硬币数，初始化为无穷， $dp[0]=0$ 。对每个金额，尝试每种硬币，从 $dp[value - coin] + 1$ 转移。和零钱兑换 II 不同，这里求最少数量，不是组合数。若无法凑出，返回 -1。复杂度 $O(amount \cdot coins)$ 。

300 最长递增子序列。二次 DP 定义 $dp[i]$ 为以 i 结尾的 LIS 长度，枚举所有前驱，复杂度 $O(n^2)$ 。二分优化维护 $tails[len]$ ，表示长度为 $len+1$ 的递增子序列的最小可能结尾。扫描每个数，用 $lower_bound$ 替换第一个大于等于它的位置。tails 长度是答案，但 tails 本身不一定是一条真实子序列。

139 单词拆分。 $dp[i]$ 表示前 i 个字符能否被字典拼出。枚举最后一个单词起点 j ，若 $dp[j]$ 为真且 $s[j..i]$ 在字典中，则 $dp[i]$ 为真。暴力递归会反复处理同一个后缀。优化时可以限制最大单词长度，减少枚举起点；也要注意 `substr` 的拷贝成本。复杂度取决于字符串长度和字典查询成本。

1143 最长公共子序列。状态 $dp[i][j]$ 表示两个字符串前缀的 LCS 长度。若末尾字符相同，从左上角加一；否则取上方和左方最大值。它和最长公共子串不同，子序列允许不连续，因此不匹配时不是归零。复杂度 $O(nm)$ 。一维优化需要保存左上角旧值，初学阶段先写二维更可靠。

72 编辑距离。状态 $dp[i][j]$ 表示第一个单词前 i 个字符变成第二个单词前 j 个字符的最少操作。字符相同则继承左上角；不同则在删除、插入、替换三种操作中取最小。空串边界分别是插入或删除所有字符。暴力递归会重复求同一对前缀。复杂度 $O(nm)$ ，可用两行滚动数组优化空间。

19 删除链表倒数第 N 个节点。两遍扫描先求长度再删除是完全正确的基础解。快慢指针优化让快指针先走 n 步，再同步前进，保持距离不变量。虚拟头节点统一删除头节点和中间节点。循环结束时慢指针在待删节点前驱，修改它的 `next` 即可。测试要覆盖删除头、删除尾、单节点。

21 合并两个有序链表。维护结果链表尾指针，每次接入两个链表头部较小的节点。由于输入已经有序，局部较小者就是全局下一个节点。剩余链表可以整体接上。复杂度 $O(m+n)$ ，额外空间 $O(1)$ 。若题目要求不修改原链表，则需要创建新节点。

25 K 个一组翻转链表。难点是分组边界。每轮先从上一组尾部向后找第 K 个节点，不足 K 个则结束。保存下一组头节点后，在当前闭区间内反转指针，再把上一组尾接到新头，把新尾接到下一组。不要边数不足时也反转。画出上一组尾、当前组头、当前组尾、下一组头四个位置，代码就清楚。

239 滑动窗口最大值。暴力每个窗口扫描最大值，复杂度 $O(nk)$ 。单调队列保存可能成为最大值的下标，队列中的值递减。右端新元素进入时，弹出所有不大于它的尾部候选；队首若过期则弹出。窗口形成后队首就是最大值。每个下标进出队一次，复杂度 $O(n)$ 。必须存下标才能判断过期。

56 合并区间。按起点排序后线性扫描。当前区间若起点不超过结果最后区间的终点，则合并并更新终点；否则追加为新区间。排序让所有可能与当前结果尾部相交的区间连续出现。复杂度

$O(n \log n)$ 。先明确闭区间还是半开区间，决定等号是否成立。

75 颜色分类。三指针维护四段：已放好的 0、已放好的 1、未知区、已放好的 2。遇到 0 和低端交换并移动低端和扫描指针；遇到 1 只移动扫描指针；遇到 2 和高端交换并只移动高端，因为换回来的元素还没检查。复杂度 $O(n)$ ，空间 $O(1)$ 。

55 跳跃游戏。贪心维护当前能到达的最远位置。扫描到位置 i 时，如果 i 已经超过最远可达，说明无法到达；否则用 $i + \text{nums}[i]$ 更新最远位置。若最远位置覆盖最后下标，返回真。暴力从每个位置尝试所有跳法会指数爆炸。贪心正确性来自：只关心可达前缀内能扩展到的最远边界，不需要记录具体路径。

45 跳跃游戏 II。求最少跳数。扫描当前跳数能覆盖的区间，同时维护下一跳能到达的最远位置。当扫描到当前区间右端时，必须跳一次，并把区间扩展到下一跳最远位置。它相当于在隐式图上按层 BFS，但不显式建队列。复杂度 $O(n)$ 。易错点是不要在最后一个位置再额外增加跳数。

547 省份数量。城市直接连接关系形成无向图，省份就是连通分量。可以 BFS/DFS，也可以并查集。邻接矩阵只需扫描上三角，遇到连接就合并。最后统计根节点数量。复杂度 $O(n^2)$ 。并查集适合强调动态合并，BFS 适合直接遍历图，两者都合理。

684 冗余连接。树加一条边会形成唯一环。按输入顺序扫描边，用并查集维护已加入边形成的连通关系。如果一条边的两个端点已经连通，再加入它就会成环，这条边就是答案。暴力每加一条边就 DFS 检查连通性会重复扫描。并查集复杂度近似 $O(n)$ 。

743 网络延迟时间。有向正权图，从起点到所有点的最短路最大值。边权非负，适合 Dijkstra。邻接表建图，小顶堆保存当前距离最小的待处理状态；堆中旧状态用距离不等于当前最短距离来跳过。若最终有节点距离仍为无穷，返回 -1。复杂度 $O(m \log n)$ 。不能用普通 BFS，因为边权不相同。

1631 最小体力消耗路径。路径代价是经过边的最大高度差，而不是边权和。Dijkstra 仍可用，因为扩展路径时新代价是当前代价和新边代价的最大值，具有单调性。也可以二分体力上限，再 BFS 或并查集检查可达性，依据是上限越大越容易到达。面试中能说出两种模型会很加分。

416 分割等和子集。总和为奇数直接失败；否则转成能否选若干数凑出 $\text{sum}/2$ 。每个数只能用一次，是 0-1 背包可达性。滚动数组要倒序遍历容量，避免同一个数在一轮内被重复使用。复杂度 $O(n \cdot \text{target})$ ，这是伪多项式复杂度，和数值大小有关。

494 目标和。设正号集合和为 P ，负号集合和为 N ，总和 S ，有 $P - N = \text{target}$ 且 $P + N = S$ ，推出 $P = (S + \text{target})/2$ 。问题转成从数组中选若干数凑出 P 的方案数。若 $S + \text{target}$ 为奇数或目标绝对值超过总和，答案为 0。0 元素会让方案数翻倍，标准背包计数能自然处理。

518 零钱兑换 II。硬币无限使用，问组合数。外层遍历硬币，内层正序遍历金额，这样每种组合只会按固定硬币顺序被统计一次。若外层遍历金额、内层遍历硬币，会把不同顺序算作不同排列。状态 $\text{dp}[\text{value}]$ 表示凑出金额 value 的组合数， $\text{dp}[0] = 1$ 。复杂度 $O(\text{amount} \cdot \text{coins})$ 。

121 买卖股票的最佳时机。只允许一次买卖。暴力枚举买卖日是 $O(n^2)$ 。优化依据是：卖出日固定时，最佳买入日一定是此前最低价格。扫描价格，维护历史最低价和最大利润。复杂度 $O(n)$ 。若价格一直下降，最大利润应为 0，表示不交易。

309 最佳买卖股票时机含冷冻期。冷冻期让“只维护最低价”失效，因为今天能否买取决于昨天是否卖出。状态机更稳：**hold** 表示持有，**sold** 表示今天刚卖，**rest** 表示不持有且不处于刚卖状态。买入只能从 **rest** 来，卖出只能从 **hold** 来，休息可从 **rest** 或 **sold** 来。转移时先保存昨天状态，避免同日污染。

2 两数相加。链表按低位到高位保存数字，要求返回两数之和。暴力可以把链表转成整数再相加，但数字长度可能超过内置整数范围。正确模型是模拟竖式加法：两个指针分别扫描两条链表，维护进位，每一位创建一个新节点。循环条件是任一链表未结束或进位不为 0。易错点是最后进位、两链长度不同、不要使用可能溢出的整数。这个题训练的是链表遍历和状态变量，而不是数学大数库。

5 最长回文子串。暴力枚举所有子串并检查回文，复杂度可达 $O(n^3)$ 。中心扩展利用回文关于中心对称的性质：每个位置可以作为奇数长度中心，每两个相邻位置之间可以作为偶数长度中心，向两侧扩展并更新最长区间。复杂度 $O(n^2)$ ，空间 $O(1)$ 。DP 也可做，状态表示子串是否回文，但中心扩展更容易写。易错点是偶数长度回文，例如 **abba**。

31 下一个排列。题目要求原地得到字典序中的下一个排列。若整个序列从右到左非递减，即从左到右非递增，说明已经是最大排列，直接反转成最小排列。否则从右向左找到第一个 $\text{nums}[i] < \text{nums}[i+1]$ 的位置，再从右侧找到第一个大于它的元素交换，最后反转右侧后缀。优化依据是右侧后缀本来是降序，交换后反转能得到最小的更大后缀。它不是排序题，而是字典序结构题。

54 螺旋矩阵。直接模拟四个边界：上、下、左、右。每一轮依次走上边、右边、下边、左边，走完收缩对应边界。暴力用访问数组也能做，但四边界更简洁。易错点是单行、单列、最后一圈只剩一行或一列时不要重复加入元素。循环条件可以写成 `top <= bottom && left <= right`，每走完一条边都检查边界是否仍合法。

69 x 的平方根。求不超过平方根的整数。二分答案，范围从 0 到 x。判断 `mid * mid <= x` 时要避免整数溢出，可以改成 `mid <= x / mid`。若条件成立，记录 mid 并向右找更大值；否则向左。复杂度 $O(\log x)$ 。这个题训练的是二分答案和溢出意识，不是浮点计算。

136 只出现一次的数字。所有数字除一个外都出现两次，使用异或。异或满足交换律、结合律，且 `x xor x = 0`、`x xor 0 = x`，所以把所有数异或起来，成对数字抵消，剩下的就是答案。暴力哈希计数当然可做，但空间 $O(n)$ 。位运算解法空间 $O(1)$ 。如果题目改成其他数字出现三次，就要按位计数取模，不能继续简单异或。

146 LRU 缓存。需要 `get` 和 `put` 都是常数时间，并在容量满时淘汰最久未使用项。哈希表能常数定位 key，但不能维护新旧顺序；双向链表能常数移动节点和删除尾部，但不能常数按 key 查找。组合起来：哈希表保存 key 到链表节点迭代器，链表头表示最新使用，尾表示最久未使用。每次访问或更新都把节点移动到头部。核心是两个结构职责互补。

150 逆波兰表达式求值。后缀表达式中，操作符出现在操作数之后。扫描 token，遇到数字入栈，遇到运算符就弹出两个操作数计算后再压回。注意弹出顺序：先弹出的是右操作数，后弹出的是左操作数，减法和除法不能写反。复杂度 $O(n)$ 。它和中缀表达式求值的区别是没有括号和优先级解析压力，因为表达式顺序已经编码了计算依赖。

160 相交链表。交点是节点地址相同，不是值相同。哈希集合能做，双指针更省空间。两个指针分别走自己的链，走到空后切换到另一条链。若有交点，两个指针走过相同总长度后相遇；若无交点，同时为空。这个题要明确不能修改链表，否则可以临时连尾成环再检测，但那会引入副作用和恢复问题。

169 多数元素。如果某个元素出现次数超过一半，可以用 Boyer-Moore 投票。维护候选值和计数，遇到相同值计数加一，不同值计数减一，计数归零时更换候选。直觉是多数元素可以和其他元素两两抵消，最后剩下的候选一定是多数元素。若题目不保证多数元素存在，最后还要再扫描验证候选出现次数。复杂度 $O(n)$ ，空间 $O(1)$ 。

189 轮转数组。暴力每次右移一位，重复 k 次，复杂度 $O(nk)$ 。优化有三种：使用额外数组把元素放到 `(i+k) mod n`；使用环状替换；使用三次反转。三次反转最常见：先整体反转，再反转前 k 个，再反转后 n-k 个。注意 k 需要对 n 取模。这个题训练原地数组操作和下标边界。

206 反转链表。迭代写法维护 `previous`、`current`、`next`。每次先保存 `current->next`，再把当前节点指向前驱，然后两个指针前进。暴力用数组保存节点再重连也可以，但不如原地反转体现链表基本功。正确性不变量是：`previous` 指向已经反转好的前缀头，`current` 指向尚未处理的第一个节点。

217 存在重复元素。哈希集合扫描，若插入前已经存在当前值，返回 true。排序后检查相邻也可以，时间 $O(n \log n)$ ，空间可更低。哈希平均 $O(n)$ ，但需要额外空间。面试中要根据是否允许修改输入来选择：排序会改变顺序，若输入顺序后续有意义，哈希更稳。

238 除自身以外数组的乘积。暴力对每个位置乘其他所有元素，复杂度 $O(n^2)$ ；用总乘积再除以当前数会被 0 破坏，也可能不允许除法。前缀积和后缀积解决：答案先保存左侧所有元素乘积，再从右向左乘上右侧乘积。空间可以做到输出数组以外 $O(1)$ 。易错点是多个 0 和一个 0 的情况，前后缀写法自然处理。

240 搜索二维矩阵 II。每行升序、每列升序。从右上角开始，若当前值大于目标，当前列下面更大，列可以左移；若当前值小于目标，当前行左侧更小，行可以下移。每一步排除一行或一列，复杂度 $O(m+n)$ 。从左上角无法决定排除方向，因为右边和下边都更大。这个题训练利用二维单调性选择起点。

287 寻找重复数。数组长度 n+1，数值在 1 到 n，存在重复。不能修改数组且要求常数空间时，可以把数组看成函数图：下标指向 `nums[index]`，重复数就是环入口。使用快慢指针先相遇，再从起点和相遇点同步走找到入口。也可以二分值域，统计小于等于 mid 的数量，根据抽屉原理判断重复数在哪一侧。两种方法都不依赖排序。

394 字符串解码。形如 `3[a2[c]]` 的嵌套结构适合栈。扫描字符串，数字累积成重复次数；遇到左括号，把当前字符串和次数入栈，重置当前字符串；遇到右括号，弹出上一层，把当前字符串重

复指定次数后接回上一层。暴力递归也自然，但迭代栈更显式。易错点是多位数字和嵌套层次。

496 下一个更大元素 I。对第二个数组使用单调递减栈，扫描时如果当前元素大于栈顶，就说明当前元素是栈顶元素的下一个更大值，弹出并记录映射。最后第一个数组按映射查询答案。暴力对每个元素向右找更大值是 $O(n^2)$ 。单调栈把每个元素最多压入弹出一次，复杂度 $O(n)$ 。

503 下一个更大元素 II。环形数组使右侧可以绕回开头。做法是扫描两遍下标 $0..2n-1$ ，真实位置用 $i \bmod n$ 。栈仍保存还没找到下一个更大值的下标。为了避免重复入栈，通常只在第一遍把下标入栈，第二遍只负责帮助第一遍元素找到答案。复杂度 $O(n)$ 。

581 最短无序连续子数组。排序后比较原数组和排序数组，找到左右第一个不同位置是一种清晰解法，复杂度 $O(n \log n)$ 。线性做法从左到右维护历史最大值，若当前值小于历史最大值，说明它必须进入待排序区间，更新右边界；从右到左维护历史最小值，若当前值大于历史最小值，更新左边界。核心是找出破坏全局有序性的最远位置。

621 任务调度器。相同任务之间需要冷却时间。可以用大顶堆模拟每个时间片可执行的最高频任务，也可以用最高频公式计算最短长度。公式的骨架由最高频任务决定：前 $\text{maxFreq} - 1$ 轮每轮至少长度 $n+1$ ，最后放所有最高频任务的尾部。答案是任务总数和骨架长度的较大值。若其他任务足够填满空隙，就不需要空闲。

704 二分查找。最基础但最容易暴露边界混乱。推荐固定一种区间语义，例如左闭右开 $[\text{left}, \text{right})$ ：初始 $\text{left}=0, \text{right}=n$ ，循环条件 $\text{left}<\text{right}$ ，当 $\text{nums}[\text{mid}] < \text{target}$ 时 $\text{left}=\text{mid}+1$ ，否则 $\text{right}=\text{mid}$ 。最后检查 left 是否越界和值是否等于目标。统一语义比背多个模板更可靠。

763 划分字母区间。先记录每个字符最后出现位置。扫描字符串时，当前片段右边界是已见字符最后位置的最大值。当扫描下标到达右边界，说明当前片段中所有字符都不会出现在后面，可以切分。贪心正确性来自最早合法切分：一旦能切就切，可以最大化片段数量。复杂度 $O(n)$ 。

853 车队。每辆车到终点所需时间是 $(\text{target} - \text{position}) / \text{speed}$ 。按位置从靠近终点到远离终点排序扫描。后车如果到达时间小于等于前车车队时间，会追上并成为同一车队；否则形成新车队。这个题的关键是从右往左看，因为前车决定后车能否被阻挡。不要按速度排序，速度本身不能决定车队。

973 最接近原点的 K 个点。距离比较可用平方距离，避免开方。方案一完整排序，复杂度 $O(n \log n)$ ；方案二维护大小为 K 的大顶堆，堆顶是当前 K 个候选里最远的点，新点更近则替换，复杂度 $O(n \log k)$ ；方案三快速选择平均 $O(n)$ 。如果 K 很小，堆很实用；如果要原地平均线性，快速选择更强但实现风险更高。

981 基于时间的键值存储。每个 key 对应按时间递增的列表。插入时追加 $(\text{timestamp}, \text{value})$ ；查询某个时间戳时，在该 key 的列表中二分找最后一个时间不大于查询时间的记录。暴力线性向前找会超时。这个题本质是“每个 key 内部的有序数组 + upper_bound ”。易错点是不存在 key 或所有时间都大于查询时间时返回空。

1011 在 D 天内送达包裹的能力。船容量越大，需要天数越少，具有单调性。二分容量，下界是最大包裹重量，上界是总重量。检查函数按顺序装包，当前天放不下就开新一天，统计需要天数。若天数不超过 D，容量可行，尝试更小。它和分割数组最大值是同一个模型：最小化最大段和。

1046 最后一块石头的重量。每次取两块最重石头粉碎，适合大顶堆。暴力每轮排序复杂度 $O(n^2)$ ；堆能把每轮取最大和插入差值降到 $O(\log n)$ 。这个题不需要证明复杂贪心，因为规则本身指定每次取最重两块，堆只是高效实现规则。注意当两块相等时不需要放回 0。

1288 删除被覆盖区间。区间 $[a, b]$ 被 $[c, d]$ 覆盖，要求 $c \leq a$ 且 $b \leq d$ 。排序时按起点升序，起点相同按终点降序，这样大区间会在小区间前面。扫描维护当前最大右端点，若当前右端点不超过最大右端点，说明被覆盖；否则保留并更新最大右端点。起点相同按终点降序是关键，否则小区间先出现会误判。

148 排序链表。若要求 $O(n \log n)$ 时间和较低空间，链表归并排序是主线。快慢指针找中点并断开，再合并两个有序链表。递归版本清晰，但有栈深度；迭代自底向上版本更工程化。不要把值复制到数组排序再写回视为完全等价，因为真实节点可能携带更多数据或被外部引用。

152 乘积最大子数组。和最大子数组不同，负数会让最大值和最小值互换。扫描时同时维护以当前位置结尾的最大乘积和最小乘积；遇到负数时，旧最小乘以负数可能变成新最大。若当前元素单独开始更好，也要允许重新开始。复杂度 $O(n)$ 。易错点是 0 会截断乘积，状态转移自然处理它。

221 最大正方形。二维 DP。状态 $\text{dp}[i][j]$ 表示以当前位置为右下角的最大正方形边长。如果当前位置是 1，那么它取决于上方、左方、左上方三个状态的最小值加一；如果当前位置是 0，则

为 0。为什么取最小？因为一个更大的正方形需要三个方向都能支持。答案是最大边长平方。复杂度 $O(mn)$ 。

279 完全平方数。求和为 n 的最少完全平方数个数。可以看作完全背包最少物品数：物品是 $1, 4, 9, \dots$ ，容量是 n ，每种平方数可无限使用。也可以看成图最短路：每个数是节点，从 x 到 $x+\text{square}$ 有一条边，BFS 到 n 。DP 写法更直接，初始化无穷， $\text{dp}[0]=0$ ，对每个金额尝试平方数。复杂度约 $O(n\sqrt{n})$ 。

337 打家劫舍 III。树形 DP。每个节点有两种状态：偷当前节点时，左右孩子不能偷；不偷当前节点时，左右孩子可偷可不偷，取各自最大。若用递归，函数返回二元状态最清晰。工程上深树可能有递归风险，可用显式栈后序遍历实现。这个题的关键是不要只按层奇偶偷，因为树结构不一定平衡，局部最优需要节点状态。

416、494、518 背包三连复盘。416 是 0-1 背包可达性，容量倒序；494 是 0-1 背包方案数，容量也倒序，并且 0 会让方案翻倍；518 是完全背包组合数，硬币外层、金额正序。三题放在一起看，最重要的不是公式，而是“物品能用几次”和“统计组合还是排列”。循环顺序就是由这两个问题决定的。

股票题复盘。121 只允许一次交易，维护历史最低价；122 允许无限次交易，可以累加所有正收益差，也可以写持有和不持有状态；123 至多两次交易，需要交易次数维度；188 至多 k 次交易，是 123 的泛化；309 有冷冻期，买入来源受限制；714 有手续费，卖出或买入时扣费。统一方法是状态机：每天结束时持有什么状态，动作如何改变状态。

单调栈复盘。每日温度、下一个更大元素、柱状图最大矩形、接雨水都能用单调栈，但栈含义不同。每日温度和下一个更大元素中，栈保存等待被更大值解决的下标；柱状图中，栈保存递增高度，遇到更矮柱子时结算被弹出柱子的最大宽度；接雨水中，弹出的是低洼底部，左右边界由新栈顶和当前柱子提供。不要只说“用单调栈”，要说弹出时解决了什么问题。

单调队列复盘。滑动窗口最大值和受限子序列和都维护窗口内最优候选。队列保存下标，前者按数组值递减，后者按 DP 值递减。队首过期就弹出，队尾被新候选支配就弹出。每个下标最多进出一次，所以线性复杂度。单调队列适合“转移只需要最近 k 个状态的最大或最小值”的问题。

并查集复盘。并查集只回答连通性，不回答路径长度。冗余连接用合并失败找环；省份数量统计根节点；账户合并把邮箱作为节点；最小体力路径可以二分阈值后用并查集检查连通。并查集的两个优化是路径压缩和按大小或按秩合并。若题目需要维护比例、距离或奇偶性，就要扩展为带权并查集，不能直接套普通版本。

Dijkstra 复盘。使用条件是边权非负，目标是最短路径。堆中保存 $(\text{distance}, \text{node})$ ，因为 C++ 的 `priority_queue` 不支持 `decrease-key`，所以更新距离时压入新状态，弹出时跳过过期状态。网络延迟时间是普通最短路；最小体力路径把路径代价定义为最大边权，也满足单调扩展；若有负边，应该考虑 Bellman-Ford 或重新建模。

二分答案复盘。二分答案不是看数组是否有序，而是看答案是否具有单调可行性。爱吃香蕉、分割数组最大值、船运包裹、最小体力路径都属于这一类。套路是：定义答案范围，写检查函数，证明若某个答案可行，则更宽松答案也可行。最常见错误是检查函数里使用了不满足单调性的贪心，导致二分外壳正确但内部判断错误。

C++ 容器选择复盘。需要随机访问和连续内存，优先 `vector`；需要两端进出和窗口下标，考虑 `deque`；需要按 key 平均常数查询，使用 `unordered_map`，并注意 `reserve`；需要有序遍历、前驱后继、区间边界，使用 `map` 或 `set`；需要不断取最大最小候选，使用 `priority_queue`；需要节点稳定和常数移动，少数场景使用 `list`。容器不是背名字，而是匹配访问模式。

源码阅读复盘。看标准库源码时先问接口语义，再看核心数据成员，最后跟关键操作路径。`vector` 看大小、容量、扩容和迭代器失效；`deque` 看分段数组；`unordered_map` 看桶、负载因子和 `rehash`；`map` 看树结构和有序迭代；`priority_queue` 看底层容器和堆算法。刷题时能把 API 成本说清楚，代码选择就更可信。

Linux 源码阅读关联。Linux 内核常用侵入式链表、红黑树、基数树和 XArray，这些结构和 STL 容器的最大差异是所有权模型。STL 容器通常拥有元素或管理元素存储，内核结构常把链接节点嵌入业务对象中，数据结构只负责把已有对象挂到索引里。刷题不需要写侵入式结构，但理解这种设计能帮助你解释 LRU、调度队列、内存页缓存这类系统问题：结构服务于访问模式、生命周期和并发约束。

面试表达复盘。一道题的完整回答应包含六句话：第一，暴力方法是什么；第二，暴力慢在哪

里；第三，优化利用了哪个性质；第四，数据结构里保存什么不变量；第五，复杂度是多少；第六，边界用例有哪些。代码只是其中一部分。如果只能写代码但讲不清为什么，换一个题面就会失效。

如果一道题三次都需要看题解，不要继续刷新题，先回到它所属章节，重新写暴力到优化推理链。

术语表

暴力法 直接枚举解空间的方法，是理解题目和建立正确性的起点。

不变量 算法执行过程中始终成立的性质，用来证明优化解正确。

滑动窗口 用左右边界维护连续区间，并增量更新窗口状态的方法。

前缀和 把区间和查询转化为两个前缀差值的数据表示。

哈希表 通过哈希函数把 key 映射到桶，平均支持常数时间查询的数据结构。

BFS 广度优先搜索，适合无权最短路和层次扩展。

DFS 深度优先搜索，适合连通块、路径搜索和回溯。

拓扑排序 对有向无环图按依赖顺序排列节点的方法。

索引

algorithm, ii

data structure, ii

数据结构, ii

算法, ii